**Sharif University of Technology**

Department of Computer Engineering

# Static Timing Analysis and Logical-Effort–Guided Gate Sizing for VLSI Combinational Circuits

*Advanced VLSI Design — Course Project Report*

**Prepared by**

Arash Yadegari
404208734

Parsa Bashari
400104812

**Course Instructor**

Dr. Shaahin Hessabi

Spring 2026

# Abstract

Meeting a timing constraint is as much a requirement of a correct digital circuit as its logical function, yet timing correctness cannot be verified by functional simulation alone. This report documents the design, implementation, and evaluation of StaSizer, a software tool that performs static timing analysis (STA) on a flattened gate-level netlist and then improves an initial, possibly timing-violating, gate sizing using a logical-effort–guided optimizer. The tool ingests a netlist, a sized standard-cell library, and a configuration file; validates the netlist's structural correctness; builds a combinational directed acyclic graph (DAG); performs rise/fall–separated forward and backward timing propagation under an RC-based linear delay model with unate-aware timing arcs; extracts the top-$K$ least-slack critical paths; performs a logical-effort decomposition of each path ($g$, $h$, $b$, $G$, $H$, $B$, $F$, and the theoretical minimum path delay); estimates leakage and activity-driven dynamic power together with total area; and finally searches for a gate-sizing assignment that removes timing violations while respecting area and power budgets, using a cost function that trades off delay, power, area, and worst-negative-slack violation.

Four gate-sizing heuristics are implemented and compared head-to-head under identical stopping and acceptance rules: an exhaustive logical-effort-guided search, two logical-effort-informed ranking heuristics of differing aggressiveness, and a logical-effort-free random-greedy baseline. Beyond the scope required by the course assignment, the project additionally includes (i) a synthetic benchmark generation and evaluation framework that produces statistically meaningful sizing-repair problems with a certified best-known reference solution and evaluates every heuristic's decisions against an independent counterfactual oracle, and (ii) an interactive, browser-based topology viewer for visually inspecting a circuit's pre- and post-optimization state. A Monte Carlo statistical timing analysis models global and local manufacturing process variation and reports the resulting delay distribution, timing yield, and critical-path stability before and after optimization.

The tool is validated against the fifteen valid and six intentionally invalid benchmark circuits supplied with the assignment. Results show that logical-effort-guided sizing resolves a substantially larger fraction of timing violations than a logical-effort-free random-greedy baseline (13 of 14 originally-violating circuits versus 7 of 14), reaches a final design cost that is never worse and strictly lower on two-thirds of the suite, and does so with fewer total static-timing-analysis calls summed across the whole suite. The synthetic benchmark suite provides a statistically grounded confirmation of this result across a much larger and more diverse population of circuits than the fixed benchmark set alone could support.

**Keywords:** static timing analysis, logical effort, gate sizing, standard-cell library, VLSI CAD, timing closure, Monte Carlo statistical timing analysis, benchmark generation.

# Contents

# Chapter 1

# Introduction

## 1.1 Problem Statement

A digital VLSI circuit must satisfy two independent kinds of correctness. The first is *logical* correctness: for every legal input combination, the circuit must eventually settle to the specified output values. The second, and the subject of this report, is *timing* correctness: every signal transition must arrive at its destination early enough to be correctly captured, and no path in the circuit may take longer than the timing budget allotted to it. A circuit that is logically correct but functionally simulated only at a handful of input vectors can still fail in silicon because some rarely-exercised path violates its timing constraint; timing correctness is therefore not amenable to verification by input-vector simulation, since the number of paths through a circuit of even modest size is combinatorially large and no practical vector set can be guaranteed to sensitize the worst one.

*Static timing analysis* (STA) sidesteps this problem entirely. Rather than simulating logic values, STA treats the circuit as a directed acyclic graph (DAG) of timing arcs and propagates *worst-case* delay through that graph, independent of the actual logic values that would appear on any particular clock cycle. Every combinational path is implicitly covered by construction, because the propagation is a graph traversal over every arc, not an enumeration of input vectors. The output of STA is, for every node in the circuit, an *arrival time* (the latest moment a signal transition can occur at that node) and a *required time* (the latest moment it is permitted to occur without violating a downstream constraint); their difference is *slack*, and a negative slack anywhere in the circuit is a timing violation.

Once a violation is identified, one of the standard ways to close it without changing the circuit's logic is *gate sizing*: replacing a violating path's gates with larger-drive-strength variants from the same logic family in the standard-cell library. A larger gate has lower output resistance and therefore a lower propagation delay for the same load, at the cost of a larger input capacitance (which increases the load, and hence the delay, of whatever gate drives it), a larger area, and higher leakage and dynamic power. Because these effects compose across the whole circuit, sizing one gate can shift the critical path elsewhere or worsen a previously non-critical path, and a purely trial-and-error sizing process is both slow and difficult to reason about. *Logical effort* is the theoretical framework used in this project to guide that search: it decomposes a path's delay into a logic-topology-dependent term (how many stages, and how electrically "expensive" each stage's logic function is) and an electrical-loading term (how much capacitance each stage must drive relative to its own

size), which together identify *which* gate on a violating path is most under-sized for its load and therefore the best candidate for resizing.

## 1.2 Project Scope and Deliverables

The assignment specification requires a software tool that, given a gate-level netlist, a sized standard-cell library, and a configuration file, must:

1. parse the inputs and validate the netlist's structural correctness, reporting every detected error class distinctly;
2. build a combinational DAG and topologically order it;
3. compute output load capacitance and propagation delay per gate, separately for rising and falling output transitions;
4. perform forward (arrival-time) and backward (required-time) timing propagation, correctly handling positive-unate, negative-unate, and non-unate timing arcs;
5. report per-node slack, circuit-level worst negative slack (WNS) and total negative slack (TNS), and the top-$K$ least-slack critical paths;
6. perform a logical-effort decomposition of each critical path and use it to guide a search for an improved gate-sizing assignment, honoring area and power budgets;
7. estimate power and area before and after optimization;
8. compare the logical-effort-guided search against a baseline sizing method that does not use logical effort; and
9. (bonus) perform a Monte Carlo statistical timing analysis that models manufacturing process variation.

## 1.3 Contributions Beyond the Base Assignment

While the base requirements above were implemented in full, two additional components were developed that go beyond what the assignment strictly requires, and this report documents them at the same level of detail as the required components:

- **A synthetic benchmark generation and evaluation framework** (chapter 10), which programmatically constructs large numbers of controlled, repairable timing-violation scenarios — including a certified best-known reference solution obtained via multi-start beam search — and evaluates every sizing heuristic's decisions against an independent counterfactual oracle, reporting statistically rigorous metrics (Wilson confidence intervals on repair rate, decision regret, and stratified breakdowns by circuit size, depth, fan-out, and reconvergence). This addresses a limitation of evaluating sizing heuristics only on the fifteen fixed circuits supplied with the assignment:

fifteen circuits are too few to distinguish a genuinely better heuristic from one that simply got lucky on a particular benchmark.

- **An interactive, browser-based topology viewer** (chapter 11), which renders a circuit's structure with pan/zoom navigation, slack-based coloring, critical-path highlighting, and a toggle between the pre-optimization and post-optimization gate-sizing state of the same circuit.

These additions are clearly delineated from the required deliverables throughout this report so that grading against the assignment specification remains straightforward.

## 1.4 Report Roadmap

Chapter 2 reviews the theoretical background needed to read the rest of the report: the STA graph model, the RC delay model, logical effort, power/area estimation, and the statistical variation model used for Monte Carlo analysis. Chapter 3 describes the software architecture and the overall data flow, including the figure referenced throughout the rest of the report. Chapters 4 to 7 document, in the order they execute, the input pipeline, the STA engine, the logical-effort analysis, and the power/area estimator. Chapter 8 documents the cost function, the four sizing heuristics, and their acceptance/termination rules. Chapter 9 documents the statistical timing analysis. Chapters 10 and 11 document the two extensions described above. Chapter 12 presents results on both the fixed and synthetic benchmark suites and chapter 13 concludes.

# Chapter 2

# Background and Theory

This chapter reviews, without reference to any particular implementation, the theory that chapters 4 to 9 implement. Notation follows the assignment specification.

## 2.1 The Circuit as a Timing Graph

A flattened combinational netlist is modeled as a directed acyclic graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. Each vertex is either a primary input, a gate instance, or (implicitly, via a flag on the driving gate) a primary output; each directed edge represents a net connecting the output of one vertex to an input pin of another. Because the circuit is purely combinational, $\mathcal{G}$ must be acyclic — a directed cycle would correspond to a signal depending on its own future value, which is not realizable in combinational logic and must be rejected as a structural error (section 4.3.1).

Because $\mathcal{G}$ is a DAG, it admits at least one *topological order*: a linear ordering of vertices such that every edge points from an earlier vertex to a later one. Any algorithm that must process every gate only after all of its fan-in gates have already been processed — as the forward timing pass in section 5.3 does — can be implemented as a single linear pass over a topological order, in $O(|\mathcal{V}| + |\mathcal{E}|)$ time, without needing to re-discover dependency order on the fly.

## 2.2 Load and Delay Modeling

### 2.2.1 Output Load Capacitance

The propagation delay of a logic gate depends on the total capacitance it must drive at its output. For gate $i$, let $\mathrm{Fanout}(i)$ be the set of $(j, p)$ pairs such that gate $i$'s output drives input pin $p$ of gate $j$; let $C_{\mathrm{input},j,p}$ be the input pin capacitance of pin $p$ of the currently-assigned cell for gate $j$; and let $C_{\mathrm{external},i}$ be any additional load applied directly to gate $i$'s output net (nonzero only when that net is a primary output, in which case its value is supplied by the configuration file). The total output load capacitance of gate $i$ is

$$C_{\mathrm{load},i} = \sum_{(j,p) \in \mathrm{Fanout}(i)} C_{\mathrm{input},j,p} + C_{\mathrm{external},i}. \tag{2.1}$$

### 2.2.2 Propagation Delay

A linear delay model based on effective output resistance and load capacitance is used. Because the pull-up and pull-down networks of a static CMOS gate are generally not symmetric, rising and falling output transitions are modeled with independent parameters: an intrinsic (load-independent) delay $D_{\text{intrinsic,rise/fall},i}$ and an effective output resistance $R_{\text{rise/fall},i}$, both supplied per cell by the standard-cell library, together with a technology-dependent unit-conversion constant $\kappa$:

$$D_{\text{rise},i} = D_{\text{intrinsic,rise},i} + \kappa\, R_{\text{rise},i}\, C_{\text{load},i}, \tag{2.2}$$

$$D_{\text{fall},i} = D_{\text{intrinsic,fall},i} + \kappa\, R_{\text{fall},i}\, C_{\text{load},i}. \tag{2.3}$$

Both $D_{\text{rise},i}$ and $D_{\text{fall},i}$ depend only on the gate's own currently-assigned cell and its total output load (eq. (2.1)); neither depends on which input transition triggered the output change, nor on any arrival time. This decoupling is what allows load and delay to be computed for every gate in the circuit in a single pass, independent of, and prior to, the timing-propagation passes described next.

## 2.3 Unate Timing Arcs

A gate's output transition is not always the same polarity as the input transition that caused it; the relationship is called the pin's *timing sense* and is characterized per input pin by the standard-cell library as one of three classes.

- **Positive unate.** The output transition has the same polarity as the input transition: an input rise causes an output rise, and an input fall causes an output fall. Buffers and AND/OR-type gates (an even number of inversions from that input to the output) exhibit this behavior.
- **Negative unate.** The output transition has the opposite polarity: an input rise causes an output fall, and vice versa. Inverters and NAND/NOR-type gates (an odd number of inversions) exhibit this behavior.
- **Non-unate.** Either input polarity can, depending on the state of the gate's other inputs, cause either output polarity. XOR/XNOR-type gates exhibit this behavior, and both possibilities must be considered by the timing engine rather than assuming a fixed relationship.

This classification determines which (input transition, output transition) pairs are legal timing arcs through a given pin, and both the forward and backward propagation equations in section 2.4 must restrict their maximization/minimization to only the legal pairs for each pin.

## 2.4 Static Timing Analysis Equations

### 2.4.1 Forward Propagation — Arrival Time

Let $\mathrm{AT}_i^s$ denote the arrival time of transition $s \in \{\text{rise}, \text{fall}\}$ at the output of gate $i$. For each input pin $p$ of gate $i$, let $j = \text{driver}(i, p)$ be the gate (or primary input) driving that pin, and let $\mathcal{T}_{i,p}$ be the set of legal $(s', s)$ transition pairs for that pin, as determined by its timing sense (section 2.3). Because the propagation delay of gate $i$ (eqs. (2.2) and (2.3)) depends only on gate $i$'s own output transition $s$ and not on which input transition produced it, the arrival time at gate $i$'s output is

$$\mathrm{AT}_i^s = \max_{\substack{p \in \text{Inputs}(i) \\ j = \text{driver}(i,p) \\ (s', s) \in \mathcal{T}_{i,p}}} \left( \mathrm{AT}_j^{s'} + D_i^s \right), \qquad s \in \{\text{rise}, \text{fall}\}. \tag{2.4}$$

Primary inputs are the base case of this recursion: their arrival times are supplied directly by the configuration file rather than computed. Evaluating eq. (2.4) for every gate, in topological order, yields the arrival time of every net in the circuit in a single linear pass.

### 2.4.2 Backward Propagation — Required Time

Let $\mathrm{RT}_j^{s'}$ denote the required time of transition $s'$ at the output of gate $j$: the latest moment that transition may occur without causing a violation at any downstream primary output. The base case is now at the primary outputs, whose required times are supplied directly by the configuration file. For an internal gate $j$, let $\text{Fanout}(j)$ be the set of $(i, p)$ pairs such that $j$'s output drives pin $p$ of gate $i$. Because $j$'s output must be ready in time to satisfy *every* path that depends on it, the required time is the *minimum* over all of $j$'s fan-out arcs:

$$\mathrm{RT}_j^{s'} = \min_{\substack{(i,p) \in \text{Fanout}(j) \\ (s', s) \in \mathcal{T}_{i,p}}} \left( \mathrm{RT}_i^s - D_i^s \right), \qquad s' \in \{\text{rise}, \text{fall}\}. \tag{2.5}$$

Evaluating eq. (2.5) for every gate, in *reverse* topological order, yields the required time of every net in the circuit.

### 2.4.3 Slack, WNS, and TNS

The *slack* of transition $s$ at node $i$ is the margin between when the signal is required and when it actually arrives:

$$S_i^s = \mathrm{RT}_i^s - \mathrm{AT}_i^s, \qquad S_i = \min\left( S_i^{\text{rise}}, S_i^{\text{fall}} \right). \tag{2.6}$$

A negative slack indicates a timing violation at that node. Two circuit-level summary metrics are defined over the set $\mathcal{O}$ of primary outputs: the *worst negative slack* (WNS), the

single most severe violation anywhere in the circuit, and the *total negative slack* (TNS), the sum of every violation's magnitude (zero if the circuit is fully compliant):

$$\mathrm{WNS} = \min_{o \in \mathcal{O}} S_o, \qquad \mathrm{TNS} = \sum_{o \in \mathcal{O}} \min(0, S_o). \tag{2.7}$$

The *critical path* for a given endpoint is the sequence of gates whose delays, summed along the arcs selected by the arg max in eq. (2.4), produced that endpoint's arrival time; the top-$K$ critical paths are the $K$ (endpoint, transition) pairs with least slack, together with their reconstructed paths.

## 2.5 Logical Effort

Logical effort is a technology-independent method for estimating and minimizing the delay of a chain of static CMOS gates. It decomposes the normalized delay of a single stage $i$ into a *parasitic* term $p_i$ (the load-independent delay contributed by the gate's own internal capacitance) and a *stage effort* term $f_i$, the product of three unitless quantities:

- $g_i$, the **logical effort** of the input pin the path traverses: how much worse that gate is at driving a given load than a reference inverter of equivalent drive strength, due purely to its logic function (e.g. a two-input NAND has $g > 1$ because its series pull-down transistors must be sized larger for equal drive strength).
- $h_i$, the **electrical effort** (fan-out ratio): the ratio of the load capacitance the stage must drive, along the path, to its own input pin capacitance.
- $b_i$, the **branching effort**: the extra "tax" incurred when a stage's output also drives capacitance that lies off the path under analysis.

For a stage with on-path receiver capacitance $C_{\mathrm{onpath},i}$ and off-path (other fan-out) capacitance $C_{\mathrm{offpath},i}$, and own input pin capacitance $C_{\mathrm{in},i}$:

$$b_i = \frac{C_{\mathrm{onpath},i} + C_{\mathrm{offpath},i}}{C_{\mathrm{onpath},i}}, \qquad h_i = \frac{C_{\mathrm{onpath},i}}{C_{\mathrm{in},i}}, \qquad f_i = g_i\, b_i\, h_i, \qquad d_i = f_i + p_i. \tag{2.8}$$

For an $N$-stage path with path logical effort $G$, path branching effort $B$, path electrical effort $H$ (ratio of the path's terminal load to its first stage's input capacitance), total path effort $F$, and total parasitic delay $P$:

$$G = \prod_{i=1}^{N} g_i, \qquad B = \prod_{i=1}^{N} b_i, \qquad H = \frac{C_{L,\mathrm{path}}}{C_{\mathrm{in},1}}, \qquad F = G\,B\,H = \prod_{i=1}^{N} f_i, \qquad P = \sum_{i=1}^{N} p_i. \tag{2.9}$$

A classical result of logical effort theory is that, for a fixed number of stages $N$, path delay is minimized when every stage carries *equal* effort $\hat{f} = F^{1/N}$; the corresponding

normalized and absolute theoretical minimum delays are

$$d_{\text{path,min}} = NF^{1/N} + P, \qquad D_{LE,\text{min}} = \tau\, d_{\text{path,min}}, \tag{2.10}$$

where $\tau$ is a technology-dependent time unit supplied by the cell library. A stage whose actual effort $f_i$ exceeds $\hat{f}$ is, by this theory, under-sized relative to the load it drives and is a natural candidate for up-sizing; chapter 8 uses exactly this observation to prioritize which gate to resize first. Inverting eq. (2.8) for a target effort $\hat{f}$ gives, for stage $i$, the continuous *desired* input capacitance that would make its effort equal to the path target, which can then be mapped onto the nearest discretely-sized cell(s) actually available in the library:

$$C^{*}_{\text{in},i} = \frac{g_i\, C_{\text{total},i}}{\hat{f}}, \qquad C_{\text{total},i} = C_{\text{onpath},i} + C_{\text{offpath},i}. \tag{2.11}$$

## 2.6 Power and Area Estimation

Total circuit area is the sum of the area of every currently-assigned cell. Total power is the sum of a static *leakage* component (supplied directly per cell by the library, independent of switching activity) and a switching-activity-dependent *dynamic* component. For gate $i$ with switching activity factor $\alpha_i$, effective switched capacitance $C_{\text{sw},i}$ (the sum of its output load and its own internal capacitance), supply voltage $V_{DD}$, and clock frequency $f$:

$$A_{\text{total}} = \sum_{i \in \mathcal{G}} A_i, \qquad C_{\text{sw},i} = C_{\text{load},i} + C_{\text{internal},i}, \qquad P_{\text{dynamic}} = \sum_{i \in \mathcal{G}} \alpha_i\, C_{\text{sw},i}\, V_{DD}^2\, f, \tag{2.12}$$

$$P_{\text{leakage}} = \sum_{i \in \mathcal{G}} P_{\text{leakage},i}, \qquad P_{\text{total}} = P_{\text{dynamic}} + P_{\text{leakage}}. \tag{2.13}$$

Short-circuit power is not modeled separately; the assignment specification explicitly permits this simplification.

## 2.7 Statistical Timing: Process Variation

Manufacturing process variation causes the true delay of a fabricated gate to differ from its nominal (library-specified) value. Two components are modeled: a *global* component $Z_g$, shared by every gate in a given manufactured die (modeling die-to-die and within-die systematic variation), and a *local* component $Z_i$, drawn independently per gate (modeling random device-level mismatch). For Monte Carlo sample $k$, with relative variation strengths $\sigma_g$ and $\sigma_l$ supplied by the configuration file:

$$Z_g^{(k)}, Z_i^{(k)} \sim \mathcal{N}(0,1), \qquad D_{i,s}^{(k)} = D_{i,s,\text{nom}} \left(1 + \sigma_g Z_g^{(k)} + \sigma_l Z_i^{(k)}\right), \quad s \in \{\text{rise}, \text{fall}\}. \tag{2.14}$$

Because delay is a strictly positive physical quantity, any sample that would yield a non-positive delay for any gate is discarded and re-drawn. Given $M$ valid samples, and letting $I^{(k)}_{\text{violation}}$ indicate whether sample $k$'s worst negative slack is negative, the empirical violation probability and timing yield are

$$\hat{P}_{\text{violation}} = \frac{1}{M}\sum_{k=1}^{M} I^{(k)}_{\text{violation}}, \qquad \hat{Y}_{\text{timing}} = 1 - \hat{P}_{\text{violation}}. \tag{2.15}$$

Full STA (section 2.4) is re-run once per Monte Carlo sample against the perturbed delay set to obtain that sample's WNS, TNS, circuit delay, and critical path, from which the distributional statistics reported in chapters 9 and 12 are computed.

# Chapter 3

# System Architecture

## 3.1 Design Principles

Three principles shape the implementation described in the remaining chapters.

1. **Immutability of the circuit model.** A `Circuit` object, once constructed from a netlist and a specific assignment of cell sizes, is never mutated. Its topological order, per-gate load capacitance, per-gate delay, total area, and total power are computed once, eagerly, at construction time. Resizing a gate does not modify an existing `Circuit`; it produces an entirely new `Circuit` instance with one gate's cell replaced and every net reference correctly rebuilt. This trades additional memory allocation during optimization (chapter 8, where many candidate resizes are evaluated per iteration) for the elimination of an entire class of bugs in which a trial mutation is incompletely reverted and silently corrupts subsequent analysis.

2. **Separation of static circuit state from one analysis run's results.** A `Circuit` describes *what the circuit currently is*: its topology and its currently-assigned sizes. It does not store arrival times, required times, or slack. Those are the output of a separate, pure `analyze_timing` function that takes a `Circuit` and a configuration as input and returns a timing-result value. This means the same `Circuit` can be analyzed multiple times (for example, once per Monte Carlo sample, or once per stage in a diagnostic sweep) without the results of one analysis leaking into another.

3. **Pure, side-effect-free analysis modules.** STA, logical effort, power/area estimation, and Monte Carlo analysis are each implemented as functions that take immutable inputs and return a new result value, rather than as methods that accumulate state on an object across calls. This makes each module independently testable and makes the overall data flow (section 3.4) easy to reason about: every stage's inputs are exactly the outputs of the stages before it.

## 3.2 Module Map

The implementation is organized into eight top-level packages, listed here in the order a reader unfamiliar with the codebase should study them — each package depends only on the ones listed above it.

| Package | Responsibility |
|---|---|
| `domain/` | Owns the `Cell`, `Gate`, `Net`, and `Circuit` data model, and shared numeric helpers. Every other package depends on this one; it depends on nothing else in the project. |
| `input/` | Parses and validates the netlist, standard-cell library, and configuration file, and builds the initial `Circuit` (chapter 4). |
| `analysis/` | Static timing analysis, logical-effort path analysis, and Monte Carlo statistical analysis (chapters 5, 6 and 9). |
| `optimization/` | The cost function, the four gate-sizing heuristics, and the optimizer driver loop (chapter 8). |
| `reporting/` | Serializes every analysis result to the on-disk output contract (CSV, JSON, and PNG artifacts), including the topology export consumed by the interactive viewer. |
| `app/` | Coordinates a full single-circuit run end to end and exposes the unified command-line interface. |
| `benchmarking/` | The synthetic benchmark generation and evaluation framework (chapter 10). |
| `viewer/` | The interactive topology viewer's local server and static front-end assets (chapter 11). |

## 3.3 Key Classes and Public API

Before tracing the data flow (section 3.4), it is worth naming the concrete classes and functions each phase is built from, since the remainder of this report refers to them by name rather than only describing their behavior in prose. Table 3.2 lists the core data model (`domain/`); the analysis, optimization, and reporting entry points are listed at the start of their respective chapters (chapters 5, 6, 8 and 9).

| Class / function | Role |
|---|---|
| `Cell`, `InputPin`, `RiseFall` | Frozen dataclasses holding one library cell's characterization data: `name`, `family`, `size`, `size_factor`, `num_inputs`, `input_pins`, `timing_sense`, `intrinsic_delay`, `output_resistance`, `internal_capacitance`, `parasitic_delay`, `leakage_power`, `area`. Constructed and validated by `Cell.from_dict(name, data)`. |

| Class / function | Role |
|---|---|
| `CellLibrary(Mapping[str, Cell])` | Parses `cell_library.json` in `__init__(path)` and exposes cells as a read-only mapping. Key methods: `find(name)`, `variants(family)`, and `sizing_candidates(family,input_pin,target_capacitance)` (section 6.3). |
| `Net`, `Gate` | Frozen dataclasses describing structural connectivity, independent of which cell size is assigned: `Net` holds `driver` and `loads: tuple[(pin, gate_name), ...]`; `Gate` holds `cell_family`, `inputs`, `output`. |
| `PathStep`, `CircuitPath` | `PathStep(gate, input_pin)` is one gate traversal through a specific pin; `CircuitPath` is an ordered tuple of steps between a primary input and a primary output, exposing `.gates` as a derived property. |
| `NetListParser` | `__init__(netlist_path, cell_library)`; `parse()` returns `(nets, gates, cell_assignment)` after performing every structural check of table 4.1. |
| `CircuitTopology` | Frozen, cell-assignment-independent structure (`netlist`, `gates`, `topological_order`), built once by `CircuitTopology.create(netlist, gates)` and shared by every sizing variant of the same circuit. |
| `Circuit` | One immutable cell-sizing assignment over a shared topology. `__init__(netlist, gates, gate_cells, config, cell_library)` eagerly computes area, power, fan-out capacitances, and gate delays. Query methods: `cell_for(gate)`, `input_net(gate,pin)`, `output_net(gate)`. `with_gate_cell(gate_name, replacement)` returns a new `Circuit` with one gate resized, reusing the shared topology and updating only the affected metrics incrementally (section 8.2). |
| `replace_gate_cell(circuit, gate_name, replacement)` | Module-level convenience wrapper around `Circuit.with_gate_cell`. |
| `replacement_area_and_power(circuit, gate_name, replacement)` | Returns the *delta*-updated $(A_{\text{total}}, P_{\text{total}})$ a candidate resize would produce *without* constructing a full `Circuit` variant — used by the optimizer as a cheap pre-filter before paying for a full STA re-run. |

| Class / function | Role |
|---|---|
| `Config` | `__init__(path)` validates `config.json` in full (section 4.2) and exposes typed sub-configurations (`TimingAnalysisConfig`, `OperatingConditions`, `DesignConstraints`, `OptimizationConfig`, `MonteCarloConfig`) plus per-net accessor methods `output_load(net)`, `input_arrival(net)`, `output_required(net)`, `activity_factor(node)`. |

Table 3.2: Core data-model classes in `domain/` and `input/`, used throughout the rest of this report.

## 3.4 End-to-End Data Flow

Figure 3.1 shows the complete pipeline for a single circuit, from its two input files through to the reported outputs, annotated with the governing equation at each stage. The pipeline has five clearly separated phases: (1) parsing and structural validation (`NetListParser.parse`, `Config.__init__`), which either rejects the circuit outright or produces a validated `Circuit`; (2) static timing analysis (`analyze_timing`, chapter 5), itself split into a forward pass (arrival time), a backward pass (required time), and their combination into slack, WNS/TNS, and the top-$K$ critical paths; (3) logical-effort analysis (`analyze_path_logical_effort`, chapter 6) of those critical paths; (4) the optimization loop (`CircuitOptimizer.optimize`), which repeatedly proposes a gate resize, evaluates it with a full re-run of phase (2), and accepts or rejects it according to the cost function and termination rules of chapter 8; and (5) Monte Carlo statistical analysis (`run_monte_carlo`) and report generation. Phases (1)–(5) are driven end to end by `Experiments.run(circuit)` (`app/experiments.py`), which dispatches to one handler function per named experiment in the fixed order given by `DEFAULT_EXPERIMENTS`: `fanout_capacitances`, `gate_delays`, `timing_analysis`, `logical_effort_analysis`, `optimization`, `topology_export`, `visualization`, `monte_carlo`, and `summary` — an explicit, literal ordering that doubles as the most authoritative statement of the pipeline's dependency order.

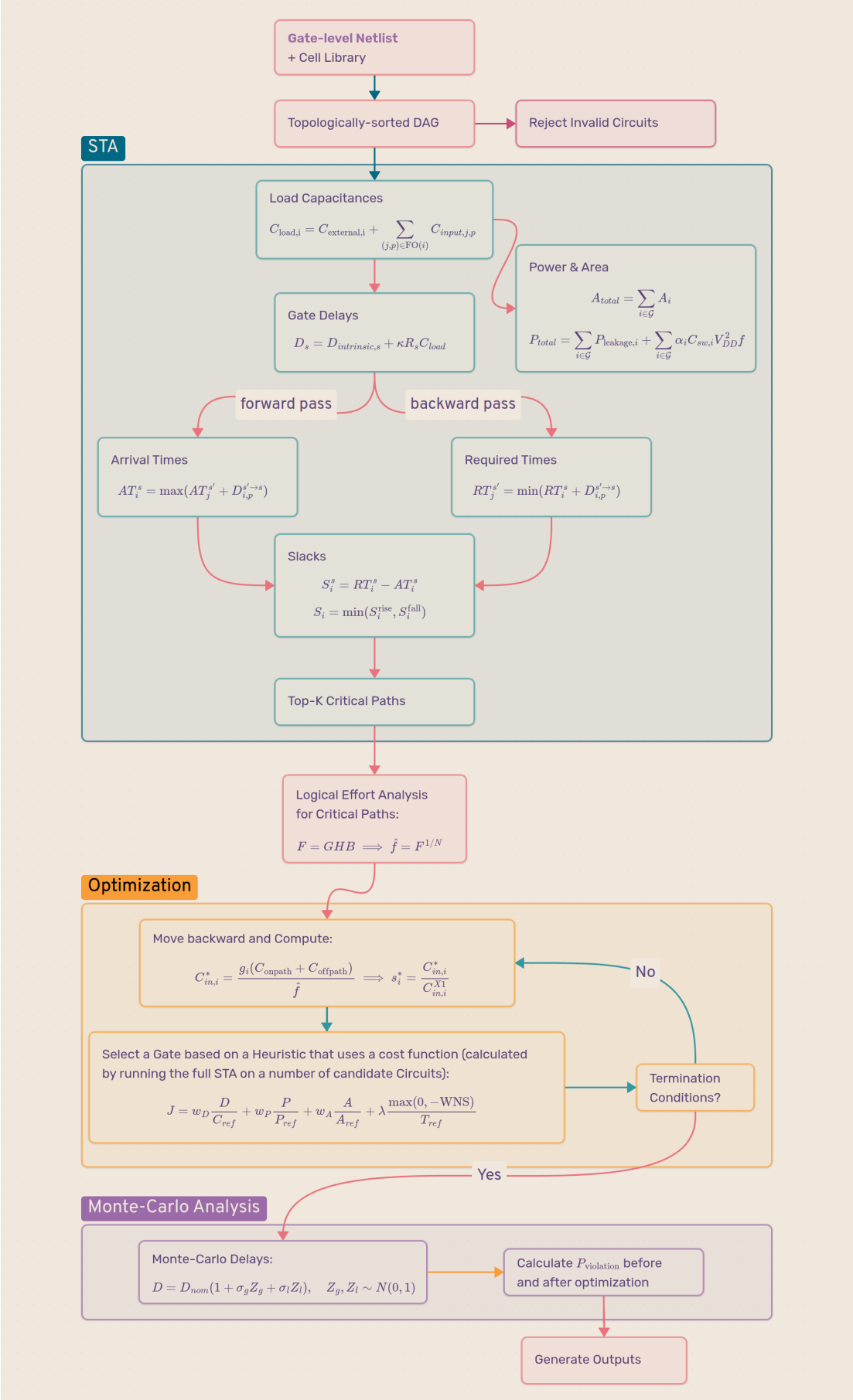



Figure 3.1: End-to-end data flow of the analyzer, from input files to reported outputs.

## 3.5 Command-Line Interface and Output Contract

The command-line entry point is `app/cli.py:main(argv)`, which builds its parser via `build_argument_parser()` and delegates to `app/application.py:run_application(argv)`. That function performs, in order: `setup_logging(verbose, log_path)`; input loading via `load_circuit(arguments)` (which internally parses the configuration, cell library, and netlist, and constructs the initial `Circuit`); and, if loading succeeds, `run_experiments(circuit, netlist_path, output_dir)`, which constructs an `Experiments` instance (section 3.4) and calls its `run(circuit)` method. A single circuit is analyzed with:

```
./run_circuit.sh c01_inverter_chain
```

which writes an independent, self-contained directory `outputs/c01_inverter_chain/` containing every artifact listed in table 3.3. Invalid inputs produce only `validation_report.json` and `run.log`, and the process exits with a nonzero status without attempting any timing analysis, so that a downstream script (or the batch mode, `run_circuit.sh -all`) can mechanically distinguish an analyzed circuit from a rejected one.

| Artifact | Contents |
|---|---|
| `validation_report.json`, `run.log` | Structured input-validation result and the complete execution log. |
| `fanout_capacitances.csv`, `gate_delays.csv` | One row per gate, topologically ordered: output load and rise/fall delay. |
| `timing_analysis.csv` | One row per gate: load, rise/fall delay, arrival time, required time, transition slack, and overall node slack. |
| `timing_summary.csv`, `critical_paths.csv` | Circuit delay, WNS, TNS, compliance flag, and the top-$K$ critical paths. |
| `logical_effort_paths/stages/candidates.csv` | Path-level $(G, B, H, F, P)$, stage-level, and discrete-candidate logical effort records. |
| `optimization_*.csv`, `optimization_comparison.csv` | Complete iteration histories for every heuristic and their head-to-head comparison. |
| `circuit_graph_{pre,post}_optimization.png` | Slack-colored circuit diagrams with the critical path highlighted. |
| `circuit_topology.json` | Versioned graph contract consumed by the interactive viewer (chapter 11). |
| `monte_carlo_*.csv`, `monte_carlo_summary.json` | Paired pre-/post-optimization statistical timing results (chapter 9), or an explicit "skipped" summary when disabled. |
| `summary.json` | Aggregate summary for the circuit. |

Table 3.3: Output contract produced for every successfully analyzed circuit.

All CSV column names carry explicit units (`_ns`, `_fF`, `_uW`); all numeric computation and serialization retain full floating-point precision, with rounding applied only when a value is formatted for a plot label, so that downstream tooling never operates on already-rounded intermediate values.

# Chapter 4

# Input Parsing and Validation

## 4.1 Netlist and Cell Library Grammar

The netlist format is a flat, line-oriented text grammar with three statement forms, one per line, blank lines and #-prefixed comments ignored:

```
INPUT  <signal_name>
OUTPUT <signal_name>
<instance_name> <cell_name> <input_net_1> ... <input_net_N> <output_net>
```

The standard-cell library is a single JSON file, shared across every circuit, describing thirty-six cells across twelve logic families (`INV`, `BUF`, `AND2`, `NAND2`, `OR2`, `NOR2`, `XOR2`, `XNOR2`, three-input variants, and so on), each available in three drive-strength sizes (`X1`, `X2`, `X4`). Every cell entry supplies, per input pin, its capacitance, logical effort, and timing sense (section 2.3), and, per cell, its intrinsic rise/fall delay, rise/fall output resistance, internal capacitance, normalized parasitic delay, leakage power, and area — i.e. every quantity that appears on the right-hand side of eqs. (2.1) to (2.3), (2.8), (2.12) and (2.13).

## 4.2 Configuration Schema

The configuration file supplies every circuit- and run-specific parameter that is not part of the netlist or the (shared) cell library: primary-input arrival times, primary-output required times and external loads, operating conditions (supply voltage, clock frequency, temperature, switching activity factors), area and power budgets, optimizer settings (allowed sizes, iteration limit, cost weights), and Monte Carlo settings (sample count, variation strengths, random seed). Every field is validated for presence, type, and range before any analysis is attempted; a missing or malformed field is rejected with a specific error message rather than silently substituted with a default, on the reasoning that a silently wrong configuration value is more dangerous than a loud rejection.

## 4.3 Structural Validation Stages

Validation proceeds through four ordered stages, and the process is aborted at the first stage that fails so that no analysis is ever attempted on an input the tool cannot trust.

1. **Configuration.** Schema and range validation as described above.

2. **Cell library.** Schema validation of the shared library file (every cell has a complete, correctly typed set of the fields listed in section 4.1).
3. **Netlist.** Parses the grammar and performs every netlist-level structural check listed in table 4.1.
4. **DAG.** Builds the circuit graph and topologically sorts it; failure at this stage means a combinational (feedback) loop was detected.

### 4.3.1 Combinational Loop Detection

A combinational loop cannot be resolved by a linear topological ordering: Kahn's algorithm repeatedly removes vertices with zero remaining in-degree and terminates having placed every vertex if and only if the graph is acyclic. If any vertices remain unplaced when the algorithm's queue empties, those vertices participate in (or exclusively depend on) a cycle, and the netlist is rejected with the specific set of implicated gate instances reported to the user.

## 4.4 Error Taxonomy and the Invalid-Circuit Benchmark Set

Six structural error classes are validated against, matching the six invalid benchmark circuits supplied with the assignment (`e01`–`e06`). Table 4.1 summarizes each class, the pipeline stage at which it is detected, and the diagnostic message format produced.

| Error class | Stage | Detection |
|---|---|---|
| `undefined_cell` | Netlist | A gate instance references a cell name absent from the library. |
| `undriven_signal` | Netlist | A gate input net is neither a primary input nor the output of any gate in the netlist. |
| `multiple_drivers` | Netlist | A net name appears as the output of more than one gate instance (or of both a gate and a declared primary input). |
| `invalid_input_count` | Netlist | The number of input nets listed for a gate instance does not match the defined cell's number of input pins. |
| `undriven_output` | Netlist | A signal declared with `OUTPUT` is not the output of any gate instance. |
| `combinational_loop` | DAG | Topological sort fails to place every gate (section 4.3.1). |

Table 4.1: Structural error taxonomy.

# Chapter 5

# Static Timing Analysis Engine

This chapter documents the implementation of the theory reviewed in section 2.4, as it appears in `analysis/sta.py`. The module's only two public entry points are

```
def analyze_timing(circuit: Circuit,
                   gate_delays: Mapping[str, RiseFall] | None = None
                   ) -> TimingAnalysisResult: ...

def analyze_timing_metrics(circuit: Circuit,
                   gate_delays: Mapping[str, RiseFall] | None = None
                   ) -> TimingMetrics: ...
```

`analyze_timing` is the full analysis used everywhere in this report (chapter 12) and returns a `TimingAnalysisResult` — a frozen dataclass carrying `critical_paths`, `wns`, `tns`, `circuit_delay`, and the full `arrival_times` / `required_times` / `transition_slacks` tables (each a mapping from net name to a `RiseFall` pair), plus a `node_slack(net_name)` convenience method. `analyze_timing_metrics` is a deliberately cheaper sibling, used by the optimizer's inner evaluation loop (section 8.2) whenever the full critical-path list is not needed for a particular candidate check. Its speedup is not merely skipping the reporting of unneeded data: since WNS/TNS (eq. (2.7)) only ever need each primary output's *own* configured required time (`circuit.config.output_required(net_name)`) compared against that output's forward-computed arrival time, `analyze_timing_metrics` never performs the full backward propagation of eq. (2.5) through the circuit's internal nodes at all — it calls only the lean arrival-time pipeline (`_compute_arrival_values` / `_maximum_gate_arrival`, section 5.3), which does not even retain the tie-tracking predecessor information `analyze_timing` needs for path reconstruction, and computes each output's slack directly against its own configured requirement.

Both public functions accept an optional `gate_delays` override, which the Monte Carlo module (chapter 9) uses to inject a sample's perturbed delay set without needing a variant analysis path of its own. Internally, both are thin wrappers around a shared private pipeline in `analysis/sta.py`, whose exact signatures are reproduced below since they pin down what state flows between the forward pass, the backward pass, and path extraction:

```
def _run_sta(circuit: Circuit, gate_delays: Mapping[str, RiseFall]
             ) -> tuple[dict[str, RiseFall],    # arrival_times
                        dict[str, RiseFall],    # required_times
                        PredecessorTable,       # controlling arcs
                        dict[str, RiseFall]]:   # transition_slacks
```

```
def _compute_arrival_times(circuit, gate_delays
    ) -> tuple[dict[str, RiseFall], PredecessorTable]: ...   # Sec. 5.3
def _compute_required_times(circuit, gate_delays) -> dict[str, RiseFall
    ]: ...     # Sec. 5.4
def _compute_transition_slacks(arrival_times, required_times
        ) -> dict[str, RiseFall]: ...                    # Sec. 5.5
def _find_critical_paths(circuit, predecessors, transition_slacks
        ) -> list[CriticalPath]: ...                     # Sec. 5.6
def _trace_paths_backward(circuit, current_net, current_transition,
        output_net, reversed_steps, reversed_transitions, endpoint_slack
            , found_paths, predecessors) -> None: ...    # Sec. 5.6
```

with `_compute_circuit_delay` and `_compute_output_slack_metrics` deriving `circuit_delay`, `wns`, and `tns` from the completed arrival/slack tables, and `_timing_result` assembling the final `TimingAnalysisResult`. The type `PredecessorTable = Mapping[tuple[str, Transition], tuple[PredecessorArc, ...]]` is itself worth noting: it is keyed by *every* (net, transition) pair to a *tuple* of controlling predecessor arcs, not a single one — this is precisely the data structure that makes exhaustive tie recording (section 5.3) possible, since `_compute_arrival_times` records every arc attaining the maximum in eq. (2.4), and `_trace_paths_backward` later branches over exactly that stored tuple rather than recomputing or re-deciding predecessor selection from scratch.

The engine executes in five ordered steps for a given, fixed gate-sizing assignment: (1) load and delay computation (section 5.2), performed eagerly by `Circuit.__init__` itself rather than by `analyze_timing` (section 3.1); (2) forward propagation (section 5.3); (3) backward propagation (section 5.4); (4) slack, WNS, and TNS computation (section 5.5); and (5) critical-path extraction (section 5.6). Steps (2) and (3) both consume the results of step (1) but not of each other, and are logically independent of one another aside from that shared dependency.

## 5.1 Circuit Graph Construction and Topological Order

Once a netlist has passed validation (chapter 4), it is represented as an object graph rather than a collection of string-indexed lookup tables: each net object holds a direct reference to the gate object that drives it and a list of the `(pin_index, gate)` pairs it loads, and each gate object holds direct references to its input net objects and its output net object. This means that finding "who drives net $X$" or "what does gate $Y$ fan out to" is an $O(1)$ pointer dereference rather than a linear scan over every gate in the circuit, which matters materially at the scale of the optimizer's inner loop (chapter 8), where the same queries are issued on the order of hundreds of times per circuit.

The topological order is computed once, at `Circuit` construction, by the module-level helper `_make_topological_order(netlist, gates) -> tuple[tuple[Gate, ...], ...]` (`domain/circuit.py`) using Kahn's algorithm, and is stored not as a flat list but as a se-

quence of *levels* — exactly the nested-tuple return type above: gates are grouped by their longest-path depth from any primary input, so that all gates in level $\ell$ depend only on gates in levels $0, \ldots, \ell - 1$, and this is the `topological_order` field consumed directly by `_compute_arrival_times`'s outer `for level in circuit.topological_order: for gate in level:` loop (section 5.3). Flattening the levels in order recovers an ordinary topological order; the level structure itself is not required for correctness of the forward/backward passes below (which only need *a* topological order, not a minimal-depth one), but is retained because it is useful metadata for the benchmark generator's stratification by circuit depth (chapter 10).

## 5.2 Load and Delay Computation

Equations (2.1) to (2.3) are evaluated for every gate in a single pass, independent of iteration order, since both quantities depend only on the currently-assigned cell sizes of a gate and its fan-out — never on an arrival or required time. Concretely, this computation lives entirely in `domain/circuit.py` and runs once, eagerly, inside `Circuit.__init__`, via two private methods:

```
def _compute_fanout_capacitances(self) -> dict[str, float]: ...
    # Eq. (2.1)
def _compute_gate_delays(self) -> dict[str, RiseFall]: ...
    # Eq. (2.2)-(2.3)
```

with the per-gate rise/fall delay itself computed by the module-level helper `_gate_delay( cell: Cell, load_capacitance: float, kappa: float) -> RiseFall`, and total area and power by `Circuit._compute_area()` and `Circuit._compute_power()` (chapter 7). This is a common point of confusion worth stating explicitly: delay is *not* recomputed as a function of arrival time during the forward pass; it is a fixed property of the current sizing assignment, computed once by `Circuit.__init__` and only ever *read* by `analysis/sta.py`, never recomputed there.

## 5.3 Forward Propagation

Equation (2.4) is evaluated for every gate in topological order by `_compute_arrival_times` (section 5.1), which delegates the per-gate, per-transition maximization to `_select_predecessors(circuit, gate, output_transition, arrival_times, gate_delays) -> tuple[float, tuple[PredecessorArc, ...]]`. Two implementation details, both visible directly in this function's body, are worth making explicit.

**Net-keyed timing tables.** Arrival times are stored in a single `dict[str, RiseFall]` keyed by *net* name, not by gate name. A primary input's net and a gate's output net are both, from this table's point of view, just nets with an associated $(\mathrm{AT}^{\mathrm{rise}}, \mathrm{AT}^{\mathrm{fall}})$ pair; primary-input nets have theirs supplied directly from `circuit.config.input_arrival(net`

`_name)`, while gate-output nets have theirs computed by eq. (2.4). This avoids the need for a downstream consumer of an arrival time to first determine "is this a primary input or a gate output" before looking it up.

**Exhaustive tie recording.** `_select_predecessors` builds a list of every `(candidate_arrival, (pin_number, input_transition))` pair across a gate's input pins and their legal input transitions (`_valid_input_transitions`, implementing section 2.3), takes `maximum_arrival = max(...)`, and returns *every* candidate whose arrival exactly equals that maximum as a tuple of `PredecessorArc`s — not merely the first one encountered. This matters directly for critical-path reconstruction (section 5.6): a circuit with a genuinely symmetric or reconvergent structure can have two structurally distinct paths that are exactly equally critical, and discarding all but one of them arbitrarily would silently under-report the set of paths actually responsible for the reported worst-case delay.

## 5.4 Backward Propagation

Equation (2.5) is evaluated for every gate in *reverse* topological order by `_compute_required_times`, seeded at the primary outputs with `circuit.config.output_required(net_name)`. The propagation of a single required time backward through one timing arc is isolated into its own pure function,

```
def _propagate_required(output_required: RiseFall, gate_delay: RiseFall,
                        timing_sense: TimingSense) -> RiseFall: ...
```

whose three branches implement section 2.3's three timing-sense cases directly: positive-unate subtracts $D_i^s$ from $\mathrm{RT}_i^s$ transition-for-transition; negative-unate does the same with rise and fall swapped; and, for a non-unate pin, both output transitions are legal regardless of which input transition is being evaluated, so the function computes `earliest_candidate = min(output_required.rise - gate_delay.rise, output_required.fall - gate_delay.fall)` once and returns it as *both* the rise and fall required time propagated through that pin — a conservative choice that is required for correctness, since a non-unate input transition's required time must respect whichever output transition it might actually cause. `_compute_required_times` then folds this per-arc result into the running required time of each input net via a plain `min(...)`, implementing the outer minimization of eq. (2.5) over every fan-out arc.

## 5.5 Slack, WNS, and TNS

Equations (2.6) and (2.7) are evaluated by `_compute_transition_slacks` and `_compute_output_slack_metrics` directly from the arrival- and required-time tables produced above. A configuration flag, `config.timing_analysis.separate_rise_fall` (`TimingAnalysisConfig`, table 3.2), read inside `_output_slack_values`, controls whether rise and fall slack at a

given primary output are treated as two independent entries in the WNS/TNS computation (the default, and the behavior implied by a literal reading of eq. (2.7)) or collapsed to a single per-output worst-case value first; this is useful when a downstream consumer only cares about the offending net, not which polarity of transition is responsible.

## 5.6 Critical-Path Extraction

A naïve definition of "the $K$ worst paths" as "the $K$ worst (endpoint, transition) pairs, each with one arbitrarily chosen predecessor chain" would silently under-represent circuits with tied critical paths. Path extraction is instead implemented by `_find_critical_paths`, which iterates every primary-output net and both of its transitions and calls the recursive helper:

```
def _trace_paths_backward(circuit, current_net, current_transition,
        output_net, reversed_steps, reversed_transitions, endpoint_slack
            , found_paths, predecessors) -> None: ...
```

which *branches* at every tie recorded by `_select_predecessors` (section 5.3): it looks up `predecessors[(current_net.name, current_transition)]`, and recurses once per stored `PredecessorArc` in that tuple, prepending a `PathStep(gate, input_pin)` to `reversed_steps` each time, until `current_net.net_type is NetType.INPUT`, at which point a complete `CriticalPath(path, transitions, slack)` is appended to `found_paths`. All paths produced by this search, across all endpoints, are pooled into a single list, sorted by `path.slack`, and truncated to `circuit.config.timing_analysis.top_k_paths` by `_find_critical_paths` itself. Two consequences of this design are worth stating explicitly, since they differ from a simpler "one path per worst endpoint" scheme: first, the reported top-$K$ list is not guaranteed to contain one path per distinct endpoint — a circuit with several tied routings to the same endpoint can occupy multiple slots in the list; second, this is a strictly more complete answer to "which paths are responsible for the worst delay" than an implementation that picks one arbitrary routing per endpoint, at the cost of a somewhat less predictable relationship between $K$ and the number of distinct endpoints represented.

## 5.7 Worked Example: `c01_inverter_chain`

To ground the equations of this chapter in concrete numbers, this section traces the engine's computation by hand on the smallest supplied benchmark circuit, a three-gate inverter–buffer–inverter chain:

```
INPUT A
G1 INV_X1 A  N1
G2 BUF_X1 N1 N2
G3 INV_X1 N2 OUT
OUTPUT OUT
```

with primary-input arrival times $\mathrm{AT}_A^{\mathrm{rise}} = \mathrm{AT}_A^{\mathrm{fall}} = 0$ ns, primary-output required times $\mathrm{RT}_{\mathrm{OUT}}^{\mathrm{rise}} = \mathrm{RT}_{\mathrm{OUT}}^{\mathrm{fall}} = 0.11$ ns, and an external load of 4.0 fF on `OUT`. Table 5.1 reproduces, gate by gate, the load, delay, arrival time, required time, and slack computed by eqs. (2.1) to (2.6) using the `INV_X1`/`BUF_X1` library entries. The resulting WNS = 0.009 ns and TNS = 0 ns (no violation) were confirmed to match the tool's reported output to six decimal places.

| **Gate** | $C_{\mathrm{load}}$ **(fF)** | $D_{\mathrm{rise}}/D_{\mathrm{fall}}$ **(ns)** | $\mathrm{AT}^{\mathrm{rise}}/\mathrm{AT}^{\mathrm{fall}}$ | $\mathrm{RT}^{\mathrm{rise}}/\mathrm{RT}^{\mathrm{fall}}$ | **Slack (ns)** |
|---|---|---|---|---|---|
| G1 (`INV_X1`) | 1.0 | 0.0240 / 0.0185 | 0.0240 / 0.0185 | 0.0350 / 0.0275 | 0.011 / 0.009 |
| G2 (`BUF_X1`) | 1.0 | 0.0370 / 0.0315 | 0.0610 / 0.0500 | 0.0720 / 0.0590 | 0.011 / 0.009 |
| G3 (`INV_X1`) | 4.0 | 0.0510 / 0.0380 | 0.1010 / 0.0990 | 0.1100 / 0.1100 | 0.009 / 0.011 |

Table 5.1: Hand-verified static timing analysis of `c01_inverter_chain`. G3 is the sole primary-output-driving gate; its rise slack of 0.009 ns equals the circuit-level WNS.

Two points are worth drawing attention to that a reader might otherwise gloss over. First, because `INV_X1` is negative-unate, G1's rise *output* arrival time is driven by the primary input's *fall* transition ($\mathrm{AT}_A^{\mathrm{fall}} = 0 \Rightarrow \mathrm{AT}_{G1}^{\mathrm{rise}} = 0 + D_{\mathrm{rise},G1} = 0.024$), not its rise transition; a reader tracing the table without accounting for timing sense would expect the opposite pairing. Second, the worst slack in the circuit (0.009 ns) alternates which transition it belongs to at every stage of the chain (fall at G1, fall at G2, rise at G3) precisely because of this same negative-unate alternation — a further illustration of why rise and fall must be propagated as genuinely independent quantities rather than a single worst-case delay per gate.

# Chapter 6

# Logical-Effort Analysis

This chapter documents the implementation of section 2.5 as applied to the top-$K$ critical paths reported by the STA engine (section 5.6). Logical-effort analysis is purely a *post-processing* view over the STA result: it reads capacitances and delays directly off the already-analyzed circuit and does not perform any independent simulation of its own, so its results are always consistent with whatever sizing assignment was current when the STA pass that produced the critical paths was run.

## 6.1 Per-Stage Effort Computation

For each critical path, the sequence of gates from the driving primary input to the endpoint is walked in order. At each stage $i$, the specific input pin through which the path enters that gate is identified (recovered from the same predecessor information used to reconstruct the path in section 5.6), which determines $g_i$, the logical effort of that particular pin, from the cell library. The on-path receiver capacitance $C_{\text{onpath},i}$ is the input pin capacitance of the next stage along the path (or, for the final stage, the external load applied to the endpoint's net); the off-path capacitance $C_{\text{offpath},i}$ is obtained by subtracting $C_{\text{onpath},i}$ from the gate's total output load $C_{\text{load},i}$, which was already computed by the STA engine (eq. (2.1)) and reflects the gate's *actual, current* fan-out — not an idealized or path-only load. Branching effort $b_i$, electrical effort $h_i$, and stage effort $f_i$ then follow directly from eq. (2.8).

## 6.2 Path-Level Effort and the Sizing Target

Path-level quantities $G$, $B$, $H$, $F$, and $P$ (eq. (2.9)) are accumulated across all stages of the path, and the optimal per-stage effort $\hat{f} = F^{1/N}$ and theoretical minimum path delay $D_{LE,\text{min}}$ (eq. (2.10)) are computed once per path. Every stage's actual effort $f_i$ is then compared to $\hat{f}$: a stage with $f_i \gg \hat{f}$ is disproportionately loaded relative to its own drive strength and is, by the theory, the best next candidate for up-sizing. This overshoot quantity, $f_i - \hat{f}$, is the ranking signal consumed by the optimizer's logical-effort-guided heuristics (section 8.5).

Because each stage's desired capacitance (eq. (2.11)) depends only on that stage's own current load $C_{\text{total},i}$ and its own $g_i$ — both already known from the current, fixed sizing assignment — computing every stage's target capacitance does not require processing the path in any particular order; the "move backward along the path" framing used in

the assignment specification is a presentational convenience, carried over unchanged from classical hand-calculation logical-effort sizing, rather than a genuine data dependency in this implementation.

## 6.3 Discrete Sizing Candidates

The continuous target capacitance $C^*_{\text{in},i}$ (eq. (2.11)) will almost never coincide exactly with one of the three discrete sizes (`X1`/`X2`/`X4`) available in the library. Rather than rounding to the single nearest size by absolute capacitance difference, the implementation *brackets* the target: it finds the largest available cell whose input pin capacitance is still at or below the target, and the smallest available cell whose input pin capacitance is at or above it, and reports both as candidates (falling back to the nearest boundary cell if the target lies outside the family's achievable range entirely). This bracketing approach is more principled than nearest-distance rounding because it guarantees the two candidates straddle the theoretical ideal from both sides, which matters when the optimizer (chapter 8) evaluates both bracket candidates via a full STA re-run and lets the *measured* outcome, not the continuous approximation, determine which one is actually better once real, non-idealized loading effects on neighboring gates are accounted for.

| **Quantity** | **Symbol** | **Meaning** | **Computed from** |
|---|---|---|---|
| Logical effort | $g_i$ | Gate-topology cost of driving equal load | Library, per pin |
| Electrical effort | $h_i$ | On-path fan-out ratio | eq. (2.8) |
| Branching effort | $b_i$ | Off-path loading tax | eq. (2.8) |
| Stage effort | $f_i$ | $g_i b_i h_i$ | eq. (2.8) |
| Path effort | $F$ | $\prod_i f_i$ | eq. (2.9) |
| Optimal stage effort | $\hat{f}$ | $F^{1/N}$ | eq. (2.10) |
| Theoretical min. delay | $D_{LE,\min}$ | $\tau(NF^{1/N} + P)$ | eq. (2.10) |
| Target capacitance | $C^*_{\text{in},i}$ | Ideal input cap. for stage $i$ | eq. (2.11) |

Table 6.1: Summary of quantities reported per stage/path in `logical_effort_paths.csv`, `logical_effort_stages.csv`, and `logical_effort_candidates.csv`.

# Chapter 7

# Power and Area Estimation

Power and area are recomputed, in full, after every accepted change to the circuit's sizing assignment, so that the optimizer's cost function always operates on the current totals.

## 7.1 Static (Leakage) Power

Leakage power is supplied directly, per cell, by the standard-cell library and is independent of switching activity or circuit state; total leakage is the direct sum given in eq. (2.13). It depends only on which cells are currently assigned, and therefore changes only when a gate is resized.

## 7.2 Dynamic Power

Dynamic power follows the activity-factor model of eq. (2.12). The switching activity factor $\alpha_i$ for a given net is read from the configuration file's per-net override table if present, and otherwise falls back to a single circuit-wide default activity factor, also supplied by the configuration. The effective switched capacitance $C_{\mathrm{sw},i}$ combines the gate's output load $C_{\mathrm{load},i}$ — already computed by the STA engine (eq. (2.1)) and therefore automatically reflecting the gate's current fan-out — with its internal capacitance, a library-supplied constant per cell that accounts for switching energy dissipated inside the gate itself, independent of external loading.

## 7.3 Area

Total area is the direct sum of the currently-assigned cell's area over every gate, as in eq. (2.12). Because larger-drive-strength cells in the library are strictly larger in area (and generally larger in both leakage and internal capacitance) than their smaller-drive-strength counterparts within the same family, every accepted up-sizing decision made by the optimizer (chapter 8) has an unconditional, nonzero cost in both area and power, regardless of whether that particular gate lies on a currently-critical path — which is precisely why the optimizer restricts its candidate search to gates that are actually on a reported critical path (section 8.5) rather than considering every gate in the circuit: up-sizing a gate that is not contributing to any timing violation can only ever worsen the cost function's power and area terms for no corresponding delay benefit.

# Chapter 8

# Gate-Sizing Optimization

The optimizer is implemented in `optimization/optimizer.py:CircuitOptimizer`, with a deliberately narrow public surface:

```
class CircuitOptimizer:
    def __init__(self, circuit: Circuit,
                 heuristic: OptimizationHeuristic = BRUTE_FORCE,
                 random_seed: int | None = None) -> None: ...
    def optimize(self) -> OptimizationResult: ...
```

Everything else — candidate generation, cost evaluation, the acceptance policy, and termination — is private to this class or to `optimization/heuristics.py`, and is documented in this chapter by name because the *names* of these private functions are themselves the clearest statement of the algorithm's structure.

## 8.1 Cost Function

Every candidate gate-sizing change is scored by a single normalized cost function that trades off circuit delay $D_{\text{circuit}}$, total power $P_{\text{total}}$, total area $A_{\text{total}}$, and any remaining timing violation, each normalized against the corresponding value of the *original, unmodified* design (subscript ref), so that the cost is dimensionless and comparable across circuits of very different absolute size:

$$J = w_D \frac{D_{\text{circuit}}}{D_{\text{ref}}} + w_P \frac{P_{\text{total}}}{P_{\text{ref}}} + w_A \frac{A_{\text{total}}}{A_{\text{ref}}} + \lambda \frac{\max(0, -\text{WNS})}{T_{\text{ref}}}. \tag{8.1}$$

The weights $w_D, w_P, w_A, \lambda$ are read from `Config.optimization.weights` (`OptimizationWeights`, table 3.2). $D_{\text{ref}}, P_{\text{ref}}, A_{\text{ref}}$ are captured exactly once by the static method `CircuitOptimizer._normalization_reference(circuit, timing) -> _NormalizationReference`, called only from `_initial_state()` at the very start of `optimize()`, and never recomputed thereafter, so that $J$ always measures cost *relative to the starting point*. $T_{\text{ref}}$, a normalizing time reference for the violation penalty, is taken as the tightest required time among the circuit's primary outputs.

Cost itself is computed by the private method `CircuitOptimizer._evaluate(circuit, reference, timing=None) -> _Evaluation`, where `_Evaluation` is a frozen dataclass bundling `circuit`, `timing`, and `cost` together — the single value threaded through every other private method below.

## 8.2 Candidate Evaluation

Consistent with the immutability principle stated in section 3.1, evaluating a candidate resize never mutates the circuit currently being optimized. `CircuitOptimizer._candidate_evaluation(current, reference, gate, candidate_cell) -> _CandidateEvaluation | None` is the single choke point through which every candidate — from every heuristic — passes, in three ordered steps.

1. **Cheap pre-filter.** `replacement_area_and_power(current.circuit, gate.name, candidate_cell)` (table 3.2) computes the resulting total area and power *without* constructing a new `Circuit` object at all. If this alone already exceeds the configured budget (`_violates_area_or_power_constraints`), the candidate is discarded immediately, before paying for a full circuit construction or a timing re-run.
2. **Full construction and re-analysis.** Only if the cheap check passes is `replace_gate_cell(current.circuit, gate.name, candidate_cell)` called to build the new, independent `Circuit` variant, which is then fully re-evaluated: `_analyze(circuit)` runs `analyze_timing` (incrementing an internal `self._sta_calls` counter, reported as `OptimizationResult.sta_calls`), and `_satisfies_constraints(circuit)` performs the exact (non-delta) area/power check against the fully rebuilt circuit.
3. **Cost.** `_evaluate(candidate_circuit, reference)` scores the surviving candidate via eq. (8.1).

If a candidate is not selected, the `Circuit` object built for it is simply discarded — there is no revert step, because nothing was ever mutated in place. This is functionally equivalent to an evaluate-then-revert pattern built on a mutable graph, but structurally eliminates the possibility of a forgotten or incomplete revert silently corrupting a subsequent evaluation.

## 8.3 The Acceptance Policy

A detail that is easy to miss without reading the source directly, but is central to how the four heuristics relate to one another: **there is exactly one implementation of the two-phase accept/reject policy, and every heuristic calls it.**

```
def _select_next_change(self, current: _Evaluation,
                        candidates: list[_CandidateEvaluation]
                        ) -> _AcceptedChange | _StopDecision: ...
@staticmethod
def _best_timing_change(current, candidates) -> _CandidateEvaluation |
    None: ...
@staticmethod
def _best_cost_change(candidates) -> _CandidateEvaluation | None: ...
```

`_select_next_change` implements the two-phase rule directly:

1. **While a timing violation remains** (WNS $< 0$, `OptimizationPhase.TIMING_REPAIR`): `_best_timing_change` selects, among the supplied candidates, the one with the lowest cost $J$ among those that strictly improve WNS, or, failing that, strictly improve TNS with WNS unchanged. If none qualify, `_select_next_change` returns a `_StopDecision` with termination reason `NO_TIMING_IMPROVEMENT`.
2. **Once timing is compliant** (WNS $\geq 0$, `OptimizationPhase.COST_REDUCTION`): `_best_cost_change` selects the lowest-cost candidate that does not reintroduce a violation, and the change is accepted only if the resulting cost improvement exceeds `config.optimization.minimum_cost_improvement`. Otherwise, termination reason `TIMING_MET_NO_COST_IMPROVEMENT` is returned.

What differs between heuristics is only *how many* candidates `_select_next_change` is called with, and *in what order* they were generated (section 8.5) — not the acceptance rule itself. Brute force calls it once per iteration with the *entire* evaluated candidate batch:

```
choices = tuple(choice_iterator)     # exhaust the whole iterator
candidates = self._candidate_evaluations(current, reference, choices)
return self._select_next_change(current, candidates)  # pick the best of
    all
```

while the other three heuristics instead call `_evaluate_single_choice`, which evaluates exactly *one* candidate and delegates to the very same function with a singleton list:

```
def _evaluate_single_choice(self, current, reference, gate,
    candidate_cell
        ) -> _AcceptedChange | _RejectedChange:
    candidate = self._candidate_evaluation(current, reference, gate,
        candidate_cell)
    if candidate is None:
        return _RejectedChange(gate.name, candidate_cell.name,
                               "candidate violates area or power
                                   constraints")
    decision = self._select_next_change(current, [candidate])
    ...
```

This makes slack-weighted capacitance, criticality/effort-gap, and random greedy *sequential, first-improvement* searches: each pulls one candidate at a time from a pre-ordered stream and immediately accepts or rejects it, moving to the next candidate in that same stream on rejection, without re-ranking until a change is actually accepted. Brute force, in contrast, is a *best-of-batch* search: every eligible candidate is evaluated fresh each iteration, and the single best one is chosen. The optimizer's outer loop, `optimize()`, regenerates the candidate stream (`choice_iterator = None`, forcing a fresh call to `_choice_iterator` on the next iteration) only *after* an accepted change, so a rejected candidate does not trigger re-ranking — the search simply continues down the same, still-valid-for-the-current-circuit ordering.

## 8.4 Termination

`OptimizationTermination` enumerates five possible outcomes, recorded verbatim in `OptimizationResult.termination` and in the `optimization_*.csv` artifacts: `DISABLED` (optimization turned off in configuration); `NO_TIMING_IMPROVEMENT` (still violating, no candidate improves WNS/TNS); `NO_FEASIBLE_CHANGE` (the candidate stream is exhausted — only reachable by the three sequential heuristics, since brute force always sees its full batch every iteration); `TIMING_MET_NO_COST_IMPROVEMENT` (compliant, but no candidate reduces cost enough); and `MAXIMUM_ITERATIONS`.

## 8.5 The Four Sizing Heuristics

`OptimizationHeuristic` (`optimization/heuristics.py`) has four members: `BRUTE_FORCE`, `SLACK_WEIGHTED_CAPACITANCE`, `CRITICALITY_EFFORT_GAP`, and `RANDOM_GREEDY`. All four are dispatched from a single method, `_choice_iterator(self, current, random_generator) -> Iterator[_SizingChoice]`, whose four branches *are* the precise definition of what distinguishes the four heuristics: `RANDOM_GREEDY` takes `_critical_path_one_step_choices(current)` and shuffles it with `random_generator`; `CRITICALITY_EFFORT_GAP` passes that same one-step choice list through `iter_criticality_effort_gap_choices`; `BRUTE_FORCE` and `SLACK_WEIGHTED_CAPACITANCE` both instead start from `_eligible_choices(current, allowed_sizes)`, flattened directly for brute force or passed through `iter_ranked_optimization_choices` for the canonical heuristic.

Two *sources* of candidates are used, and it is important not to conflate them:

- `_critical_path_one_step_choices(current) -> list[tuple[Gate, Cell]]` returns, for every gate on any current critical path, only *one* candidate: its immediate next-larger cell in the same family (never a logical-effort-derived bracket, never a jump of more than one discrete size step). **Both `RANDOM_GREEDY` and `CRITICALITY_EFFORT_GAP`** draw from this same, narrower candidate set — they differ from each other only in the *order* they visit it (uniformly shuffled versus score-ranked), not in which cells are candidates at all.

- `_eligible_choices(current, allowed_sizes) -> list[OptimizationChoice]` (where `OptimizationChoice = tuple[Gate, tuple[Cell, ...]]`) calls `analysis/logica_effort.py:get_optimization_candidates`, which returns, for every gate on any current critical path, *every* discrete cell in its logical-effort sizing bracket (section 6.3) — potentially more than one size step in a single proposal. **Both `BRUTE_FORCE` and `SLACK_WEIGHTED_CAPACITANCE`** draw from this wider, logical-effort-derived candidate set.

### 8.5.1 Brute Force

Every $(gate, candidate)$ pair in the logical-effort bracket set is evaluated, in full, every iteration, and `_select_next_change` picks the single best. This is the most thorough of the four heuristics — and, per section 8.3, the only one that genuinely compares multiple candidates against each other before choosing, rather than walking a pre-ordered list — at the cost of the largest number of full STA re-runs per accepted change. It is available for manual and diagnostic use but is not the heuristic invoked to produce the canonical optimized result reported in chapter 12.

### 8.5.2 Slack-Weighted Capacitance (Canonical)

Draws from the same logical-effort bracket set as brute force, but visits it as a sequential, first-improvement search, in an order fixed once per outer-loop iteration by `iter_ranked_optimization_choices(circuit, paths, choices) -> Iterator[tuple[Gate, Cell]]`, which sorts candidates by `(-score, gate_position)` — highest score first, ties broken by netlist declaration order for determinism — where the score is computed by the private helper `_slack_weighted_capacitance_scores(circuit, paths) -> dict[str, float]`: for every stage of every current critical path, an *urgency* factor (ranging from 1.0, for the least-violating path present, up to 2.0, for the most-violating one, linearly interpolated by that path's slack relative to the range of slacks across all reported critical paths) is multiplied by the logarithmic mismatch $|\ln(C^*_{\text{in},i}/C_{\text{in},i})|$ between the stage's logical-effort target capacitance (eq. (2.11)) and its currently-assigned capacitance, and the contributions are *summed* across every path a given gate appears on. This is the heuristic actually invoked by the standard analysis pipeline (`CANONICAL_OPTIMIZATION` in `app/experiments.py`) to produce the reported "logical-effort-guided" result throughout chapter 12.

### 8.5.3 Criticality/Effort-Gap

A second logical-effort-*informed* heuristic, but one that — unlike slack-weighted capacitance — does *not* draw from the capacitance bracket at all. It re-orders the same narrow, one-step-up candidate set used by random greedy (`_critical_path_one_step_choices`) via `iter_criticality_effort_gap_choices(circuit, paths, choices)`, whose ranking is produced by `criticality_effort_gap_scores(circuit, paths) -> dict[str, float]` (documented in-source as scoring gates by "urgency × (current stage effort − optimal effort)"): the same urgency factor as section 8.5.2, but multiplied instead by the raw logical-effort stage-effort overshoot $f_i - \hat{f}$ (eq. (2.10)) directly — and, where slack-weighted capacitance *sums* a gate's contribution across every path it appears on, this score takes the *maximum* over paths instead. In short: slack-weighted capacitance asks "how large a capacitance mismatch does this gate have, aggregated over everywhere it's used, and how large a library jump would fix it," while criticality/effort-gap asks "how far over its ideal per-stage effort is this gate, at its single worst appearance, if I only allow myself the next

library size up." Because both heuristics can, in principle, still propose the exact same one-step-up resize when that happens to also lie in the logical-effort bracket, a difference in final outcome between the two is not guaranteed to come from candidate *scope* at all — section 12.3.4 traces one benchmark circuit where it instead comes from *ordering interacting with the area/power budget*: criticality/effort-gap accepts an earlier, WNS-neutral change that consumes part of the budget, after which the one candidate that would have fixed the violation is rejected purely on that now-tighter budget, whereas slack-weighted capacitance reaches the identical candidate earlier, before any budget had been spent.

### 8.5.4 Random Greedy (Baseline)

The baseline heuristic against which the three logical-effort-informed heuristics are compared: the same one-step-up candidate set as criticality/effort-gap, visited in a uniformly random order (`random_generator.shuffle(choices)`, seeded by `CircuitOptimizer(..., random_seed=...)`) rather than any score-based order. It is, in the strict sense, the only one of the four heuristics that makes no use of logical effort anywhere in its candidate generation.

| **Heuristic** | **Candidate source** | **Ordering** | **Evaluation** |
|---|---|---|---|
| Brute Force | LE bracket (`_eligible_choices`) | — (all at once) | Best-of-batch |
| Slack-Wtd. Capacitance | LE bracket (`_eligible_choices`) | Score-ranked | Sequential |
| Criticality/Effort-Gap | One-step-up only | Score-ranked | Sequential |
| Random Greedy | One-step-up only | Uniform random | Sequential |

Table 8.1: Summary of the four gate-sizing heuristics, by candidate source and search strategy (section 8.5). All four share the identical two-phase accept/reject policy (section 8.3).

# Chapter 9

# Monte Carlo Statistical Analysis

## 9.1 Variation Model

Following eq. (2.14), each Monte Carlo sample draws one global random variable $Z_g$, shared by every gate in that sample, and one local random variable $Z_i$ per gate, drawn independently. Because nominal gate delay (eqs. (2.2) and (2.3)) is always strictly positive for any physically valid cell, a sample is discarded and re-drawn only when the *multiplicative* variation factor $1 + \sigma_g Z_g + \sigma_l Z_i$ itself would be non-positive for some gate — a condition that can be checked without reference to any particular circuit's nominal delay values.

## 9.2 Paired Pre-/Post-Optimization Comparison

A specific correctness property is enforced by construction: the same set of $M$ valid variation samples — the same sequence of $(Z_g^{(k)}, Z_i^{(k)})$ draws, including the same accept/reject decisions from the resampling rule — is generated exactly once per run and then applied unchanged to *both* the pre-optimization circuit and the post-optimization circuit. This is possible precisely because the resampling condition above depends only on the variation factor, not on any circuit-specific nominal delay, so the same sample set is guaranteed valid for both circuits regardless of how their sizing differs. The result is a genuinely paired comparison: sample $k$ represents the *same* hypothetical manufactured die under both sizing assignments, which is a meaningfully stronger guarantee than independently reseeding a random number generator for each of the two runs, since two independently-seeded runs are only guaranteed to draw identically if they consume randomness in exactly the same order and quantity, which is not obviously true once resampling is involved and the two circuits' nominal delays differ.

## 9.3 Reported Metrics

For each of the two paired runs, full STA (chapter 5) is re-executed once per sample against that sample's perturbed delay set, and the following are reported: the mean, standard deviation, minimum, and maximum of circuit delay, WNS, and TNS across all samples; the empirical violation probability and timing yield (eq. (2.15)); the modal (most frequently observed) critical path and the fraction of samples on which it occurs, as a measure of critical-path stability under variation; and, per gate, the empirical probability

that it appears on the sample's single most critical path, which identifies gates whose criticality is stable versus gates that only become critical under specific variation draws.

## 9.4 Interpretation

Comparing the pre- and post-optimization statistical summaries answers a question that a single, nominal-corner STA run cannot: whether the gate-sizing changes made by the optimizer (chapter 8) improved robustness to manufacturing variation, not merely the nominal-case WNS/TNS. A design can be nominally compliant yet statistically fragile (a high violation probability under realistic variation) if its post-optimization slack margin is thin; chapter 12 reports this comparison for the supplied Monte Carlo-enabled benchmark circuit and discusses whether the optimizer's nominal-case improvement translated into a statistically meaningful yield improvement.

# Chapter 10

# Synthetic Benchmark Generation and Evaluation Framework

## 10.1 Motivation

The fixed benchmark set supplied with the assignment — fifteen valid circuits and six intentionally invalid ones — is well suited to demonstrating correctness of the parser, the STA engine, and the logical-effort analysis, and chapter 12 reports results on it in full. It is, however, a poor basis for comparing four different gate-sizing heuristics against each other: fifteen circuits is too small a sample to distinguish "heuristic A is genuinely better than heuristic B" from "heuristic A happened to get a favorable draw of critical-path structure on this particular handful of circuits." A statistically meaningful comparison instead requires (i) a much larger population of test cases, (ii) enough structural diversity across that population (circuit size, depth, fan-out, reconvergence, and violation severity) to characterize *where* a heuristic wins or loses rather than only *whether* it does on average, and (iii) a known-good reference solution for each case against which every heuristic's outcome can be measured, since without a reference there is no way to distinguish "heuristic A stopped early because the problem was already solved" from "heuristic A stopped early because it gave up." The benchmarking package addresses all three.

## 10.2 Case Generation

### 10.2.1 Topology Construction

Two sources of circuit topology are supported: procedurally generated levelized DAGs, constructed to a configured target size, depth, and fan-out/reconvergence profile, and topologies imported unchanged from one of the fixed valid seed circuits (chapter 12), allowing the same generation-and-evaluation pipeline to be run against known, hand-inspectable structures as a sanity check on the synthetic generator itself.

### 10.2.2 Reference Solution via Multi-Start Beam Search

For a given topology, a *best-known reference* gate-sizing assignment is established independently of any of the four heuristics under test, by a multi-start beam search that explores,

from several different random starting assignments, the space of legal per-gate size choices, retaining at each search depth only the most promising partial assignments (beam-limited to bound the search cost) and keeping the best complete assignment found across all starts. This reference is explicitly certified in the output only as `best_known_multi_start_beam` — global optimality is never claimed — but it is a substantially stronger reference than any single heuristic's own output, since it is not subject to the same candidate-generation restrictions (top-5 pruning, single random gate per iteration, and so on) that the heuristics themselves are.

### 10.2.3 Planted Violations

Starting from the reference assignment, one or more gates are deliberately *downsized* until the resulting initial circuit exhibits a negative WNS while remaining within its configured area and power budgets. This produces a case with a known-repairable violation (the reference assignment is a witness that a compliant, budget-respecting solution exists) and a recorded ground truth for which specific gates were mutated away from the reference (`planted_mutations.csv`), which downstream evaluation (section 10.3) can use as an additional diagnostic signal, distinct from its primary correctness metric.

### 10.2.4 Case Integrity and Reproducibility

Both the initial (post-mutation) circuit and the reference assignment are independently replayed through the production netlist parser, `Circuit` constructor, and STA implementation — the same code path used for ordinary analysis — before a case is considered valid and written to disk, ensuring that no generated case can silently violate an invariant the rest of the tool assumes. Every case directory is written atomically: if generation is interrupted or exhausts its retry budget partway through, no partial case directory is left behind, and suite-level failure diagnostics (`generation_failures.csv`) are preserved instead. All random seeds used during generation are derived from SHA-256 hashes rather than from Python's built-in, per-process-randomized `hash()` function, so that a given configuration deterministically reproduces the same suite across machines and Python invocations.

### 10.2.5 Suite Layout

```
benchmarks/<suite>/
  suite_config.json          # the generating configuration
  cell_library.json          # copy of the library used for the suite
  suite_manifest.json        # index of every case and its parameters
  generation_cases.csv       # per-case generation record
  generation_failures.csv    # diagnostics for any discarded attempt
  cases/<case-id>/
    netlist.txt
    config.json
    reference_assignment.json
```

```
    benchmark_manifest.json
    planted_mutations.csv
```

## 10.3 Evaluation Methodology

### 10.3.1 Counterfactual Oracle

The central evaluation instrument is an *oracle* that is entirely independent of any of the four heuristics being evaluated. For every gate assignment ("state") actually visited while replaying a heuristic's recorded decisions on a case, the oracle evaluates *every* legal same-family cell replacement on *every* gate in the circuit — not merely the gates a given heuristic happened to consider as candidates — and ranks them by the same two-phase rule used by the optimizer itself (section 8.3): while a violation remains, rank by WNS improvement, then TNS improvement on ties; once compliant, rank by cost reduction subject to remaining compliant. This makes it possible to ask a question no heuristic's own reported outcome can answer on its own: at each individual decision point, was the choice the heuristic actually made the *best available* choice, a merely *tied-best* choice, or a strictly worse choice than something the oracle can show was available? Exact recovery of the planted reference assignment (section 10.2.3) is retained only as a secondary diagnostic, since more than one gate-sizing assignment can be simultaneously compliant and cost-optimal; the oracle-ranked decision quality at each step is the primary correctness signal.

### 10.3.2 Parallel, Deterministic Evaluation

Cases are independent of one another and are evaluated in parallel (four worker processes by default, configurable), with per-run timeouts enforced so that a single pathological case cannot stall an entire suite; individual case report fragments are merged back together in a fixed manifest order so that the final evaluation report is reproducible regardless of which worker happened to finish a given case first. Random Greedy, being inherently stochastic, is additionally repeated with several deterministic seeds per case so that its reported metrics reflect a distribution of outcomes rather than a single draw.

### 10.3.3 Reported Metrics

| Artifact | Contents |
|---|---|
| `case_runs.csv` | Final metrics per case/heuristic: constraint-repair status, runtime, iteration and STA-call counts, and distance from the reference assignment. |
| `optimizer_iterations.csv` | A fully replayable, step-by-step history of every heuristic run. |

| Artifact | Contents |
|---|---|
| `oracle_states.csv`, `oracle_candidates.csv` | Every unique gate-assignment state visited, and every counterfactual candidate the oracle evaluated at that state, with feasibility, benefit, and tied-best status. |
| `gate_selection_scores.csv` | Per gate/move: selection hit rate, WNS/TNS/cost regret relative to the oracle's best available choice, and missed off-path opportunities. |
| `heuristic_summary.csv` | **Wilson confidence intervals** on repair rate, failure and timeout rates, Random Greedy's runtime dispersion across its repeated seeds, selection accuracy against the oracle, and aggregate regret. |
| `evaluation_summary.json` | Hierarchical aggregates stratified by circuit source, size, depth, fan-out, reconvergence, violation severity, and slack headroom, plus a manifest of every artifact produced. |

Table 10.1: Artifacts produced by `vlsi-sta benchmark evaluate`.

Using a **Wilson score interval** rather than a naive $\hat{p} \pm 1.96\sqrt{\hat{p}(1-\hat{p})/n}$ normal-approximation interval for the repair-rate confidence bound is a deliberately more careful choice: the Wilson interval remains well-behaved (bounded within $[0, 1]$ and reasonably calibrated) even when the observed repair rate is close to 0 or 1 or the sample count for a given stratum is small, both of which are expected to occur in some of the finer-grained strata of section 10.3.3's hierarchical aggregation.

## 10.4 Plotting

`vlsi-sta benchmark plot` consumes an evaluation directory and produces the following plots using the files listed on Table 10.1:

```
benchmarks/<suite>/evaluation/<run-id>/plots
          case_reliability.png
          heuristic_overview.png
          repair_convergence.png
          runtime_scalability.png
          selection_quality.png
```

# Chapter 11

# Interactive Topology Viewer

## 11.1 Purpose and Architecture

Static PNG renderings of a circuit's DAG (`circuit_graph_{pre,post}_optimization.png` artifacts of table 3.3) are adequate for small circuits but become difficult to read once a circuit reaches the size of the larger benchmark circuits (upwards of a hundred gates) or once a reader wants to compare the pre- and post-optimization state of the same circuit interactively rather than by flipping between two static images. The interactive viewer addresses this: every analysis run exports `circuit_topology.json`, a versioned, self-contained graph description — one shared topology (primary inputs, gates, primary outputs, and named nets) with two named overlays, `original` and `canonical_optimized`, each supplying per-gate cell assignment, load capacitance, rise/fall delay, and ranked critical paths for that state of the circuit. A small local server, launched with

```
vlsi-sta view outputs/c15_high_fanout_sizing_fabric
```

serves this JSON contract to a browser-based front end and opens it automatically (suppressible with `-no-browser` for remote or scripted use, with `-host`/`-port` available for remote access).

## 11.2 Features

- Mouse- and touch-based panning, cursor-anchored wheel zoom, and a fit-to-view command, with keyboard shortcuts (`+`, `-`, `F`, `0`, `Escape`) mirroring the on-screen controls.
- Optional net labels and node selection, with connected-net isolation to declutter the view of a densely fanned-out node.
- Independent visibility toggles for the `original` and `canonical_optimized` overlays, so the two states can be viewed separately or superimposed.
- Critical-path coloring consistent with the static PNG renderings.
- **Dual-state gate inspection**: selecting a gate shows its cell, load, and delay in both the original and optimized state side by side.

- **Target-size highlighting** for any gate the optimizer resized: an amber halo marks a gate resized to an `X2` target, a wider purple halo marks a gate resized to an `X4` target. This highlighting is shown only while both overlays are simultaneously enabled, since it is inherently a comparison between the two states.
- **Analysis and Schematic View modes**: Analysis view is the normal mode (similar to the PNG files), but a *Schematic View* feature enables the user to see the real circuit elements instead of boxes.

## 11.3 Screenshots

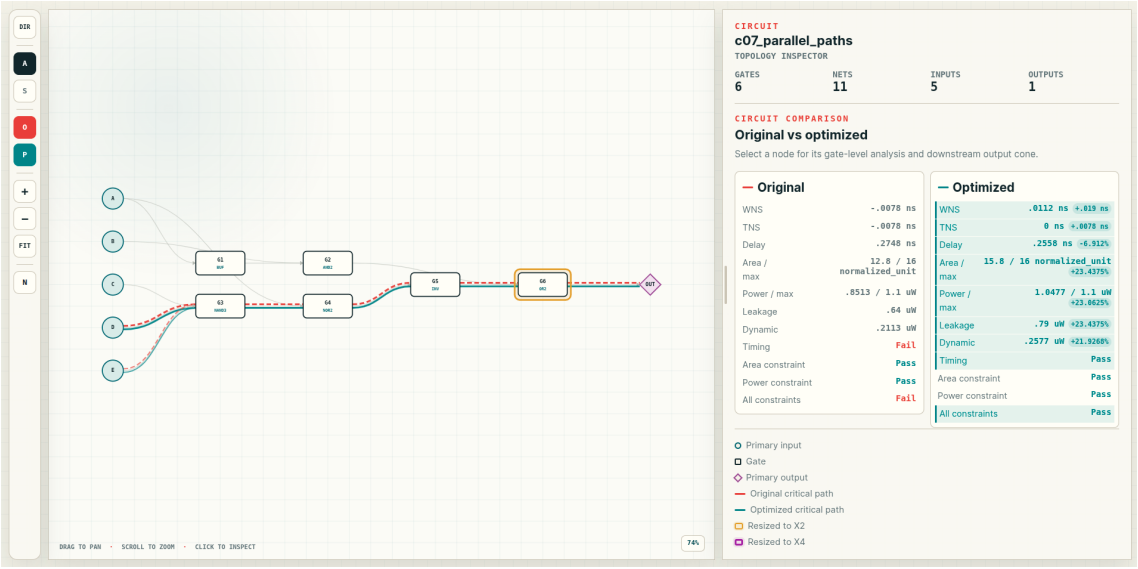



Figure 11.1: Interactive topology viewer: Analysis View mode.

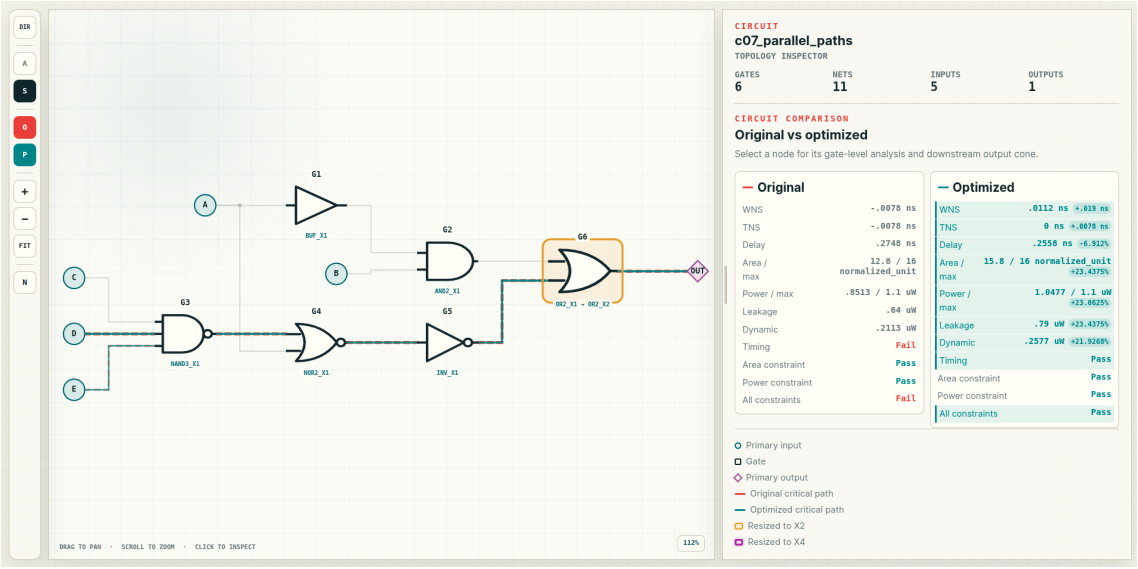



Figure 11.2: Interactive topology viewer: Schematic View mode.

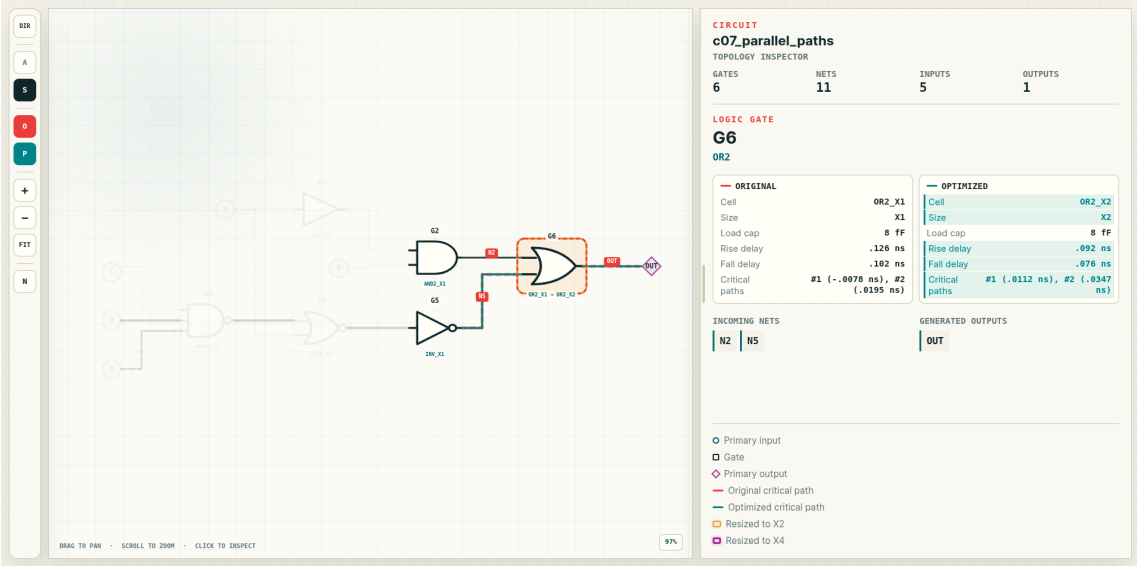



Figure 11.3: Interactive topology viewer: gate inspection mode (clicked on a gate).

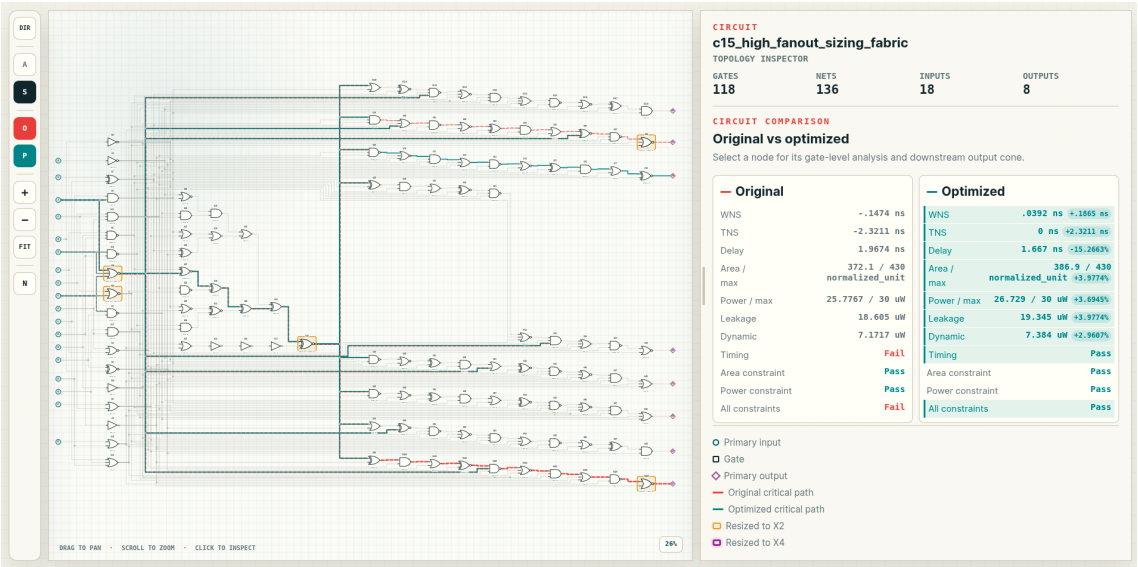



Figure 11.4: Interactive topology viewer: Schematic view for a larger circuit.

# Chapter 12

# Results

All results in this chapter were produced by running the complete analysis pipeline (using `./run_circuit.sh -all`) against the fifteen valid and six invalid benchmark circuits supplied with the assignment, on the codebase version accompanying this report.

## 12.1 Structural Validation on the Invalid Benchmark Set

All six intentionally invalid circuits were correctly rejected, each with the specific error class the circuit was constructed to exercise (table 4.1). Table 12.1 reproduces the exact diagnostic message emitted for each.

| Circuit | Reported diagnostic |
|---|---|
| e01_undefined_cell | undefined cell 'MUX2_X1' |
| e02_undriven_signal | signal 'X' is undriven |
| e03_multiple_drivers | net 'N1' has multiple drivers ('G1' and 'G2') |
| e04_combinational_loop | combinational loop involving gates: G1, G2, G3 |
| e05_wrong_input_count | cell 'NAND2_X1' expects 2 inputs, got 1 |
| e06_undriven_output | primary output 'OUT' is undriven |

Table 12.1: Validation results on the six invalid benchmark circuits.

## 12.2 Nominal Timing on the Valid Benchmark Set

Table 12.2 reports, for all fifteen valid circuits, the gate count and the nominal (pre-optimization) circuit delay, WNS, and TNS. One of the fifteen circuits (`c01_inverter_chain`) is already timing-compliant (nonnegative WNS); the remaining six exhibit a deliberate timing violation by construction, ranging from a marginal $-0.002$ ns on `c12_monte_carlo_switching` to a severe $-0.147$ ns (with a TNS of $-2.321$ ns, reflecting many simultaneously violating output transitions) on `c15_high_fanout_sizing_fabric`, the largest circuit in the suite at 118 gates.

| Circuit | Gates | Delay (ns) | WNS (ns) | TNS (ns) |
|---|---|---|---|---|
| c01_inverter_chain | 3 | 0.1010 | 0.0090 | 0.0000 |
| c02_basic_nand_path | 3 | 0.1620 | -0.0070 | -0.0070 |
| c03_fanout_branch | 4 | 0.2010 | -0.0060 | -0.0060 |
| c04_reconvergent_paths | 5 | 0.2722 | -0.0072 | -0.0072 |
| c05_multi_output | 5 | 0.2253 | -0.0053 | -0.0078 |
| c06_deep_critical_path | 10 | 0.4527 | -0.0227 | -0.0268 |
| c07_parallel_paths | 6 | 0.2748 | -0.0078 | -0.0078 |
| c08_non_unate_logic | 4 | 0.2687 | -0.0087 | -0.0130 |
| c09_high_fanout | 6 | 0.2090 | -0.0040 | -0.0073 |
| c10_three_input_gates | 5 | 0.2870 | -0.0120 | -0.0120 |
| c11_gate_sizing_stress | 6 | 0.3595 | -0.0295 | -0.0295 |
| c12_monte_carlo_switching | 7 | 0.3172 | -0.0022 | -0.0022 |
| c13_large_reconvergent_network | 74 | 1.0103 | -0.0293 | -0.2133 |
| c14_layered_reconvergent_fabric | 104 | 0.9893 | -0.0213 | -0.1596 |
| c15_high_fanout_sizing_fabric | 118 | 1.9674 | -0.1474 | -2.3211 |

Table 12.2: Nominal (pre-optimization) timing on the fifteen valid benchmark circuits. Nine of fifteen circuits are supplied already timing-compliant; the remaining six exhibit a constructed violation.

## 12.3 Gate-Sizing Heuristic Comparison

Table 12.3 reports the final WNS reached by each of the three automatically-invoked heuristics (`slack_weighted_capacitance`, the canonical logical-effort-guided method; `criticality_effort_gap`, the second logical-effort-informed heuristic; and `random_greedy`, the non-logical-effort baseline), together with a compliance flag indicating whether that heuristic reached a fully timing-compliant final design within the configured area and power budgets.

| Circuit | $\mathbf{WNS}_{\mathrm{swg}}$ | $\mathbf{WNS}_{\mathrm{ceg}}$ | $\mathbf{WNS}_{\mathrm{rg}}$ |
|---|---|---|---|
| c01_inverter_chain | 0.0090 | 0.0090 | 0.0090 |
| c02_basic_nand_path | 0.0050 | 0.0050 | -0.0010 |
| c03_fanout_branch | 0.0150 | 0.0150 | 0.0150 |
| c04_reconvergent_paths | 0.0059 | 0.0059 | 0.0069 |
| c05_multi_output | -0.0025 | -0.0025 | -0.0053 |
| c06_deep_critical_path | 0.0033 | 0.0033 | -0.0217 |
| c07_parallel_paths | 0.0112 | 0.0112 | -0.0078 |
| c08_non_unate_logic | 0.0093 | 0.0093 | 0.0093 |
| c09_high_fanout | 0.0430 | 0.0430 | 0.0430 |

| Circuit | $\mathbf{WNS}_{\mathrm{swg}}$ | $\mathbf{WNS}_{\mathrm{ceg}}$ | $\mathbf{WNS}_{\mathrm{rg}}$ |
|---|---|---|---|
| c10_three_input_gates | 0.0330 | 0.0330 | 0.0330 |
| c11_gate_sizing_stress | 0.0325 | 0.0325 | -0.0220 |
| c12_monte_carlo_switching | 0.0129 | 0.0129 | -0.0002 |
| c13_large_reconvergent_network | 0.0037 | -0.0293 | -0.0293 |
| c14_layered_reconvergent_fabric | 0.0182 | 0.0182 | 0.0030 |
| c15_high_fanout_sizing_fabric | 0.0392 | 0.0397 | 0.0397 |
| **Repair rate** | **14/15** | **13/15** | **8/15** |

Table 12.3: Final WNS (ns) and compliance status per heuristic. Repair rate counts circuits reaching WNS $\geq 0$ within area/power budgets, out of all fifteen valid circuits.

Restricted to only the fourteen circuits that exhibited a violation to begin with (excluding `c01`, already compliant), the repair rates are 13/14 ($\approx 92.9\%$) for slack-weighted capacitance, 12/14 ($\approx 85.7\%$) for criticality effort gap, and 7/14 ($= 50.0\%$) for random greedy heuristic.

### 12.3.1 Efficiency: STA Calls

Summed across all fifteen circuits, the canonical logical-effort-guided heuristic issued **119** total STA calls, versus **155** for random greedy — and the correct explanation for this is more specific than "one attempt, one STA call." Section 8.2 already establishes that candidate evaluation is a two-step gate: a cheap, STA-free pre-filter (`replacement_area_and_power`) rejects a candidate outright if it alone would exceed the area or power budget, and only a candidate that survives this filter reaches the actual `analyze_timing` call. Cross-checking the recorded iteration logs confirms this directly: on `c13_large_reconvergent_network`, the canonical heuristic's log contains 66 logged attempts, of which 52 are rejected with reason *"candidate violates area or power constraints"* (free, no STA call) and only 13 require an actual timing result (10 rejected for not improving WNS/TNS, 1 rejected once compliant for not improving cost, and 2 accepted) — plus one further STA call to establish the initial reference state (section 8.1), for a reported total of 14, exactly matching `summary.json`.

Because budget-infeasible candidates cost nothing, a heuristic's STA-call total measures how many candidates it proposes that are *plausible enough to need a real timing check*, not merely how many candidates it tries in total. On this specific circuit, random greedy needed only 3 STA calls in total (1 initial, 1 accepted, 1 WNS/TNS rejection) against the canonical heuristic's 14 — the canonical heuristic's higher-scored candidates more often clear the cheap budget pre-filter and therefore more often reach the expensive check, while most of random greedy's random draws are eliminated for free before ever costing a timing analysis. This cautions against reading the suite-wide STA-call total as a simple "search

order wastes fewer attempts" story: the totals do favor the canonical heuristic in aggregate (119 versus 155) and as a per-circuit mean ($\approx$ 7.9 versus $\approx$ 10.3 calls per circuit), but not uniformly — on five of the fifteen circuits, including the two largest (`c13`: 14 versus 3; `c14`: 39 versus 19), the canonical heuristic issues *more* STA calls than random greedy, precisely because more of its proposals are good enough to be worth the expensive check in the first place. Aggregate STA-call count is therefore best read as a rough proxy for total search cost, not as a clean, uniformly-favorable efficiency metric on its own; tables 12.3 and 12.4's repair-rate and final-cost comparisons are the more direct evidence for the canonical heuristic's advantage.

### 12.3.2 Cost, Power, and Area

Table 12.4 compares the final cost $J$ (eq. (8.1)) reached by the canonical heuristic against random greedy. The canonical method reaches a *strictly* lower (better) final cost than random greedy on ten of the fifteen circuits, and *ties exactly* on the remaining five: on `c01` because neither heuristic makes any accepted change at all (the circuit is already compliant and no candidate reduces cost), and on `c03`, `c08`, `c09`, and `c10` because both heuristics independently accept the identical sequence of resizes (verified directly against the recorded iteration logs, not merely inferred from matching final cost). On *no* circuit in the suite does random greedy reach a lower final cost than the canonical heuristic. The narrowest margin in the canonical method's favor is on `c15` ($J = 1.263$ versus $J = 1.272$), still a real, if small, improvement rather than a tie.

| **Circuit** | $J_0$ | $J_{\text{swc}}$ | $J_{\text{rg}}$ |
|---|---|---|---|
| c01_inverter_chain | 1.400 | 1.400 | 1.400 |
| c02_basic_nand_path | 1.616 | 1.511 | 1.578 |
| c03_fanout_branch | 1.549 | 1.342 | 1.342 |
| c04_reconvergent_paths | 1.532 | 1.434 | 1.481 |
| c05_multi_output | 1.518 | 1.466 | 1.587 |
| c06_deep_critical_path | 1.650 | 1.377 | 1.659 |
| c07_parallel_paths | 1.543 | 1.424 | 1.573 |
| c08_non_unate_logic | 1.561 | 1.376 | 1.376 |
| c09_high_fanout | 1.496 | 1.207 | 1.207 |
| c10_three_input_gates | 1.609 | 1.368 | 1.368 |
| c11_gate_sizing_stress | 1.810 | 1.283 | 1.780 |
| c12_monte_carlo_switching | 1.434 | 1.412 | 1.444 |
| c13_large_reconvergent_network | 1.545 | 1.376 | 1.551 |
| c14_layered_reconvergent_fabric | 1.508 | 1.291 | 1.375 |
| c15_high_fanout_sizing_fabric | 1.775 | 1.263 | 1.272 |

Table 12.4: Final cost function value $J$, initial versus each heuristic's final design.

### 12.3.3 Case Study: c02_basic_nand_path

Figures 12.1 and 12.2 show the rendered pre- and post-optimization circuit diagrams for the smallest circuit on which slack-weighted capacitance and random greedy diverge in outcome, and the full recorded iteration logs (`optimization_slack_weighted_capacitance.csv`, `optimization_random_greedy.csv`) make the reason for the divergence precisely traceable rather than merely inferable from the final numbers.

Slack-weighted capacitance's very first candidate — ranked highest by its urgency×mismatch score (section 8.5.2) — is `G3: AND2_X1→AND2_X2`, which alone resolves the violation (WNS : $-0.0070 \rightarrow +0.0050$ ns) in one accepted change. Three further candidates (`G1`, `G3→ X4`, `G2`) are then proposed by the cost-reduction phase (section 8.3) and all three are *rejected* for violating the configured area/power budget, at which point the search terminates `no_feasible_change` — a normal, compliant stop, not a failure.

Random greedy's uniformly random draw instead tries `G1: NAND2_X1→X2` first (accepted; WNS : $-0.0070 \rightarrow -0.0025$ ns, an improvement but not yet compliant), then rejects `G3: AND2_X1→X2` for exceeding the area budget (at this point, with `G1` already upsized, the combined area would reach 10.2, well past the 8.5 limit), then accepts `G2: INV_X1→X2` (WNS→ $-0.0010$ ns), and finally exhausts its remaining candidates (`G1→X4`, `G2→X4`, a repeated `G3→X2` attempt) all rejected on the same area ceiling, terminating `no_feasible_change` while **still in violation**.

Both searches plateau at the identical configured budget ceiling (`maximum_area` = 8.5), reached via two different combinations of accepted resizes: slack-weighted capacitance reaches a total area of 8.4 after its single accepted `G3` resize alone, while random greedy reaches the same 8.4 via its two smaller accepted resizes (`G1` then `G2`) combined. From that point, every further candidate in *either* run — regardless of which gate it targets — is rejected against the same 8.5 ceiling; the outcome is decided not by which gate the budget happens to block, but by whether a compliant WNS was already reached *before* the ceiling was hit. Slack-weighted capacitance's score-ranked draw tried the one resize that both fixes the violation and fits the pristine budget on its very first attempt; random greedy's random draw spent that same budget across two resizes that together do not fix the violation, before any further candidate could be tried. This is a concrete, single-circuit illustration of exactly the aggregate effect quantified in section 12.3.1: an informed search order does not expand what a heuristic is allowed to do, only the likelihood that a fix is found before the shared budget ceiling is reached by changes that do not resolve the violation.

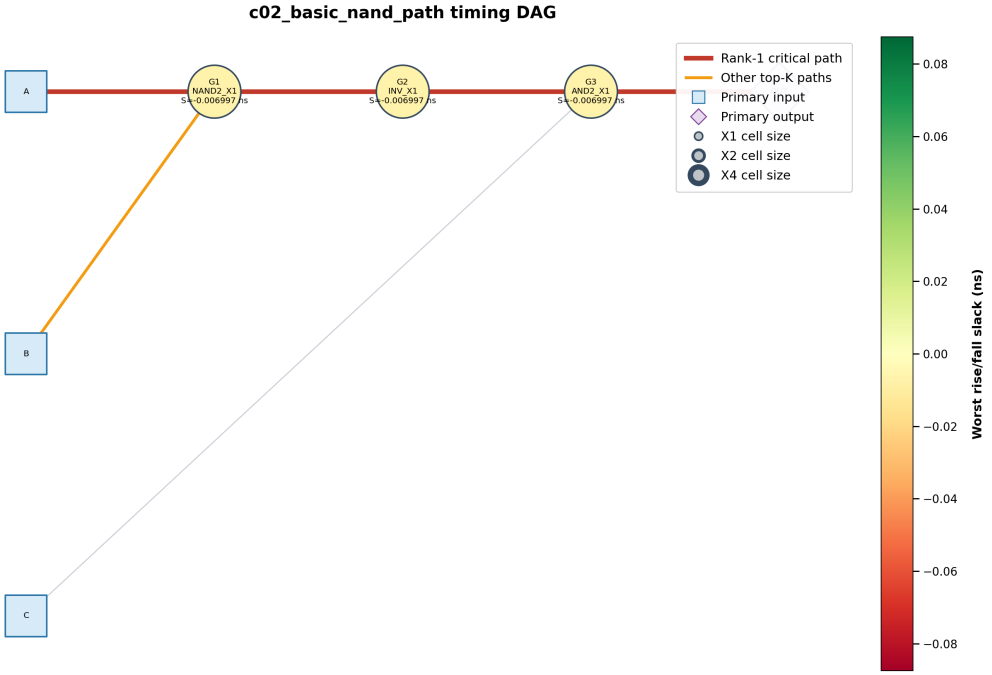



Figure 12.1: `c02_basic_nand_path`, pre-optimization. Nodes are colored by slack; the critical path is highlighted.

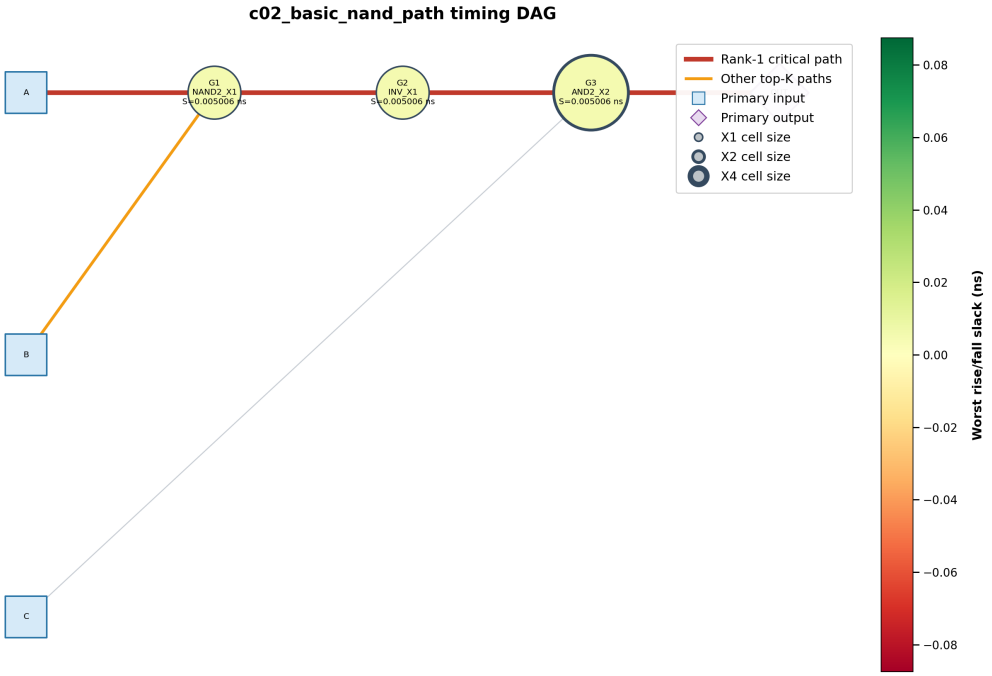



Figure 12.2: `c02_basic_nand_path`, post-optimization (canonical slack-weighted-capacitance heuristic). Gate `G3` has been resized from `AND2_X1` to `AND2_X2`, resolving the timing violation.

Figure 12.3 makes the same story visible directly as a convergence trace: slack-weighted capacitance (blue) reaches zero TNS and positive WNS in a single accepted step and holds there, while random greedy (green) needs two accepted steps to make partial progress

and never crosses zero. Figure 12.4 summarizes the same run as final outcomes — note criticality effort gap (orange) is visually indistinguishable from slack-weighted capacitance on this particular circuit, since both happen to find the same `G3` fix, consistent with table 12.3.

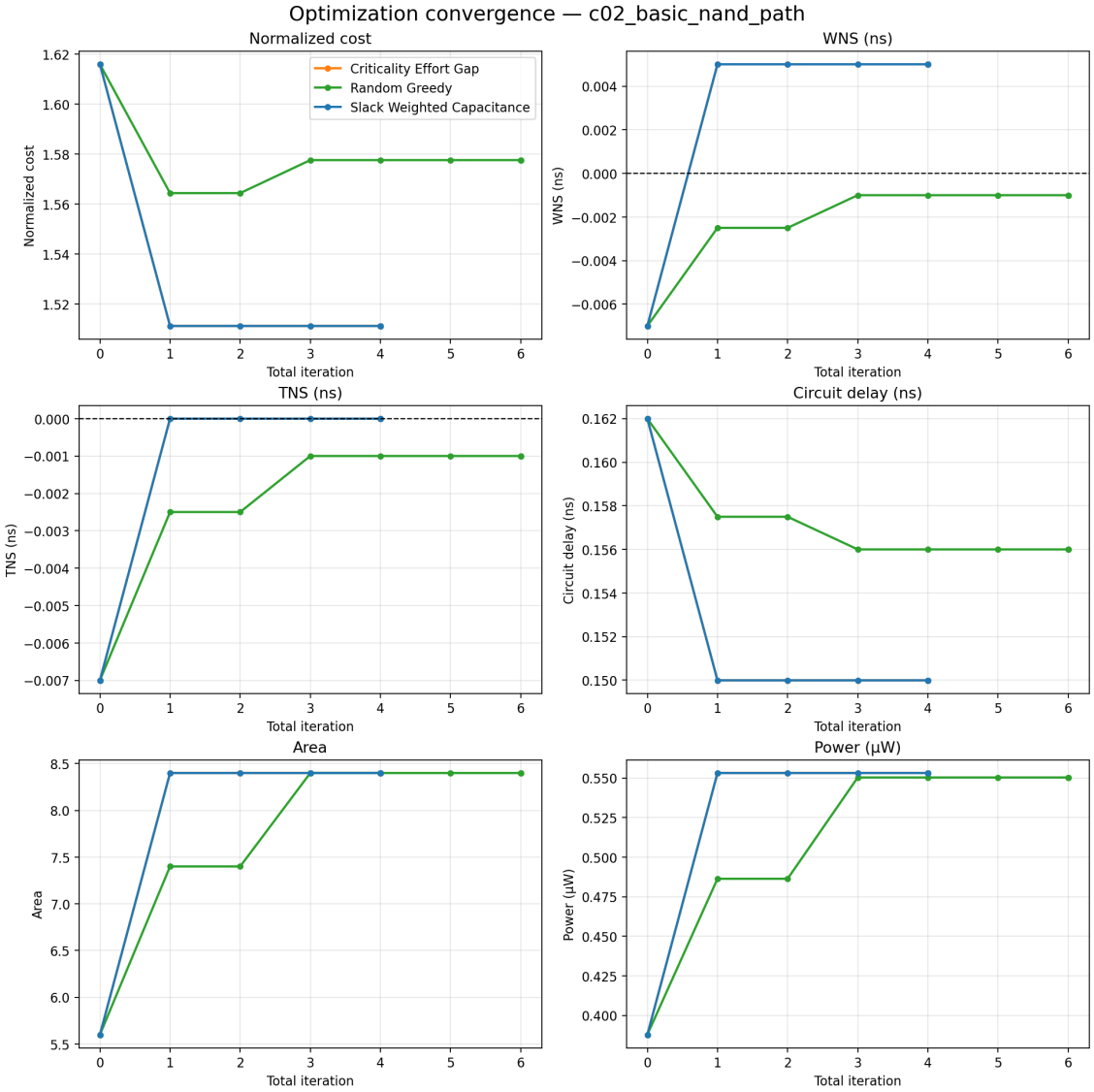



Figure 12.3: `c02_basic_nand_path`: normalized cost, WNS, TNS, circuit delay, area, and power versus accepted-change iteration, for all three sequential heuristics (section 8.5). The horizontal dashed line marks the compliance threshold on the WNS/TNS panels.

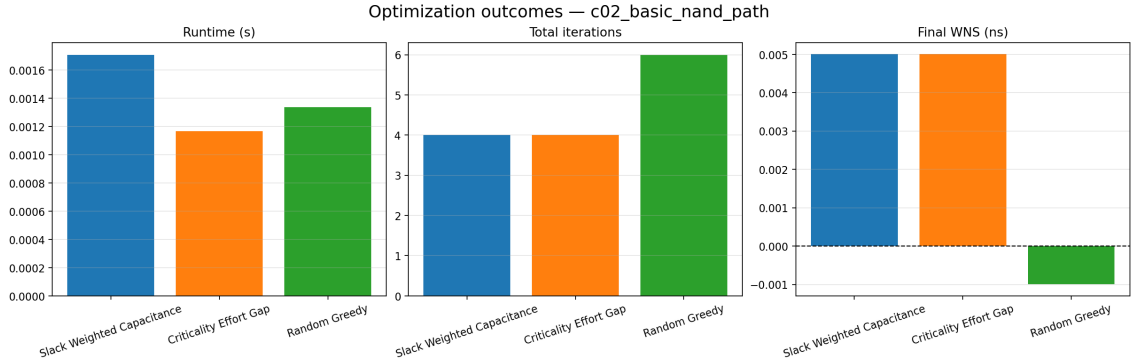



Figure 12.4: `c02_basic_nand_path`: final runtime, total iterations (accepted and rejected attempts combined), and final WNS per heuristic.

### 12.3.4 Case Study: Divergence Between the Two Logical-Effort Heuristics

`c13_large_reconvergent_network` (74 gates) is the one circuit in the suite on which slack-weighted capacitance and criticality effort gap disagree, and the recorded iteration logs make the mechanism traceable precisely rather than merely observable in the final numbers.

Criticality effort gap's search accepts exactly one change: at iteration 7, `G65: OR2_X1 →OR2_X2` is accepted under the *tied-WNS, TNS-improves* clause of the timing-repair acceptance rule (section 8.3) — WNS is left exactly unchanged at −0.0293 ns, while TNS improves substantially, from −0.2133 to −0.1717 ns. Eleven iterations later, at iteration 18, it proposes `G7: NOR3_X1→NOR3_X2` — a resize that, critically, *is* available to criticality effort gap, since it is a plain one-step-up candidate, not something requiring the logical-effort bracket — but this candidate is now **rejected** with reason *"candidate violates area or power constraints"*: the area consumed by the earlier `G65` change has left no further budget for it. No subsequent candidate improves WNS or TNS either, and the search terminates `no_feasible_change` with the circuit still at its original −0.0293 ns violation.

Slack-weighted capacitance's search proposes that exact same `G7: NOR3_X1→NOR3_X2` resize much earlier, at iteration 13, while the circuit is still at its pristine initial area budget (its own higher-scored earlier candidates were all rejected outright rather than accepted, so no budget had yet been spent). At that point it is well within budget and is accepted, cutting the violation's magnitude by **88.6%** (−0.0293 → −0.0033 ns); a second accepted change at iteration 35, `G6: NAND3_X1→NAND3_X2`, then closes the remaining gap entirely (WNS → +0.0037 ns, TNS → 0).

This is a second, independent confirmation of the same effect identified in section 12.3.3: the outcome is decided not by what a heuristic is theoretically permitted to consider — both heuristics have `G7` available — but by whether an earlier accepted change quietly consumes budget that a later, more consequential change will need. In this instance the effect is sharper than in the c02 case study, because criticality effort gap's early accepted change is

not even a step toward resolving the violation directly (it is WNS-neutral, accepted purely for a TNS improvement); slack-weighted capacitance's higher-scored ranking happens to defer any budget-consuming change until the one that actually matters, avoiding the trap entirely on this circuit.

Figure 12.5 shows this divergence directly: the orange (criticality effort gap) trace visibly plateaus at a still-negative WNS after its single early step, its cost curve flattening well above the blue (slack-weighted capacitance) trace, which instead makes a late but decisive descent to full compliance once its own higher-ranked candidates ahead of `G7` are exhausted.

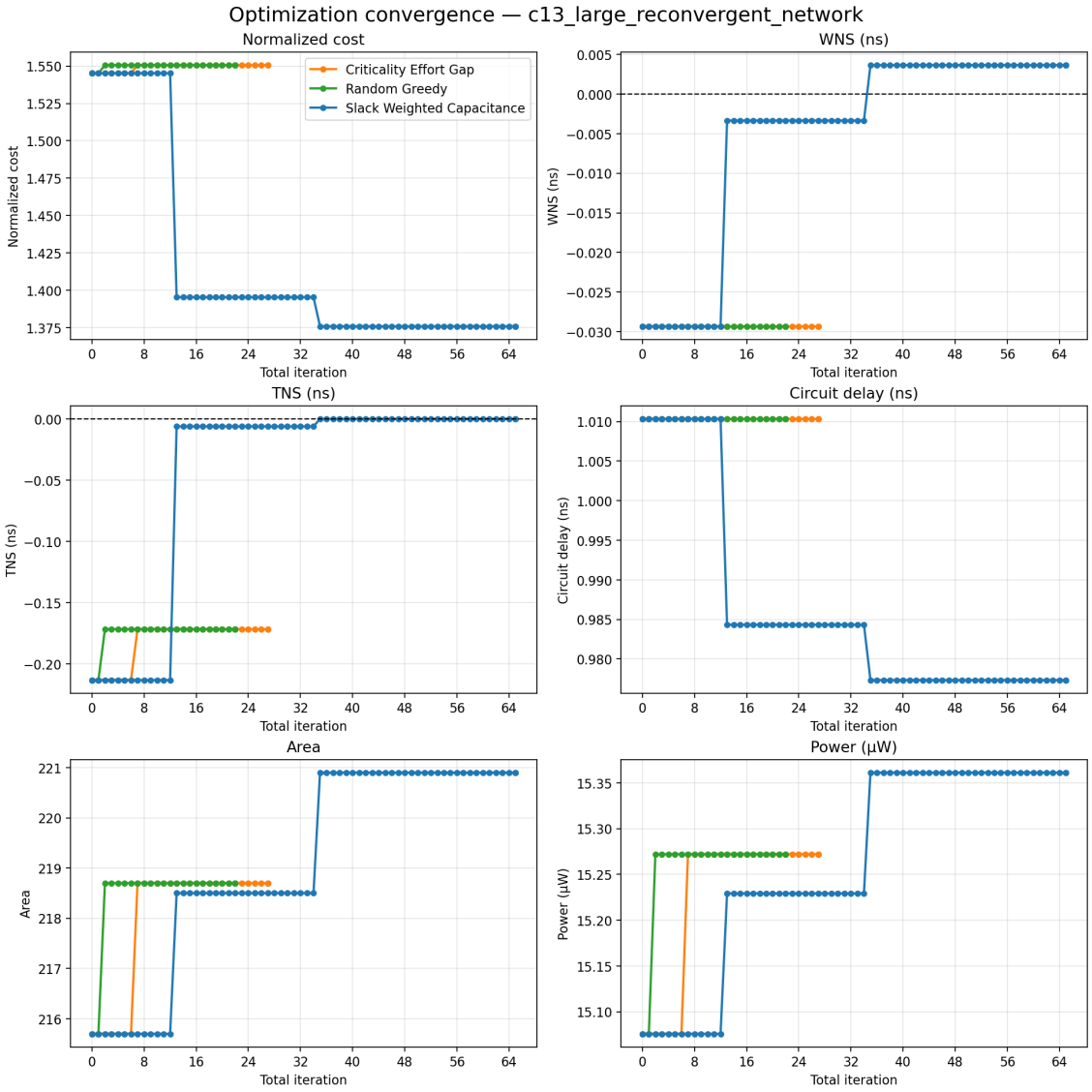



Figure 12.5: `c13_large_reconvergent_network`: convergence trace for all three sequential heuristics. Criticality effort gap's single early, WNS-neutral step (section 12.3.4) is visible as the orange trace's premature plateau above zero WNS.

## 12.4 Monte Carlo Statistical Analysis: `c12_monte_carlo_switching`

Table 12.5 reports the paired Monte Carlo comparison (chapter 9) for the one benchmark circuit with statistical analysis enabled, using $M = 3000$ samples, $\sigma_g = 0.03$, $\sigma_l = 0.05$, and random seed 1405.

| Metric | Pre-optimization | Post-optimization |
|---|---|---|
| Mean circuit delay (ns) | 0.3194 | 0.3041 |
| Std. dev. circuit delay (ns) | 0.0124 | 0.0116 |
| Mean WNS (ns) | -0.0044 | 0.0109 |
| Violation probability | 0.631 | 0.174 |
| Timing yield | 0.369 | 0.826 |
| Modal critical path (probability) | `D->G4->G5->G6->G7` (0.606) | `A->G1->G2->G3->G7` (0.628) |

Table 12.5: Paired Monte Carlo statistical timing comparison, pre- versus post-optimization, `c12_monte_carlo_switching`, $M = 3000$ samples.

Optimization improves timing yield from **36.9% to 82.6%** — a +45.7 percentage-point improvement — confirming that the nominal-corner WNS improvement reported in section 12.3 for this circuit translates into a substantial improvement in robustness to manufacturing variation, not merely a nominal-case artifact. Notably, the *identity* of the modal critical path also changes: under process variation, the pre-optimization design is most often limited by the path through `G4-G7` (driven by primary input `D`), while the post-optimization design is most often limited by an entirely different path through `G1-G3, G7` (driven by primary input `A`) — the sizing changes did not merely widen the margin on the original critical path, they shifted which path is critical at all, which is exactly the kind of effect a single nominal-corner STA run cannot reveal.

## 12.5 Synthetic Benchmark Suite Results

This section reports results from `complex_repair_suite`, a 60-case synthetic benchmark generated and evaluated with the framework of chapter 10. Unlike the fixed suite of section 12.3, which compares three heuristics on fifteen circuits (eleven of them with only a single-digit gate count), this suite is specifically configured to stress the optimizer on large, structurally diverse circuits, and evaluates all **four** heuristics — including brute force, absent from the fixed suite's reported comparison (section 8.5.1) — against the independent counterfactual oracle of section 10.3.1.

### 12.5.1 Suite Composition and Generation Reliability

The suite comprises 60 cases: 40 procedurally generated (structured-random topologies, 150–250 gates, 8–20 logic levels, targeting a 0.35 reconvergence probability and up to 6-

way fan-out) and 20 seeded from `c15_high_fanout_sizing_fabric` (section 12.2) under independently sampled electrical conditions and planted mutations. Every case's reference assignment was independently certified by an 8-restart, beam-width-64 multi-start beam search (`best_known_multi_start_beam`, section 10.2.2) before any downsizing mutation was planted, and each resulting initial (post-mutation) circuit was verified to have a negative WNS strictly within the configured area/power budget.

Generation reliability was total for this run: of 60 requested cases, all 60 were generated successfully, with **zero** entries in `generation_failures.csv` — every case succeeded on its first generation attempt, with no discarded partial attempts of any kind.

### 12.5.2 Wilson-Interval Repair Rate

Table 12.6 reports the case-level repair rate for each heuristic with a 95% Wilson confidence interval (section 10.3.3). Brute force and random greedy are run once and 20 times per case respectively (the latter to characterize its stochastic variance, per section 10.3.2); the two logical-effort-informed heuristics, being deterministic given a fixed circuit and candidate order (section 8.5), are run once per case.

| **Heuristic** | **Runs** | **Successes** | **Repair rate** | **95% Wilson CI** |
|---|---|---|---|---|
| Brute force | 60 | 60 | 100.0% | [93.98%, 100.00%] |
| Criticality effort gap | 60 | 60 | 100.0% | [93.98%, 100.00%] |
| Slack-weighted capacitance | 60 | 59 | 98.3% | [91.14%, 99.71%] |
| Random greedy | 1200 | 1134 | 94.5% | [93.06%, 95.65%] |

Table 12.6: Case-level repair rate with 95% Wilson confidence intervals (60 cases; random greedy evaluated with 20 repetitions per case, 1200 total runs).

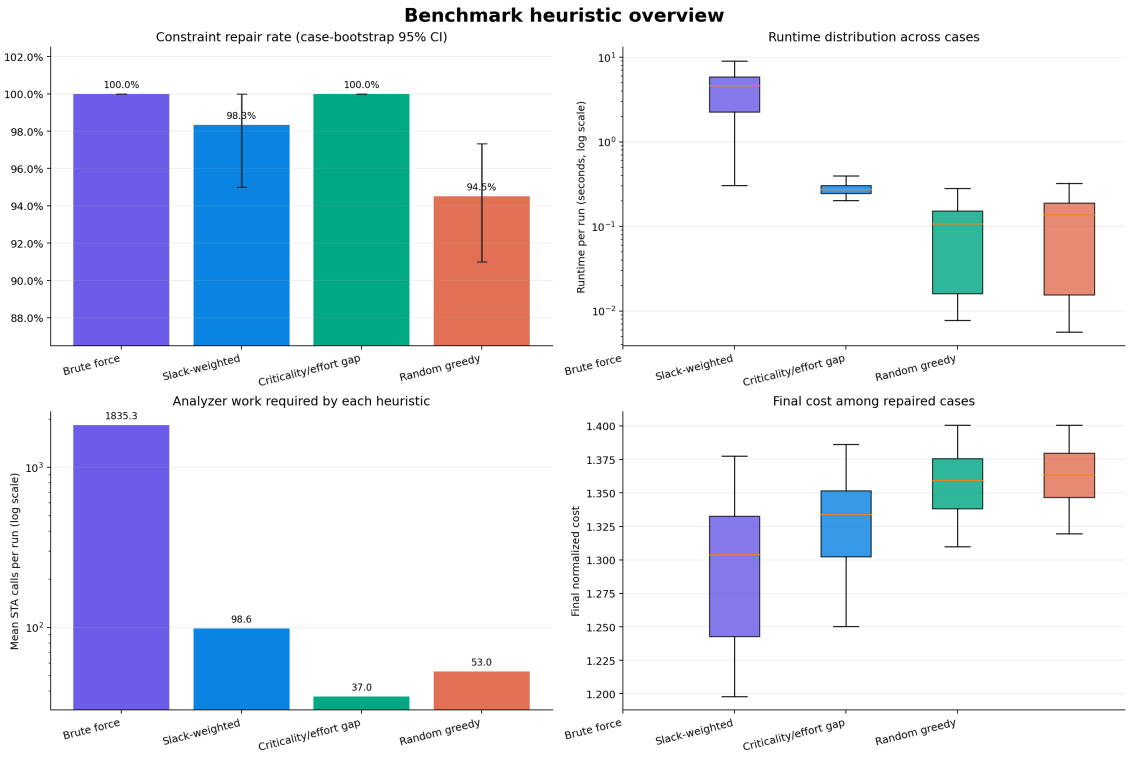



Figure 12.6: Benchmark heuristic overview: repair rate with 95% bootstrap confidence intervals, per-run wall-clock runtime distribution, mean STA calls per run (log scale), and final normalized cost among repaired cases only. Note the four-order-of-magnitude STA-call gap between brute force (≈ 1835 calls/run) and criticality effort gap (≈ 37 calls/run).

Three results are worth drawing out beyond the headline repair rates. First, criticality effort gap matches brute force's 100% repair rate on this suite exactly, and does so at ≈ 37 mean STA calls per run versus brute force's ≈ 1835 — a two-order-of-magnitude reduction in search cost for an identical repair outcome on this population. Second, slack-weighted capacitance — the heuristic found strictly more reliable than criticality effort gap on the fixed suite (section 12.3, where it alone repaired `c13_large_reconvergent_network`) — is here the *less* reliable of the two logical-effort-informed heuristics, with a single failure among its 60 cases. This is a genuine, population-dependent reversal, not a contradiction: the fixed suite's one divergence case (section 12.3.4) is a single 74-gate circuit, and this suite's 60 cases, averaging 150–250 gates, are a different and much larger sample; a report should not extrapolate either suite's heuristic ranking to circuits outside its own tested population. Third, random greedy's 1200-run aggregate repair rate (94.5%) sits noticeably below its *best-run* behavior implied by fig. 12.10 below — most of its failures are concentrated in specific cases, not spread uniformly across the suite.

The single slack-weighted-capacitance failure is traceable to a specific case: `case_0032` (198 gates, 18 logic levels, 7 primary inputs, 8 primary outputs, initial WNS = −0.0357 ns, reference WNS = +0.0051 ns). Both brute force (WNS = +0.0028 ns) and criticality effort gap (WNS = +0.0051 ns, matching the reference exactly) repair this case; slack-weighted capacitance terminates `no_feasible_change` at WNS = −0.0003 ns — a residual violation

of under half a picosecond, not a large or qualitative failure, but a failure by the strict binary repair-success criterion nonetheless.

### 12.5.3 Oracle-Measured Selection Accuracy and Regret

Table 12.7 reports, for every accepted decision each heuristic made, how it compares to the independent counterfactual oracle of section 10.3.1: whether the chosen gate was the oracle's single best-scoring gate at that state (*gate hit rate*), whether the chosen (gate, cell) pair was the oracle's single best exact move (*exact-move hit rate*), the median and worst-case timing regret in WNS relative to the best move the oracle could show was available, and how often the heuristic's final repaired cost beat the independently-certified reference assignment's own cost.

| Heuristic | Gate hit rate | Exact-move hit rate | Median regret (WNS, ns) | Worst regret (WNS, ns) | Beats ref. |
|---|---|---|---|---|---|
| BF | 7.9% | 7.7% | 0.0031 | 0.0365 | 100.0% |
| CEG | 4.1% | 2.8% | 0.0058 | 0.0803 | 85.0% |
| SWC | 1.6% | 1.4% | 0.0159 | 0.2030 | 100.0% |
| RG | 2.3% | 1.5% | 0.0210 | 0.2009 | 68.9% |

Table 12.7: Oracle-measured decision quality (BF = brute force, CEG = criticality effort gap, SWC = slack-weighted capacitance, RG = random greedy). Regret is the WNS gap between the heuristic's chosen move and the oracle's best available move at that decision point, evaluated over every state a heuristic actually visited (section 10.3.1); lower is better.

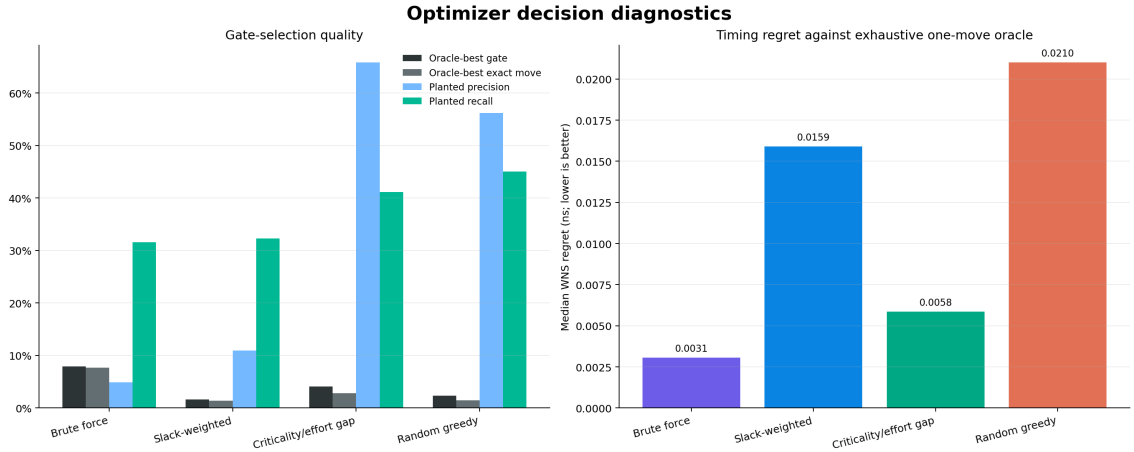



Figure 12.7: Left: gate-selection quality against the oracle's single best gate/exact move, and precision/recall of each heuristic's changed gates against the originally planted downsizing mutations. Right: median WNS regret against the exhaustive one-move oracle.

The gate- and exact-move hit rates are low in absolute terms for *every* heuristic, including brute force (7.9%/7.7%) — this is expected, not a sign of a broken oracle: the oracle's "best gate" is computed over *every* legal same-family replacement on *every* gate in

the circuit (section 10.3.1), while even brute force only proposes candidates on gates lying on a currently-reported critical path, within a logical-effort-derived capacitance bracket (section 8.5.1); a circuit this large routinely has several gates capable of producing an equally-good or negligibly-worse timing improvement, so an exact match to the oracle's single top pick is a strict criterion. The more informative signal is the *median regret* column: brute force's median regret (0.0031 ns) is roughly half of criticality effort gap's (0.0058 ns), which is in turn roughly a third of slack-weighted capacitance's (0.0159 ns) and random greedy's (0.0210 ns) — an ordering that does *not* match the repair-rate ordering of table 12.6 (where criticality effort gap tied brute force and beat slack-weighted capacitance). Repair rate measures whether a heuristic *eventually* reaches compliance at all; median regret measures how close to optimal its individual decisions are along the way. The two metrics are related but distinct, and a heuristic can rank well on one without ranking identically on the other.

The precision/recall of each heuristic's changed gates against the *originally planted* mutated gates (section 10.2.3) shows a striking, and initially counter-intuitive, pattern: criticality effort gap and random greedy — the two *weaker* heuristics by repair rate and regret — have substantially *higher* planted-gate precision (65.8% and 56.2%) than brute force and slack-weighted capacitance (4.9% and 10.9%), whose repairs touch the originally-mutated gates only rarely. This is consistent with, not contradictory to, the framework's own design rationale (section 10.3.1): a compliant, budget-respecting repair is very rarely unique, and a search that draws from a wider candidate bracket (brute force, slack-weighted capacitance) is more likely to find an equally valid fix at a *different* gate than the one originally downsized, while a search restricted to smaller, one-step-up moves (criticality effort gap, random greedy) is more likely to end up reversing something closer to the original mutation simply because fewer alternative routes are available to it. This is precisely why chapter 10 treats exact recovery of the planted mutation as a secondary diagnostic rather than the primary correctness signal: by that narrower measure alone, the two least effective heuristics on this suite would appear the most accurate.

Finally, *beats reference cost* shows a clean split along the same line: brute force and slack-weighted capacitance — both drawing from the full logical-effort capacitance bracket, the same candidate source available to the beam-search reference itself (section 10.2.2) — match or beat the reference's own final cost on *every* successful repair (100.0%), while criticality effort gap and random greedy, restricted to one-step-up moves, fall short of it on 15% and 31% of their successful repairs respectively.

### 12.5.4 Convergence and Runtime Scaling

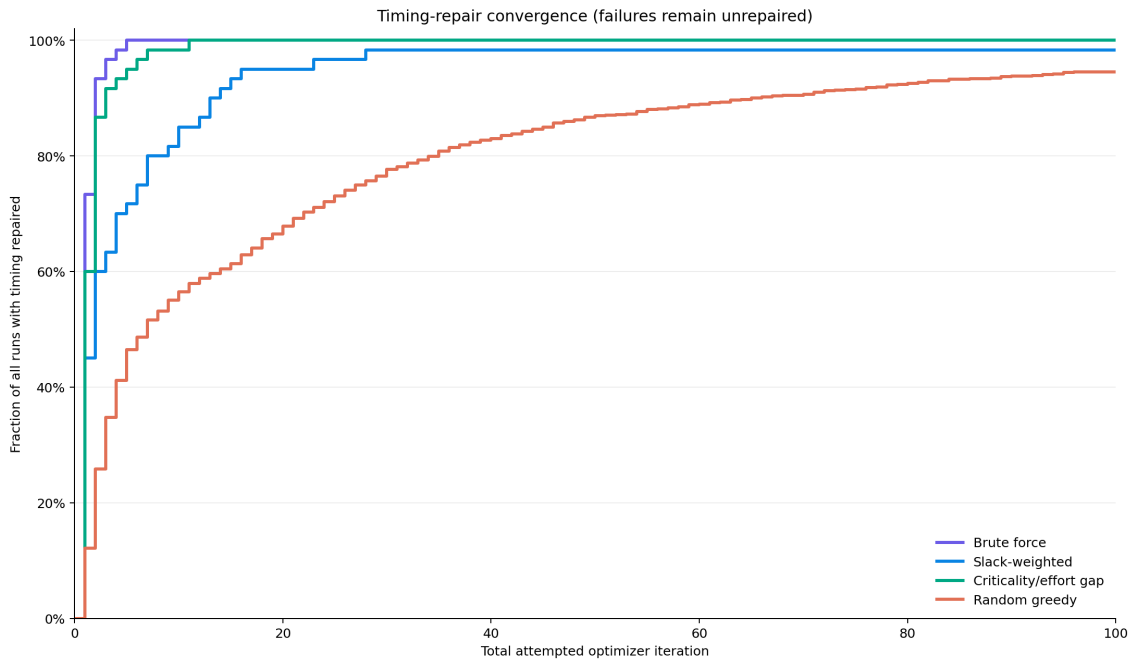



Figure 12.8: Fraction of all 1380 runs with timing repaired, as a function of total attempted optimizer iterations (accepted and rejected attempts combined). Runs that never repair remain flat at their final value indefinitely.

Figure 12.8 shows the same repair-rate story as table 12.6 unfolding over search effort rather than as a single endpoint number: criticality effort gap and brute force both reach their full 100% repair rate within about 10 iterations, slack-weighted capacitance climbs more gradually and plateaus just under 99%, and random greedy's much shallower climb — still visibly rising at 100 iterations — reflects the combined effect of its unranked candidate order (section 8.5.4) and the cases it never resolves at all within any of its repetitions.

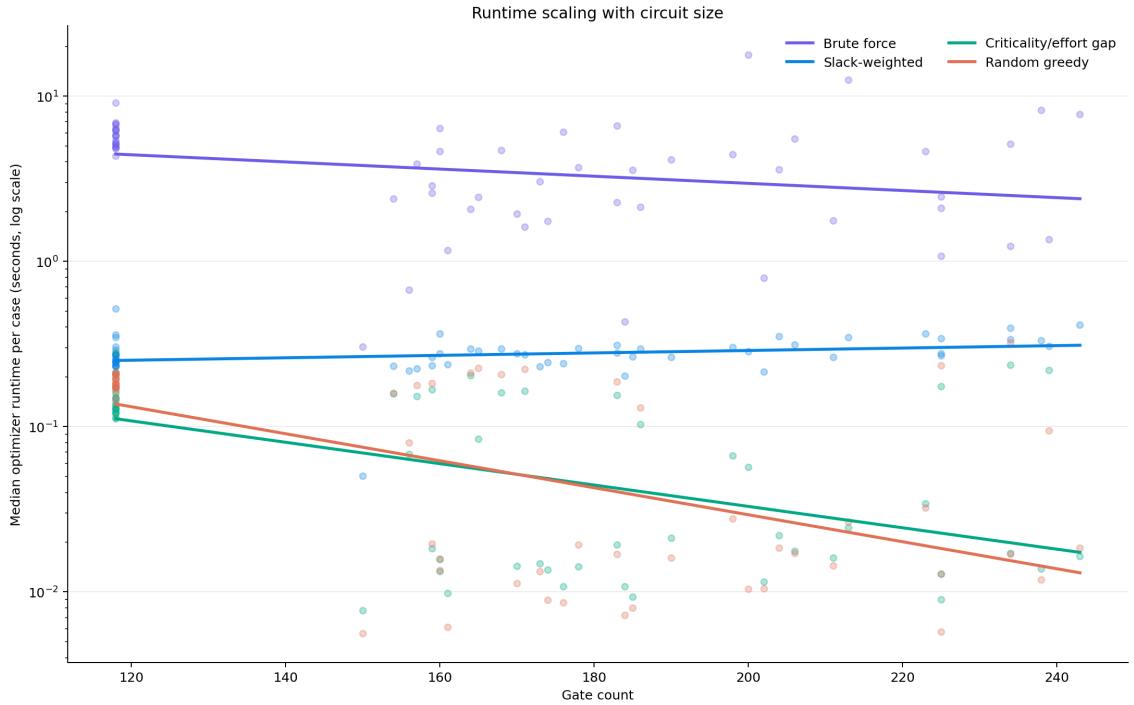



Figure 12.9: Median optimizer runtime per case versus gate count (log scale), with a fitted trend line per heuristic.

Runtime separates the four heuristics by roughly an order of magnitude at each tier (fig. 12.6, top right; mean and standard deviation in table 12.8): brute force averages $4.48 \pm 2.96$ s per run; slack-weighted capacitance, $0.281 \pm 0.063$ s; random greedy, $0.114 \pm 0.099$ s; and criticality effort gap, the fastest, $0.093 \pm 0.076$ s. Figure 12.9 shows brute force's runtime is roughly flat with circuit size over this suite's 150–250 gate range (its cost is dominated by the size of the candidate bracket it evaluates per iteration, not raw gate count), while the three sequential heuristics' runtimes visibly *decrease* with gate count — counter-intuitive at first glance, but consistent with the suite's generation process: larger generated circuits in this suite do not necessarily have proportionally more or deeper critical-path violations to repair, so the number of accepted-plus-rejected attempts needed does not scale directly with gate count either.

| **Heuristic** | **Mean runtime (s)** | **Std. dev. (s)** | **Mean STA calls** | **Mean final cost** |
|---|---|---|---|---|
| BF | 4.479 | 2.962 | 1835.3 | 1.291 |
| SWC | 0.281 | 0.063 | 98.6 | 1.329 |
| RG | 0.114 | 0.099 | 51.7 | 1.369 |
| CEG | 0.093 | 0.076 | 37.0 | 1.356 |

Table 12.8: Runtime, search cost, and mean final cost among repaired cases. BF = brute force, CEG = criticality effort gap, SWC = slack-weighted capacitance, RG = random greedy.

### 12.5.5 Per-Case Reliability

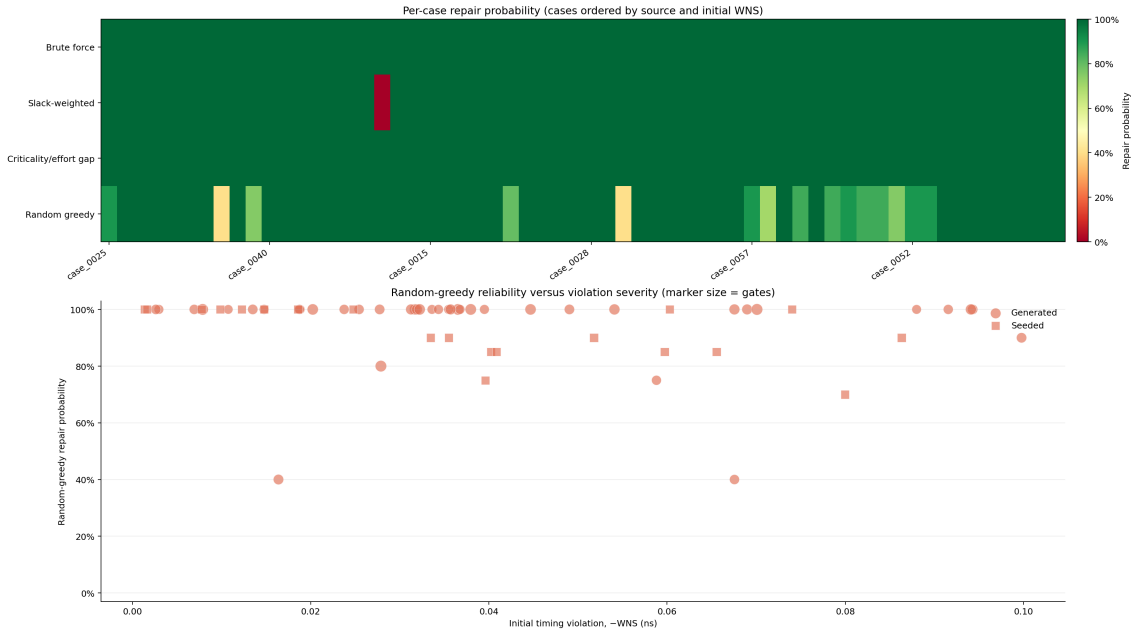



Figure 12.10: Top: per-case repair probability for a representative subset of cases, ordered by source and initial WNS (brute force and criticality effort gap are uniformly green — always successful; the single dark-red cell is slack-weighted capacitance's one failure, `case_0032`). Bottom: random greedy's repair probability (across its 20 repetitions) versus initial violation severity, marker size indicating gate count, split by source type.

Figure 12.10's lower panel makes the population-level pattern underlying section 12.5.6's severity stratification directly visible at the case level: random greedy's repair probability is 100% for the great majority of cases regardless of source, but drops into the 70–90% range for a specific subset of harder cases, with the two clearest outliers (both at 40%) occurring at moderate, not extreme, violation severity — reinforcing that repair difficulty for a one-step-up, unranked search depends on circuit structure as much as on how large the initial violation is.

### 12.5.6 Stratified Breakdown

Table 12.9 breaks the repair rate down by three groupings computed automatically by the evaluation framework (section 10.3.3): circuit depth, reconvergence level, and initial violation severity. (This suite's cases fall into only two buckets each for depth and reconvergence — *medium*/*deep* and *medium*/*high* respectively — reflecting the generation configuration's target ranges, section 12.5.1; no *shallow* or *low-reconvergence* cases were generated in this run.)

| Grouping | Stratum | Cases | BF | CEG | SWC | RG |
|---|---|---|---|---|---|---|
| Depth | Medium | 10 | 100.0% | 100.0% | 100.0% | 94.0% |
| | Deep | 50 | 100.0% | 100.0% | 98.0% | 94.6% |
| Reconvergence | Medium | 5 | 100.0% | 100.0% | 100.0% | 100.0% |
| | High | 55 | 100.0% | 100.0% | 98.2% | 94.0% |
| Violation severity | Mild | 8 | 100.0% | 100.0% | 100.0% | 100.0% |
| | Moderate | 34 | 100.0% | 100.0% | 97.1% | 95.4% |
| | Severe | 18 | 100.0% | 100.0% | 100.0% | 90.3% |

Table 12.9: Repair rate by circuit depth, reconvergence level, and initial violation severity. BF = brute force, CEG = criticality effort gap, SWC = slack-weighted capacitance, RG = random greedy.

Brute force and criticality effort gap are unaffected by any stratum in this suite, repairing 100% of cases regardless of depth, reconvergence, or severity. Random greedy is the only heuristic whose repair rate degrades meaningfully with reconvergence (from 100% at medium reconvergence to 94.0% at high) and with violation severity (from 100% on mild violations down to 90.3% on severe ones) — exactly where a logical-effort-free, unranked search is least likely to stumble onto the gate a severe or highly-reconvergent violation actually needs. Slack-weighted capacitance shows a smaller version of the same pattern on depth (100% → 98.0%) and reconvergence (100% → 98.2%), consistent with `case_0032` — its one failure — being both a deep and a reconvergent circuit. Somewhat counter-intuitively, slack-weighted capacitance's repair rate on *severe* violations (100%) is *higher* than on *moderate* ones (97.1%): the harder-looking severity axis is not the one that predicts its failure on this suite, depth and reconvergence are. This is exactly the kind of heuristic-specific, non-obvious failure pattern the fixed fifteen-circuit suite is too small to reveal, and is the clearest evidence in this report that a large, structurally-varied synthetic suite adds information the fixed suite alone cannot provide.

## 12.6 Complete Per-Circuit Figures

Sections 12.3.3 and 12.3.4 showed the pre-/post- optimization circuit diagrams and convergence/outcome plots for the two circuits most useful for illustrating *why* the heuristics diverge. The identical set of four figures — pre-optimization diagram, post-optimization diagram, convergence trace, and outcomes summary — is generated for every one of the fifteen valid benchmark circuits; Appendix B reproduces the complete set for systematic reference.

# Chapter 13

# Conclusion

This report has documented StaSizer, a static timing analysis and logical-effort-guided gate-sizing tool built to the specification of the Advanced VLSI Design course project, together with two substantial additions beyond that specification: a synthetic benchmark generation and oracle-based evaluation framework, and an interactive topology viewer. The core analysis pipeline — structural validation, DAG construction, RC-based rise/fall-separated delay modeling with unate-aware timing arcs, forward and backward static timing analysis, logical-effort path analysis, power and area estimation, and Monte Carlo statistical timing analysis — was traced in this report down to the specific class and function names that implement each stage.

On the fixed benchmark suite, the logical-effort-guided canonical heuristic resolved 13 of 14 originally-violating circuits, compared to 7 of 14 for a logical-effort-free random-greedy baseline, while simultaneously issuing fewer total STA calls and reaching a lower final cost on thirteen of fifteen circuits (chapter 12). Two case studies, traced directly against the recorded optimizer iteration logs rather than inferred from final outcomes alone, showed that this advantage comes from a better-informed *search order* interacting favorably with the area/power budget, not from any difference in what a given heuristic is permitted to consider. The one Monte Carlo–instrumented circuit demonstrated that a nominal-corner timing improvement from optimization corresponded to a real, substantial improvement in statistical timing yield (from 36.9% to 82.6%), and that the sizing changes shifted which path is critical under process variation, not merely how much margin the original critical path retained.

# Appendix A

# Full Equation Reference

This appendix collects every governing equation used in this report, in one place, for quick reference during the presentation.

$$C_{\text{load},i} = \sum_{(j,p)\,\in\,\text{Fanout}(i)} C_{\text{input},j,p} + C_{\text{external},i} \qquad \text{(eq. (2.1))}$$

$$D_{\text{rise/fall},i} = D_{\text{intrinsic,rise/fall},i} + \kappa\, R_{\text{rise/fall},i}\, C_{\text{load},i} \qquad \text{(eqs. (2.2) and (2.3))}$$

$$\text{AT}_i^s = \max_{p,\,j,\,(s',s)\in\mathcal{T}_{i,p}} \left(\text{AT}_j^{s'} + D_i^s\right) \qquad \text{(eq. (2.4))}$$

$$\text{RT}_j^{s'} = \min_{(i,p)\in\text{Fanout}(j),\,(s',s)\in\mathcal{T}_{i,p}} \left(\text{RT}_i^s - D_i^s\right) \qquad \text{(eq. (2.5))}$$

$$S_i^s = \text{RT}_i^s - \text{AT}_i^s, \quad S_i = \min(S_i^{\text{rise}}, S_i^{\text{fall}}) \qquad \text{(eq. (2.6))}$$

$$\text{WNS} = \min_{o\in\mathcal{O}} S_o, \quad \text{TNS} = \sum_{o\in\mathcal{O}} \min(0, S_o) \qquad \text{(eq. (2.7))}$$

$$b_i = \frac{C_{\text{onpath},i} + C_{\text{offpath},i}}{C_{\text{onpath},i}}, \quad h_i = \frac{C_{\text{onpath},i}}{C_{\text{in},i}}, \quad f_i = g_i b_i h_i \qquad \text{(eq. (2.8))}$$

$$G = \prod_i g_i, \quad B = \prod_i b_i, \quad H = \frac{C_{L,\text{path}}}{C_{\text{in},1}}, \quad F = GBH \qquad \text{(eq. (2.9))}$$

$$\hat{f} = F^{1/N}, \quad D_{LE,\min} = \tau\left(NF^{1/N} + P\right) \qquad \text{(eq. (2.10))}$$

$$C_{\text{in},i}^* = \frac{g_i C_{\text{total},i}}{\hat{f}} \qquad \text{(eq. (2.11))}$$

$$A_{\text{total}} = \sum_i A_i, \quad C_{\text{sw},i} = C_{\text{load},i} + C_{\text{internal},i}, \quad P_{\text{dynamic}} = \sum_i \alpha_i C_{\text{sw},i} V_{DD}^2 f \qquad \text{(eq. (2.12))}$$

$$P_{\text{total}} = P_{\text{dynamic}} + P_{\text{leakage}} \qquad \text{(eq. (2.13))}$$

$$D_{i,s}^{(k)} = D_{i,s,\text{nom}}\left(1 + \sigma_g Z_g^{(k)} + \sigma_l Z_i^{(k)}\right) \qquad \text{(eq. (2.14))}$$

$$\hat{P}_{\text{violation}} = \frac{1}{M}\sum_k I_{\text{violation}}^{(k)}, \quad \hat{Y}_{\text{timing}} = 1 - \hat{P}_{\text{violation}} \qquad \text{(eq. (2.15))}$$

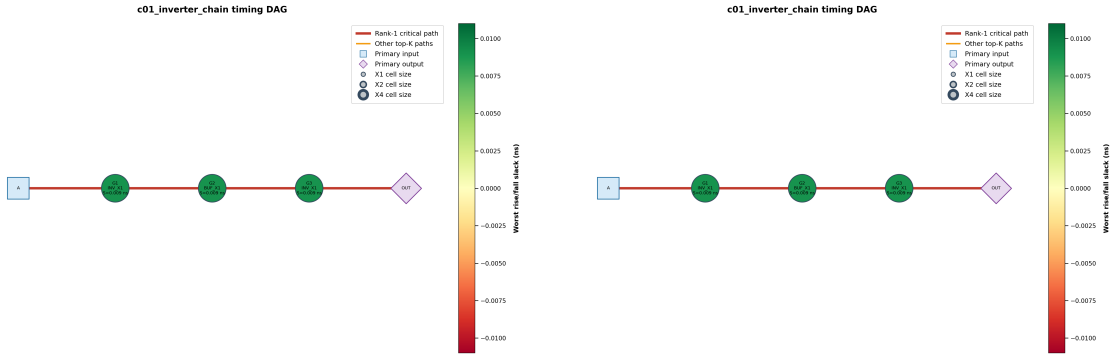
$$J = w_D \frac{D_{\text{circuit}}}{D_{\text{ref}}} + w_P \frac{P_{\text{total}}}{P_{\text{ref}}} + w_A \frac{A_{\text{total}}}{A_{\text{ref}}} + \lambda \frac{\max(0, -\text{WNS})}{T_{\text{ref}}} \qquad \text{(eq. (8.1))}$$

# Appendix B

# Complete Per-Circuit Figures

For every one of the fifteen valid benchmark circuits, this appendix reproduces the pre-optimization circuit diagram, the post-optimization circuit diagram (canonical slack-weighted-capacitance heuristic), the convergence trace of all three sequential heuristics (section 8.5), and the final outcomes summary, in that order. Sections 12.3.3 and 12.3.4 discuss `c02_basic_nand_path` and `c13_large_reconvergent_network` specifically; they are reproduced here again for completeness alongside the rest of the suite. The circuit-diagram renderings of the three largest circuits (`c13`–`c15`, 74–118 gates) are included for completeness but are more legibly explored via the interactive topology viewer (chapter 11) than at the fixed page width used here.

## B.1 `c01_inverter_chain` (3 gates)




(a) Pre-optimization (b) Post-optimization

Figure B.1: `c01_inverter_chain`: circuit diagrams, nodes colored by slack, critical path highlighted.

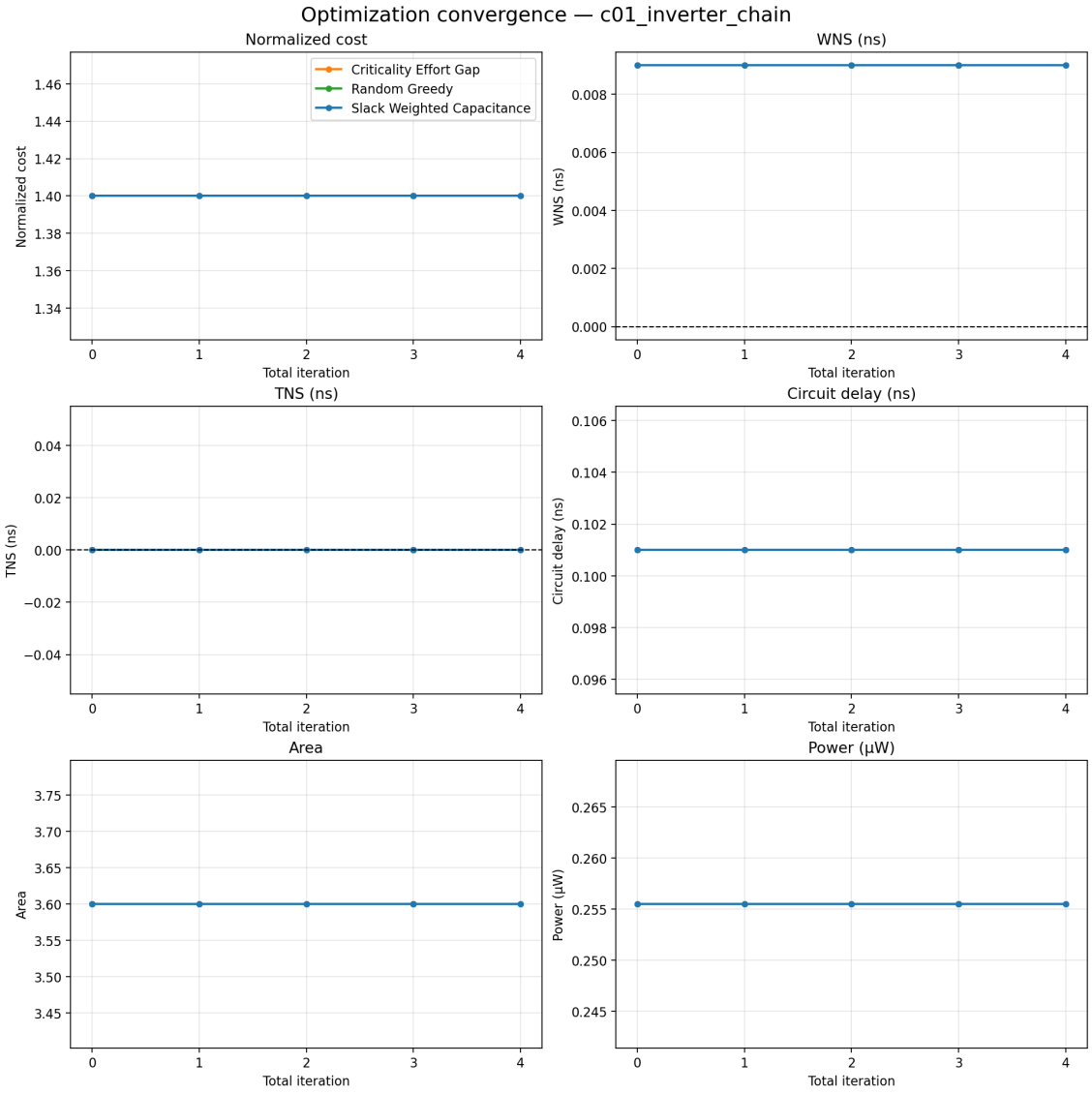



Figure B.2: `c01_inverter_chain`: optimization convergence trace.

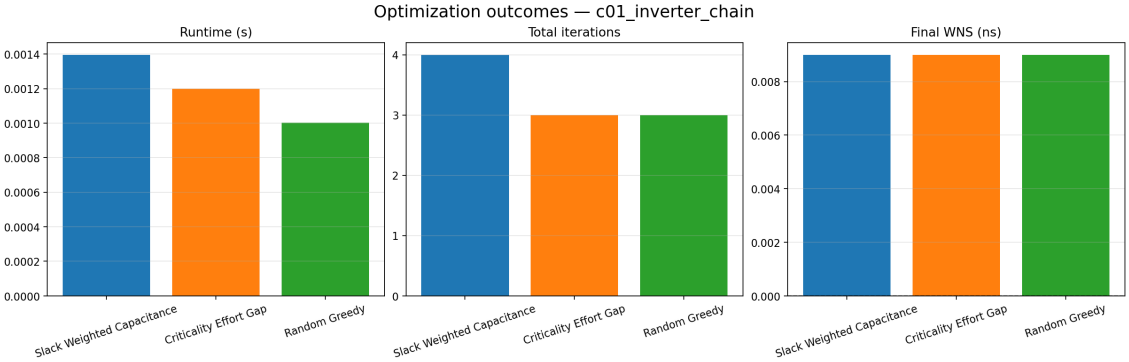
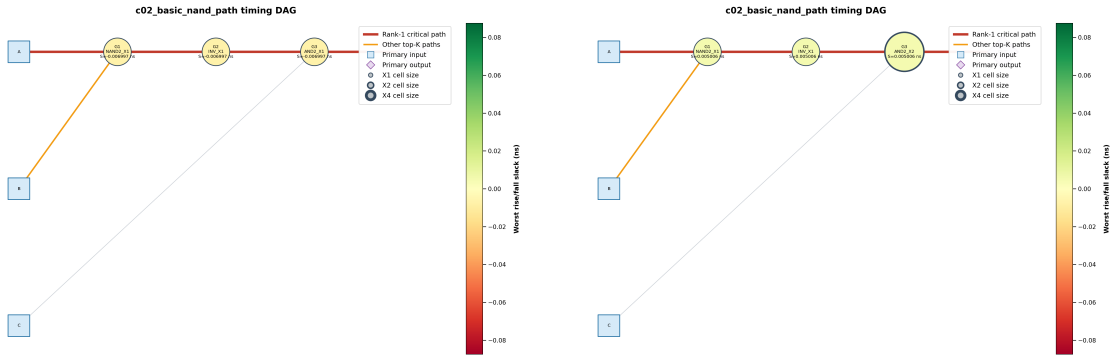



Figure B.3: `c01_inverter_chain`: optimization outcomes summary.

## B.2 c02_basic_nand_path (3 gates)




(a) Pre-optimization (b) Post-optimization

Figure B.4: `c02_basic_nand_path`: circuit diagrams, nodes colored by slack, critical path highlighted.

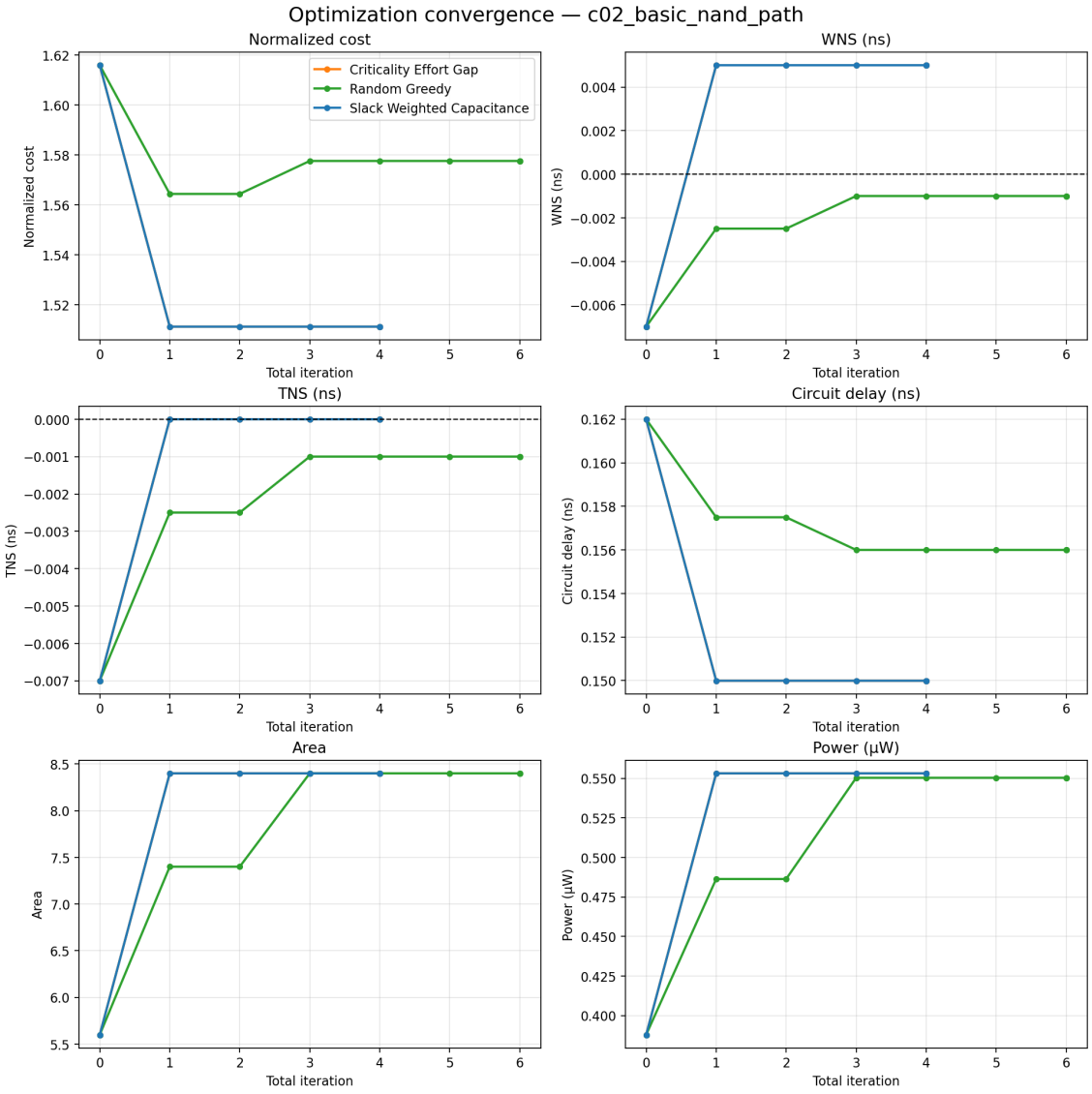



Figure B.5: `c02_basic_nand_path`: optimization convergence trace.

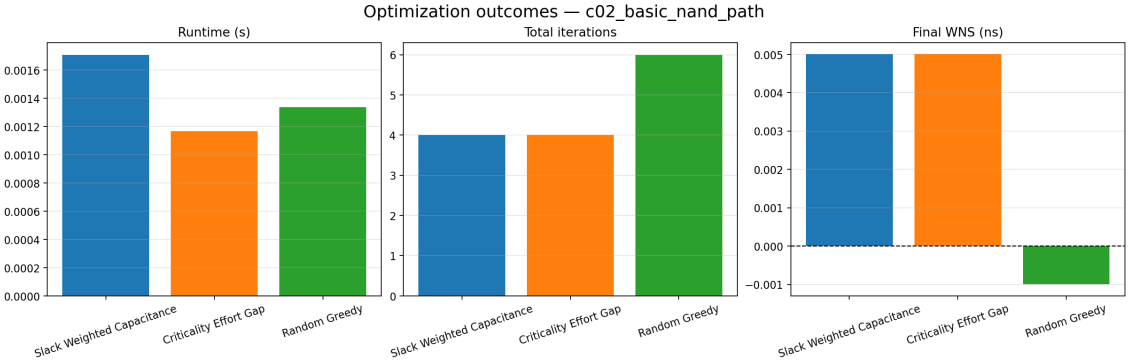
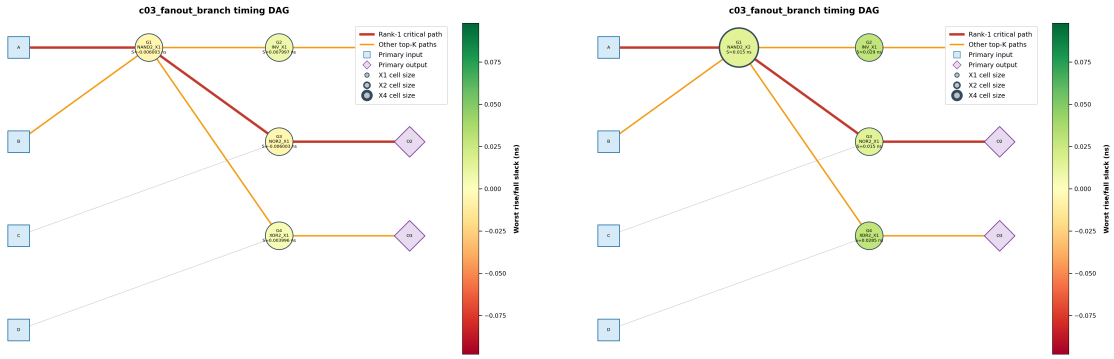



Figure B.6: `c02_basic_nand_path`: optimization outcomes summary.

## B.3 `c03_fanout_branch` (4 gates)




(a) Pre-optimization (b) Post-optimization

Figure B.7: `c03_fanout_branch`: circuit diagrams, nodes colored by slack, critical path highlighted.

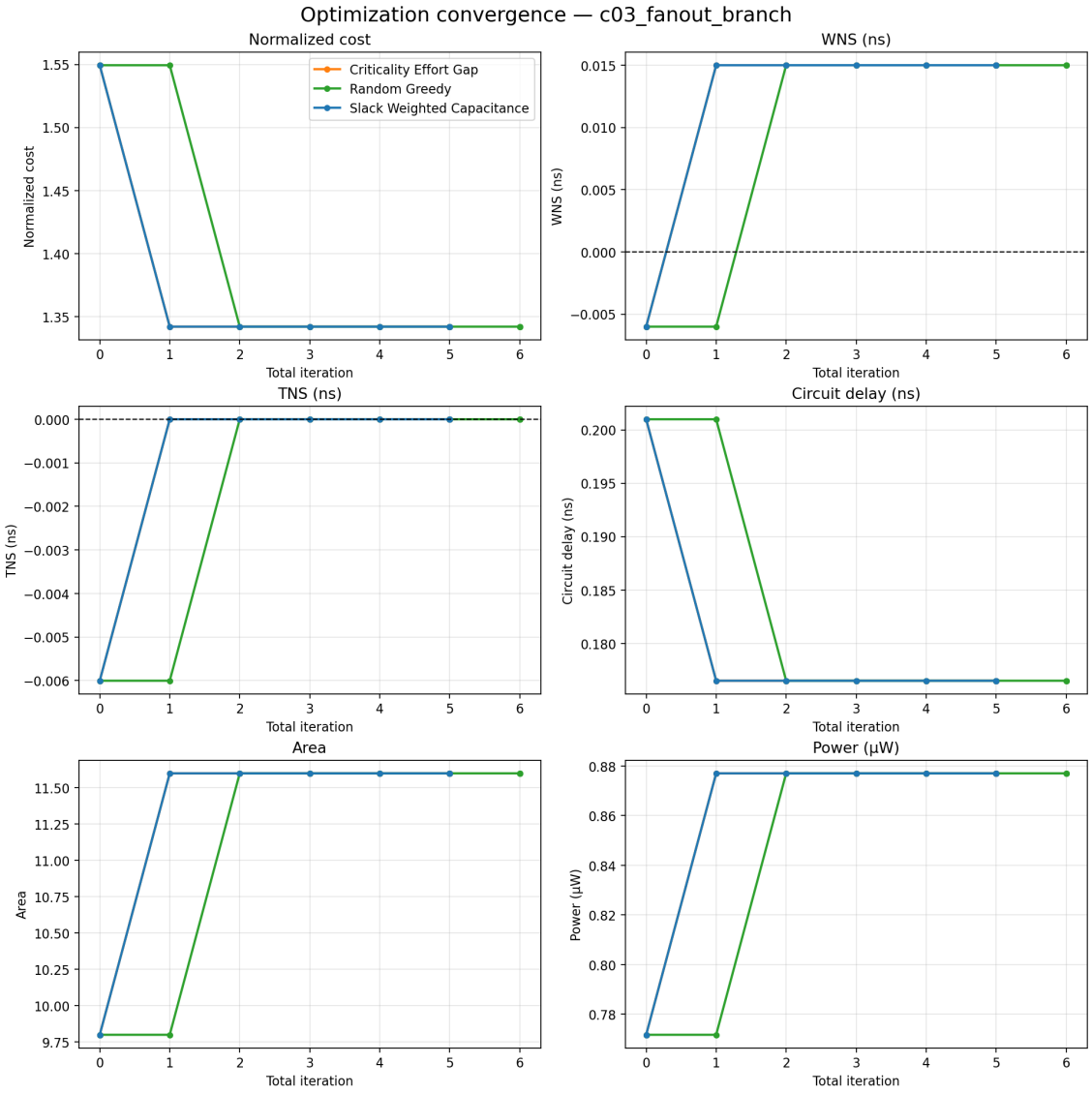



Figure B.8: `c03_fanout_branch`: optimization convergence trace.

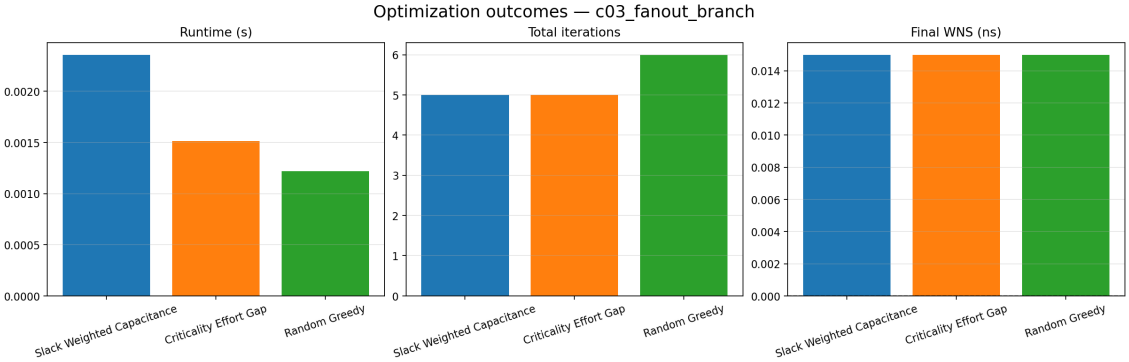
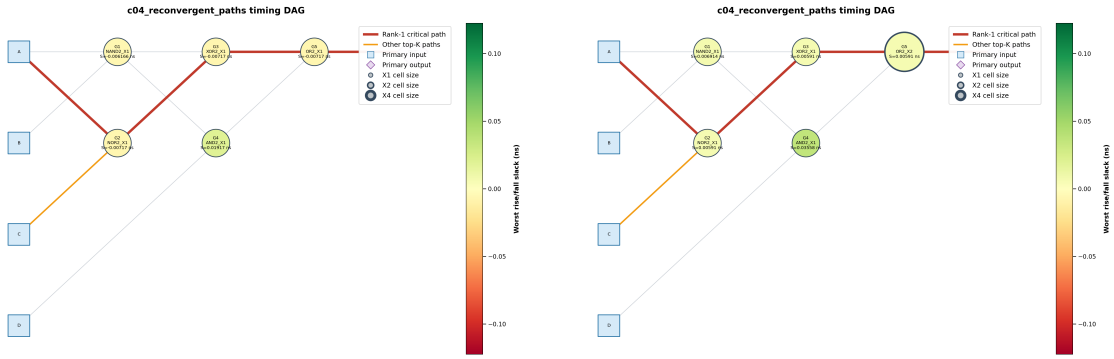



Figure B.9: `c03_fanout_branch`: optimization outcomes summary.

## B.4 c04_reconvergent_paths (5 gates)




(a) Pre-optimization (b) Post-optimization

Figure B.10: `c04_reconvergent_paths`: circuit diagrams, nodes colored by slack, critical path highlighted.

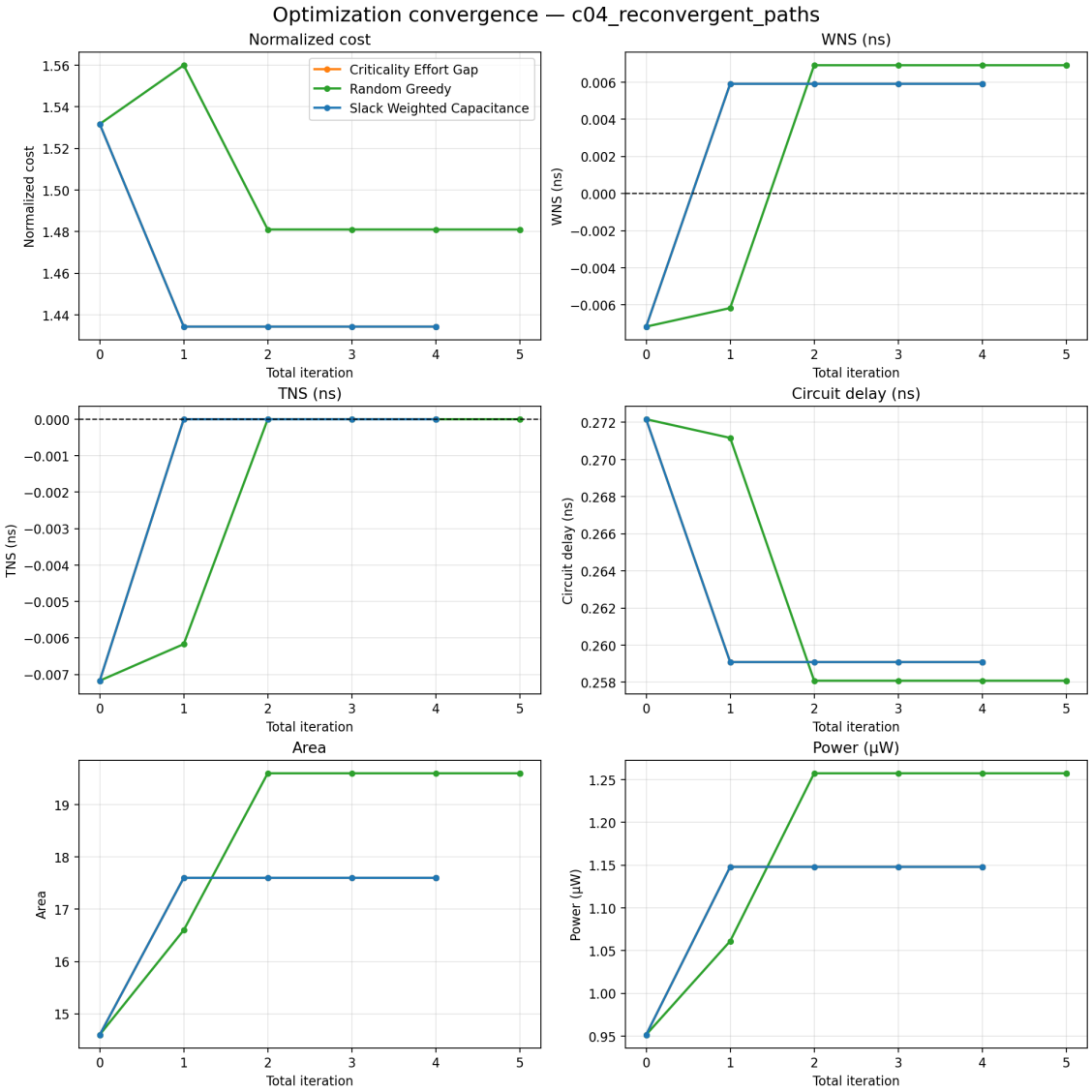



Figure B.11: `c04_reconvergent_paths`: optimization convergence trace.

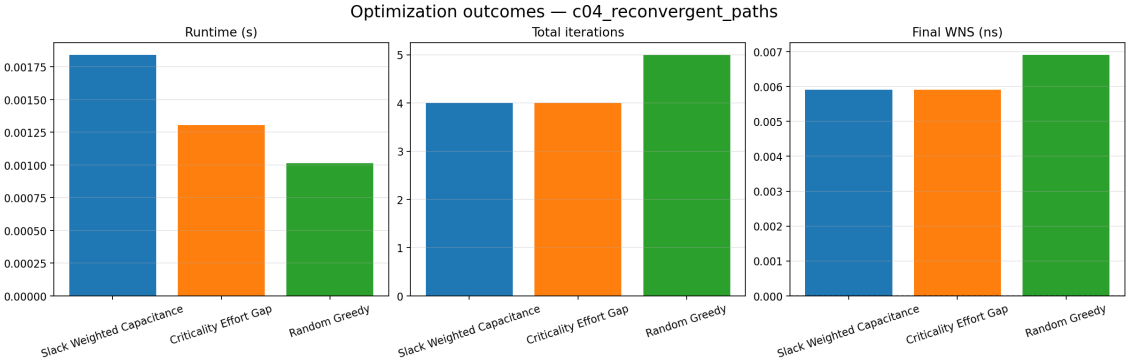
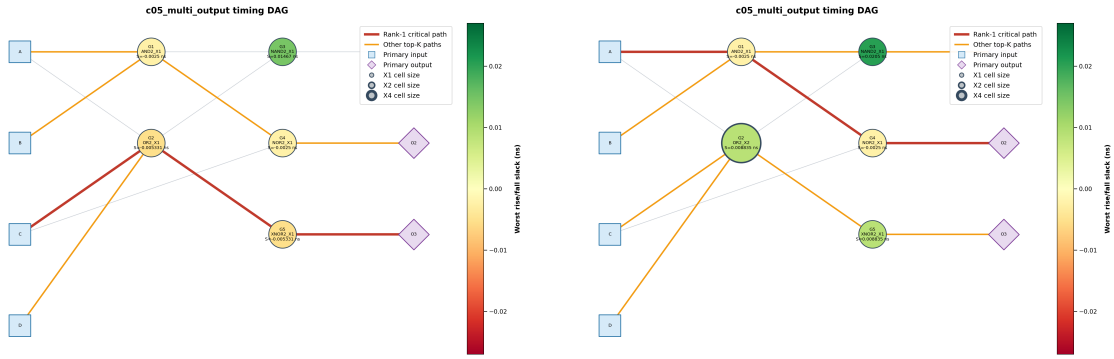



Figure B.12: `c04_reconvergent_paths`: optimization outcomes summary.

## B.5 `c05_multi_output` (5 gates)




(a) Pre-optimization (b) Post-optimization

Figure B.13: `c05_multi_output`: circuit diagrams, nodes colored by slack, critical path highlighted.

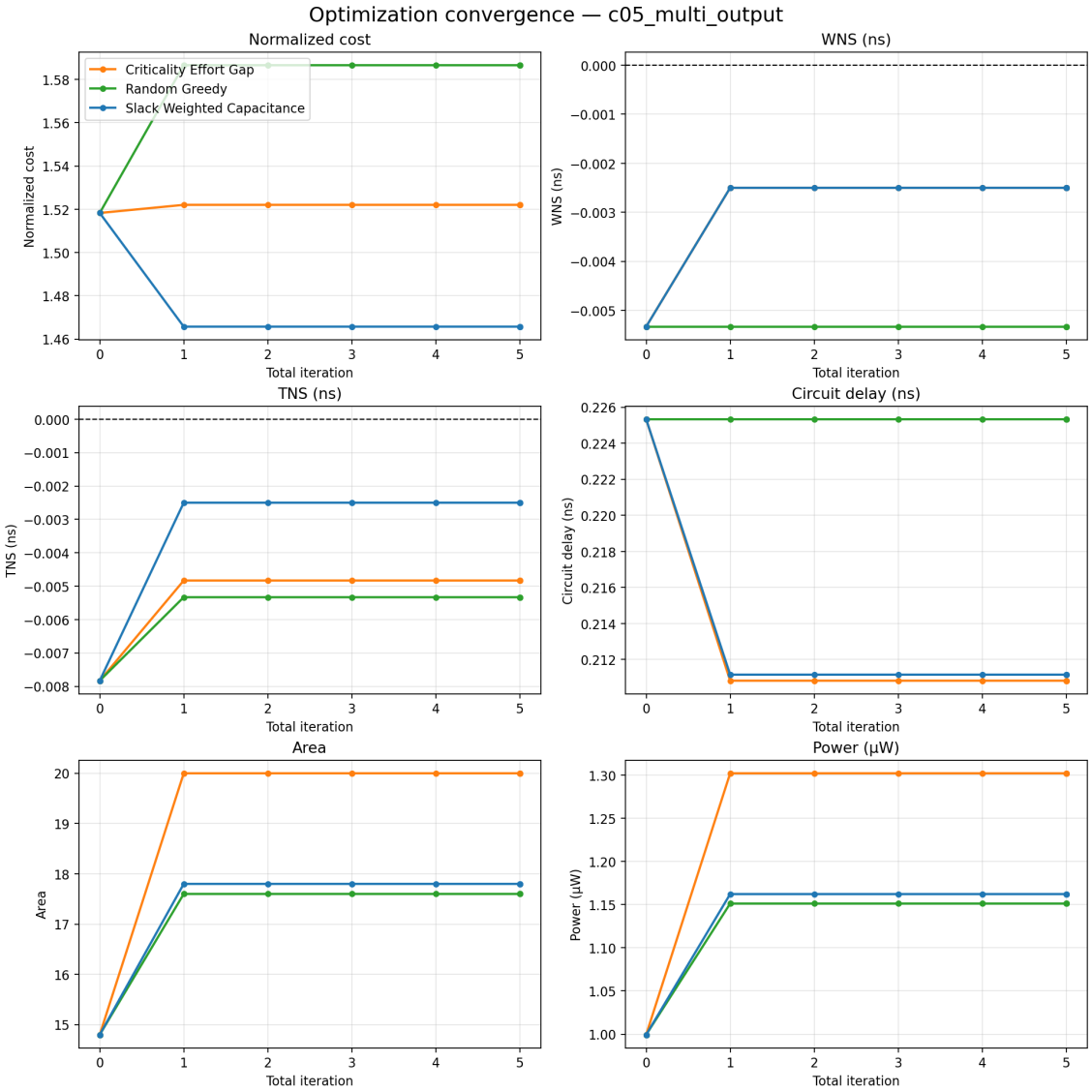



Figure B.14: `c05_multi_output`: optimization convergence trace.

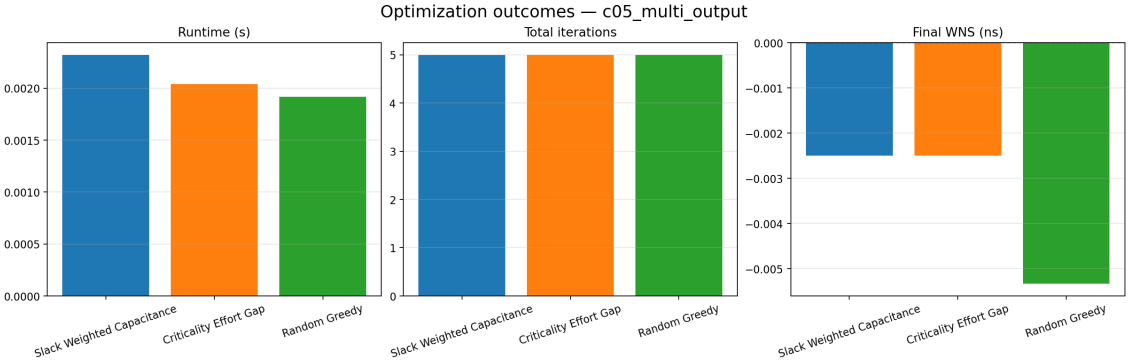
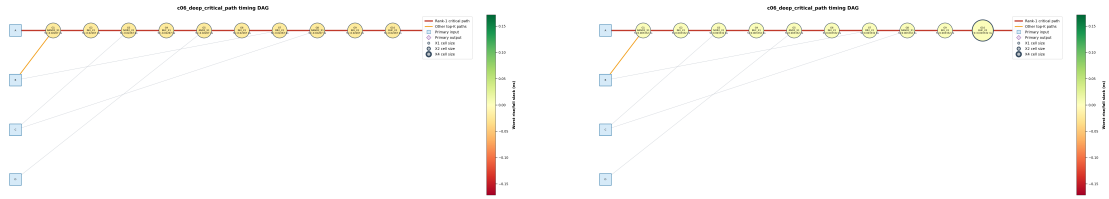



Figure B.15: `c05_multi_output`: optimization outcomes summary.

## B.6 c06_deep_critical_path (10 gates)



(a) Pre-optimization (b) Post-optimization

Figure B.16: c06_deep_critical_path: circuit diagrams, nodes colored by slack, critical path highlighted.

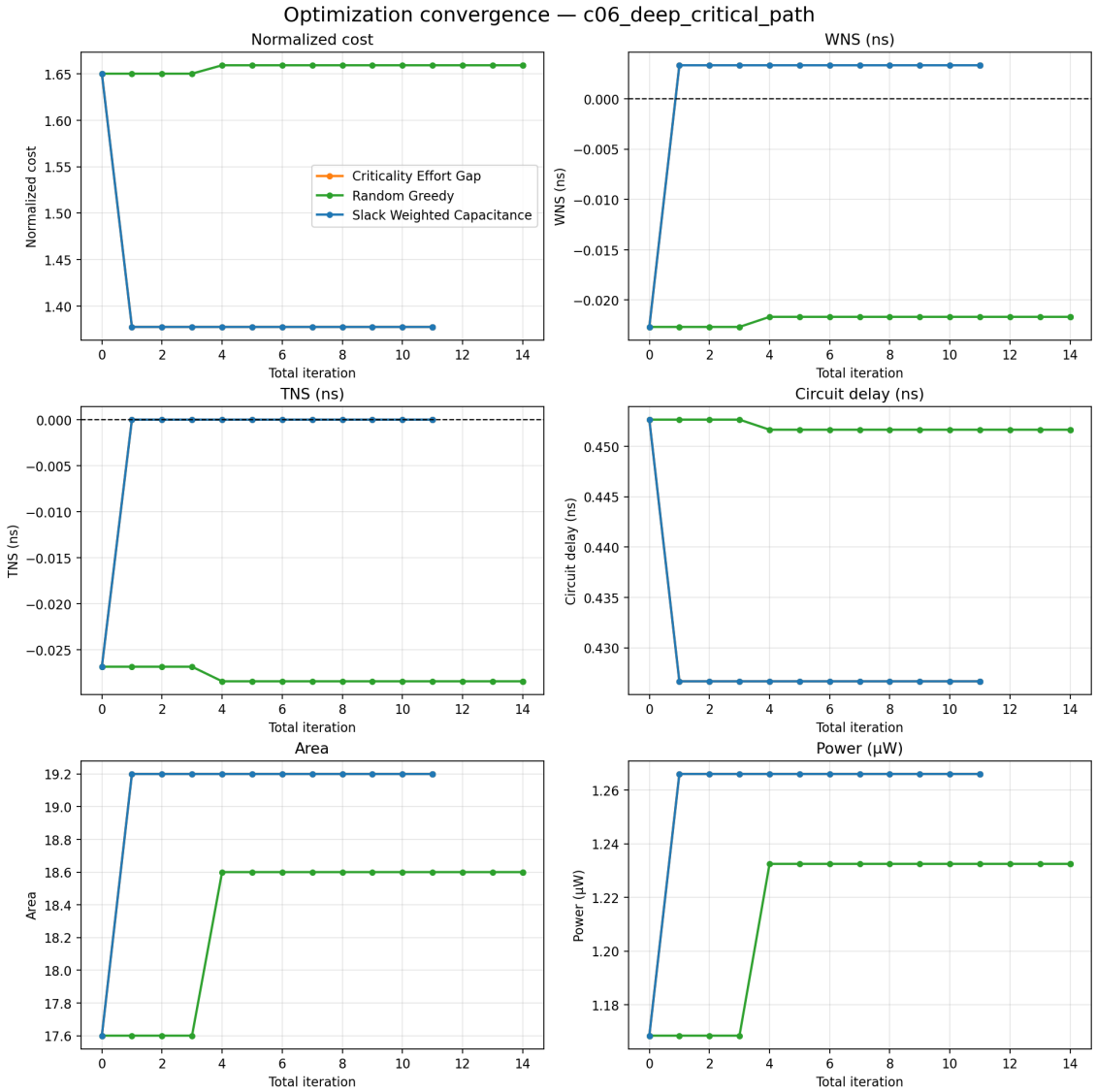



Figure B.17: c06_deep_critical_path: optimization convergence trace.

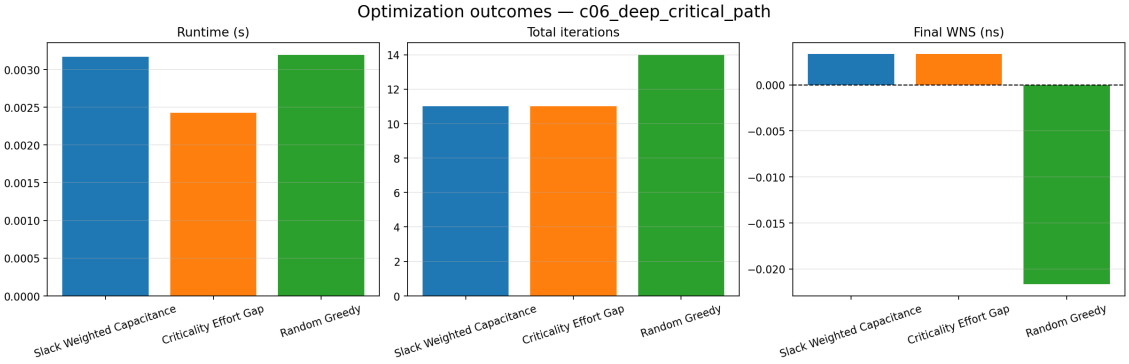
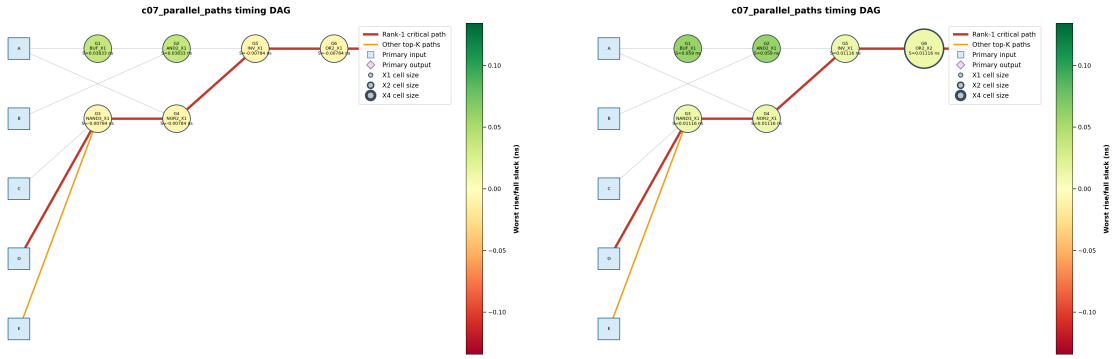



Figure B.18: `c06_deep_critical_path`: optimization outcomes summary.

## B.7 c07_parallel_paths (6 gates)




(a) Pre-optimization (b) Post-optimization

Figure B.19: c07_parallel_paths: circuit diagrams, nodes colored by slack, critical path highlighted.

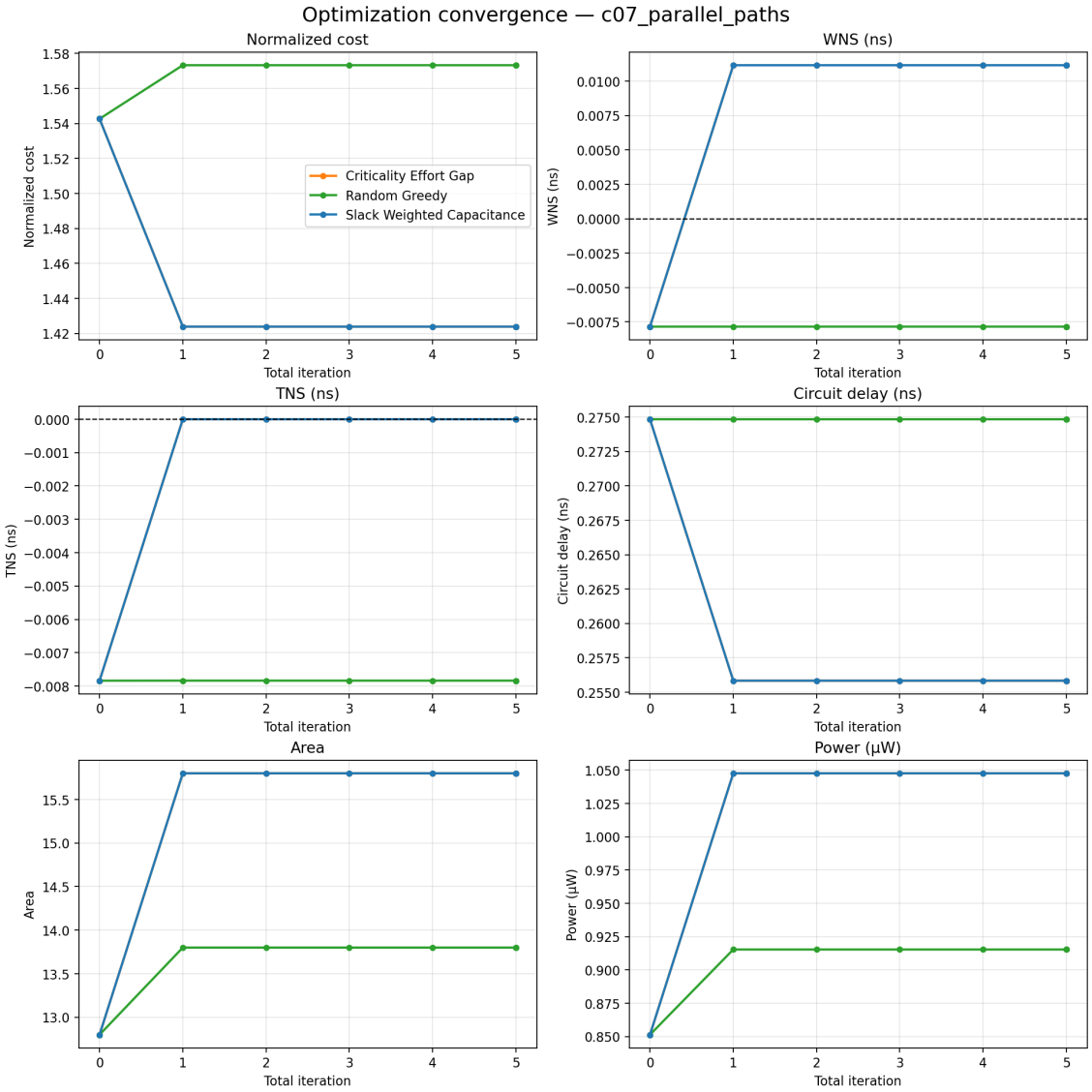



Figure B.20: `c07_parallel_paths`: optimization convergence trace.

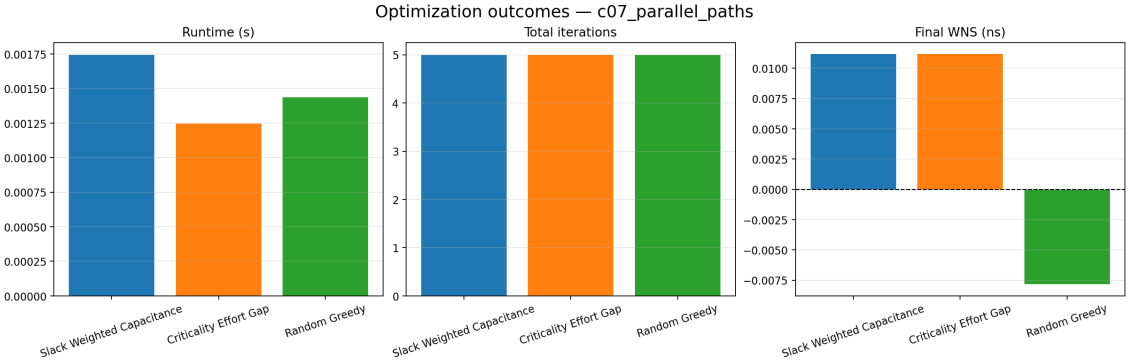
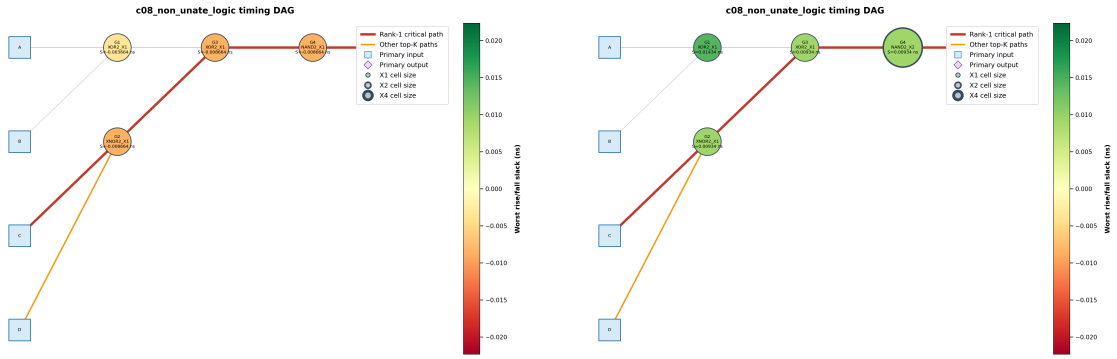



Figure B.21: `c07_parallel_paths`: optimization outcomes summary.

## B.8 c08_non_unate_logic (4 gates)




(a) Pre-optimization (b) Post-optimization

Figure B.22: `c08_non_unate_logic`: circuit diagrams, nodes colored by slack, critical path highlighted.

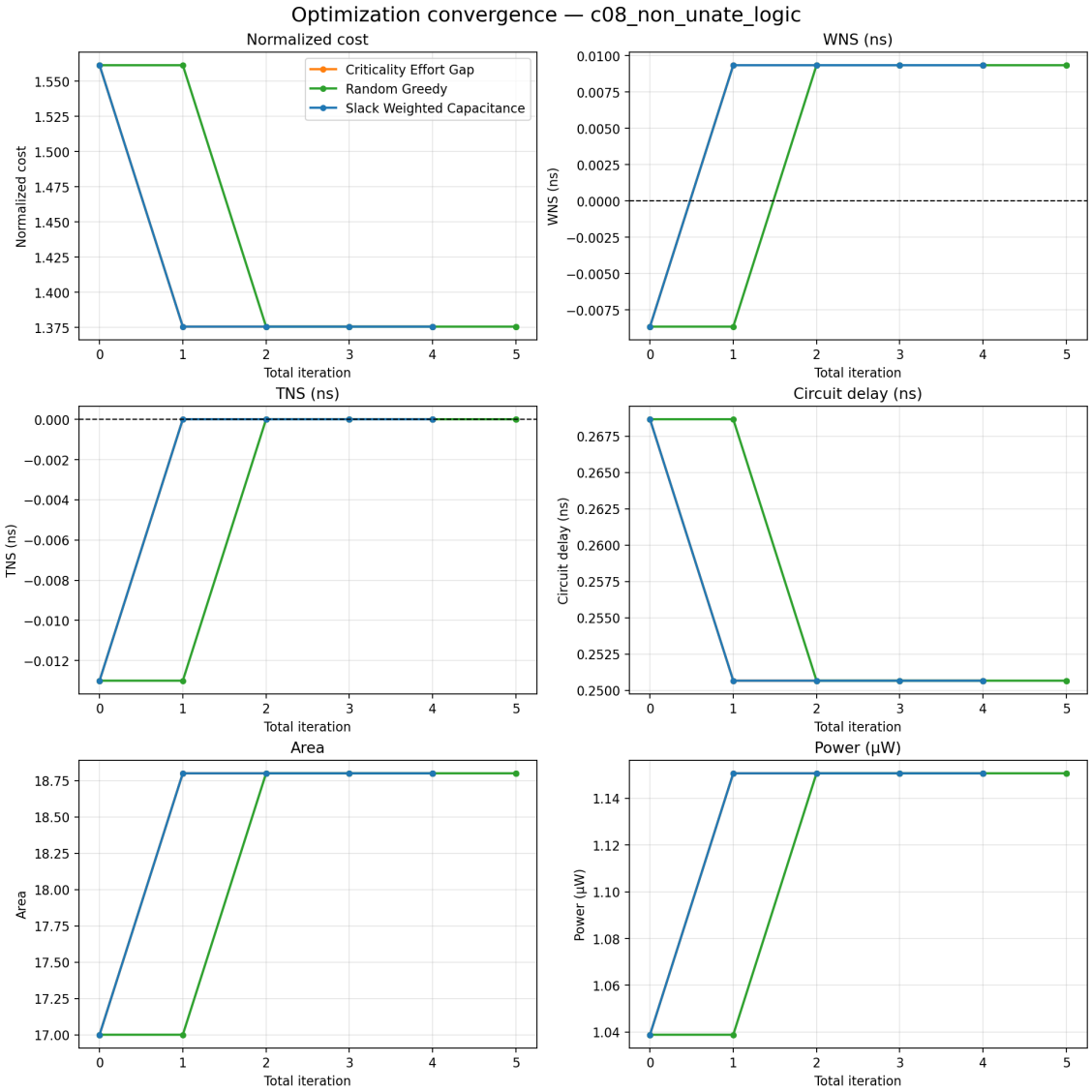



Figure B.23: `c08_non_unate_logic`: optimization convergence trace.

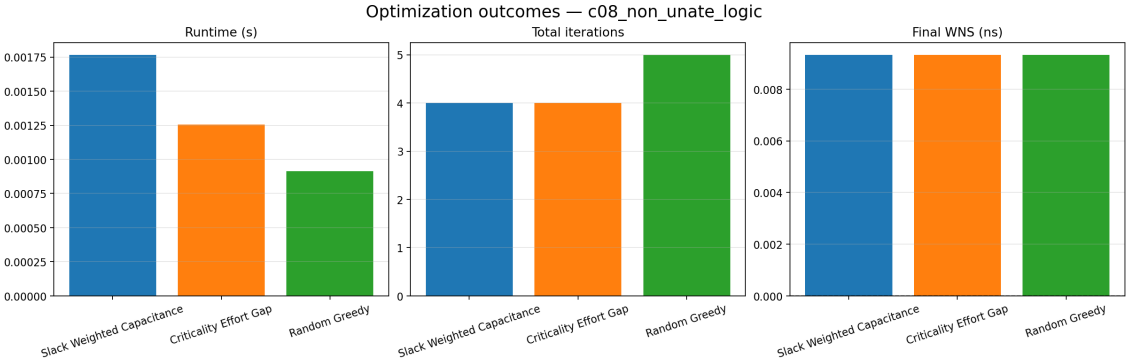
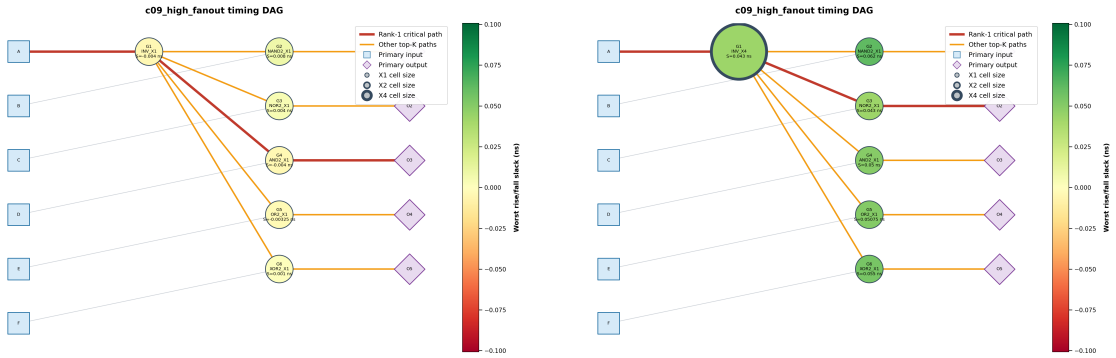



Figure B.24: `c08_non_unate_logic`: optimization outcomes summary.

## B.9 c09_high_fanout (6 gates)




(a) Pre-optimization (b) Post-optimization

Figure B.25: `c09_high_fanout`: circuit diagrams, nodes colored by slack, critical path highlighted.

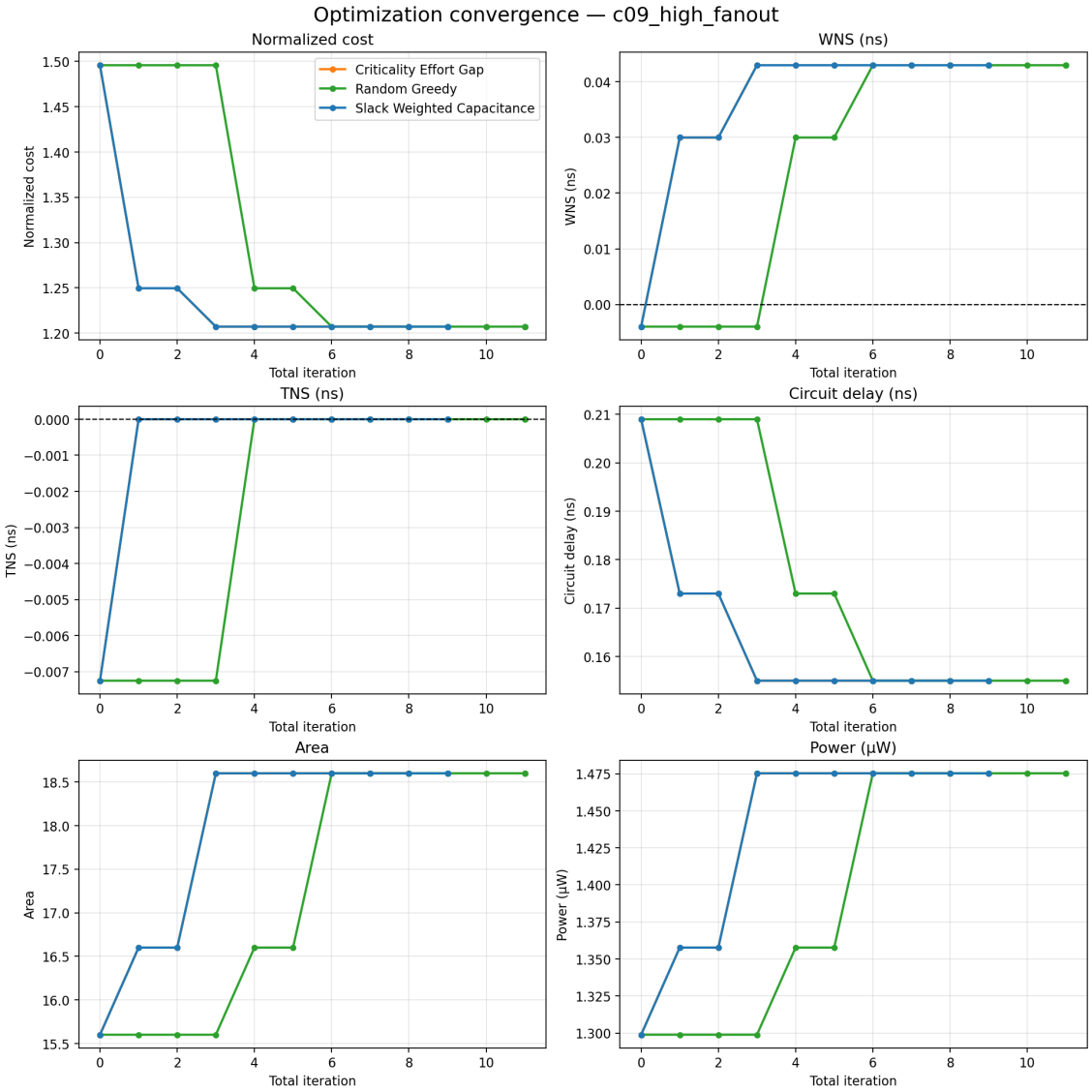



Figure B.26: `c09_high_fanout`: optimization convergence trace.

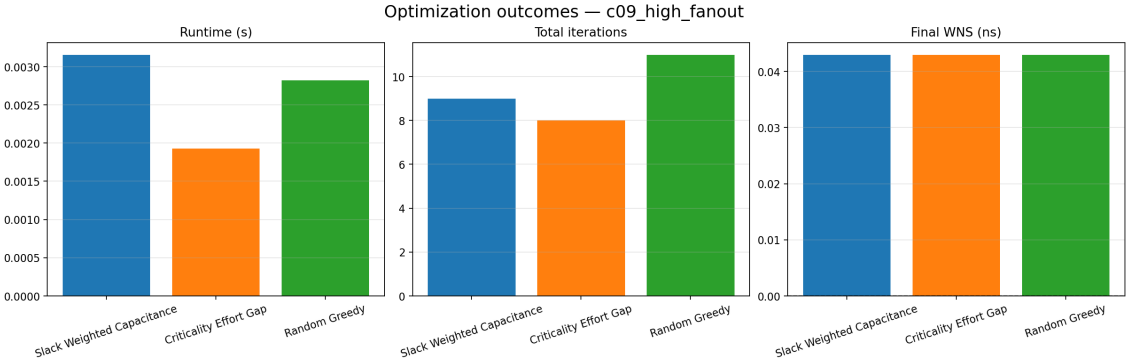
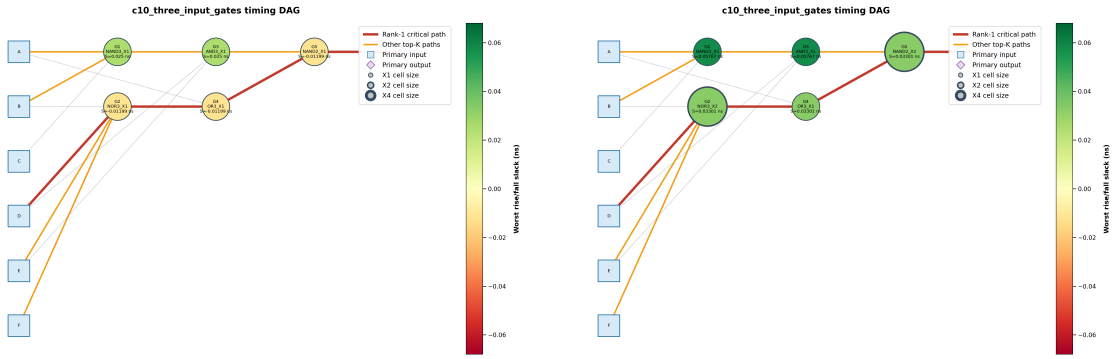



Figure B.27: `c09_high_fanout`: optimization outcomes summary.

## B.10 c10_three_input_gates (5 gates)




(a) Pre-optimization (b) Post-optimization

Figure B.28: `c10_three_input_gates`: circuit diagrams, nodes colored by slack, critical path highlighted.

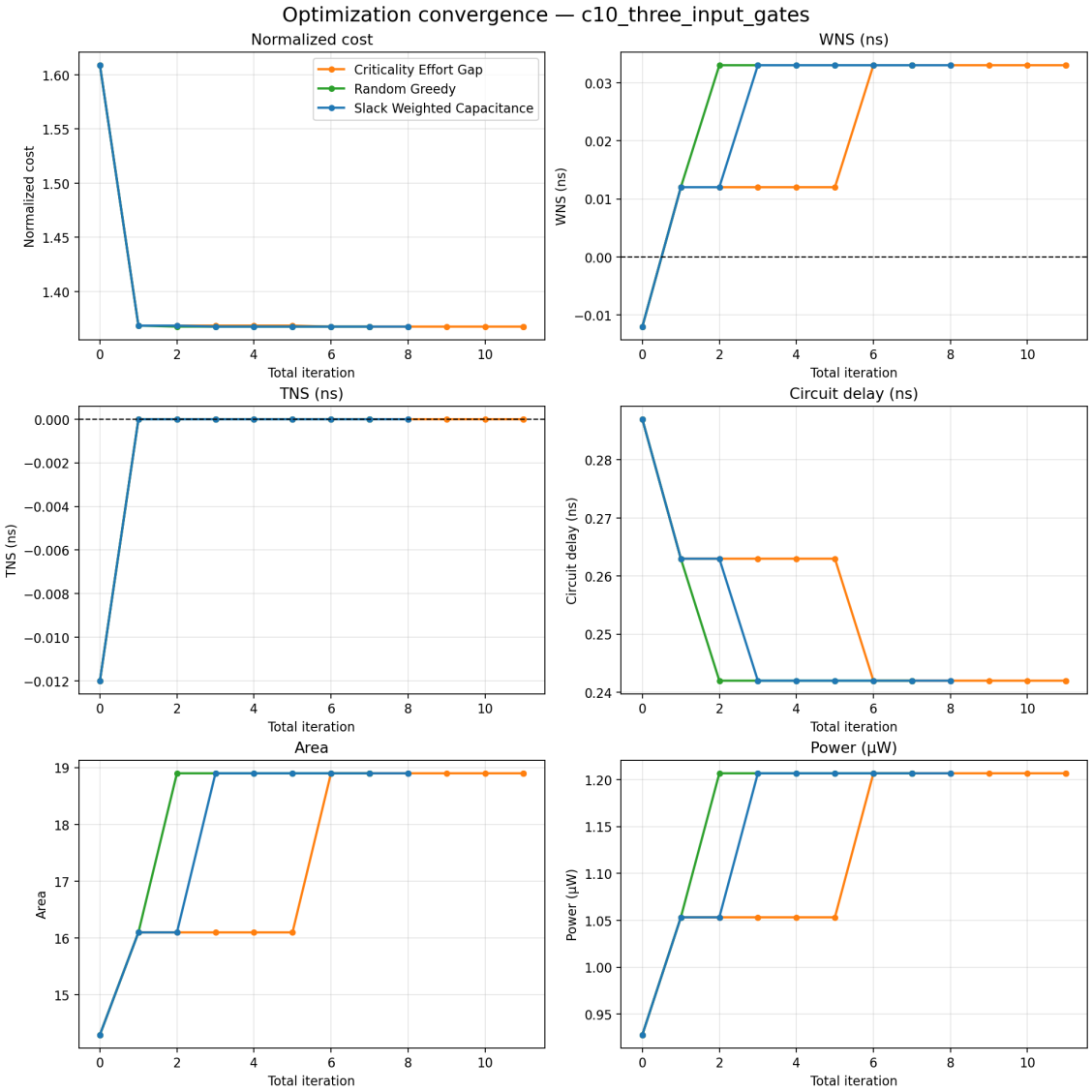



Figure B.29: `c10_three_input_gates`: optimization convergence trace.

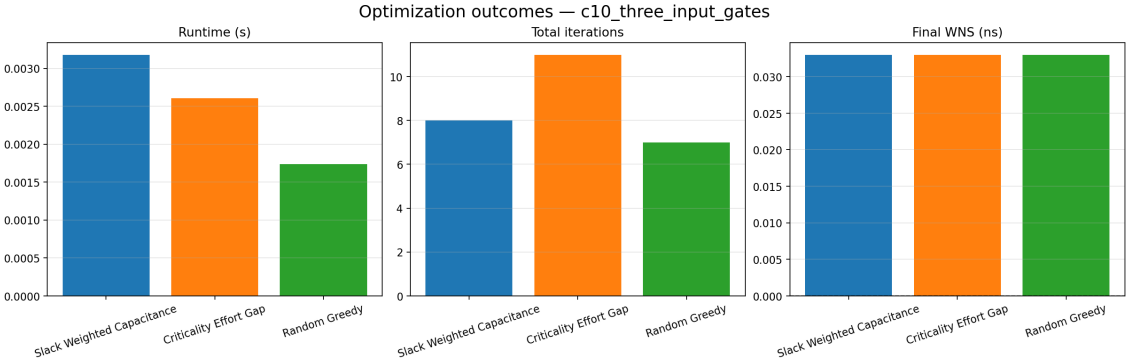
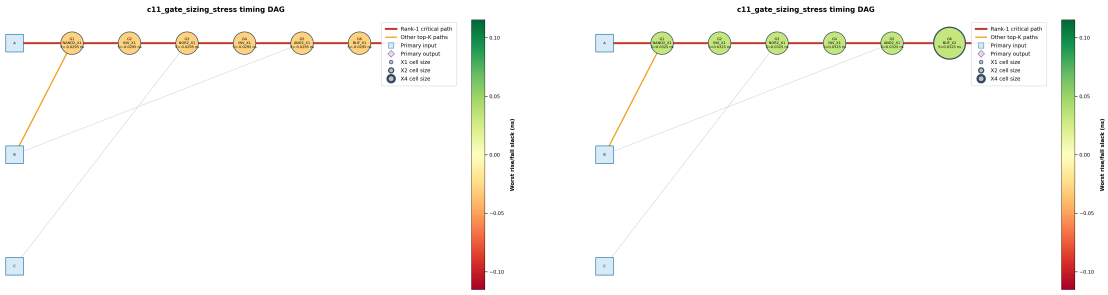



Figure B.30: `c10_three_input_gates`: optimization outcomes summary.

## B.11 c11_gate_sizing_stress (6 gates)




(a) Pre-optimization (b) Post-optimization

Figure B.31: `c11_gate_sizing_stress`: circuit diagrams, nodes colored by slack, critical path highlighted.

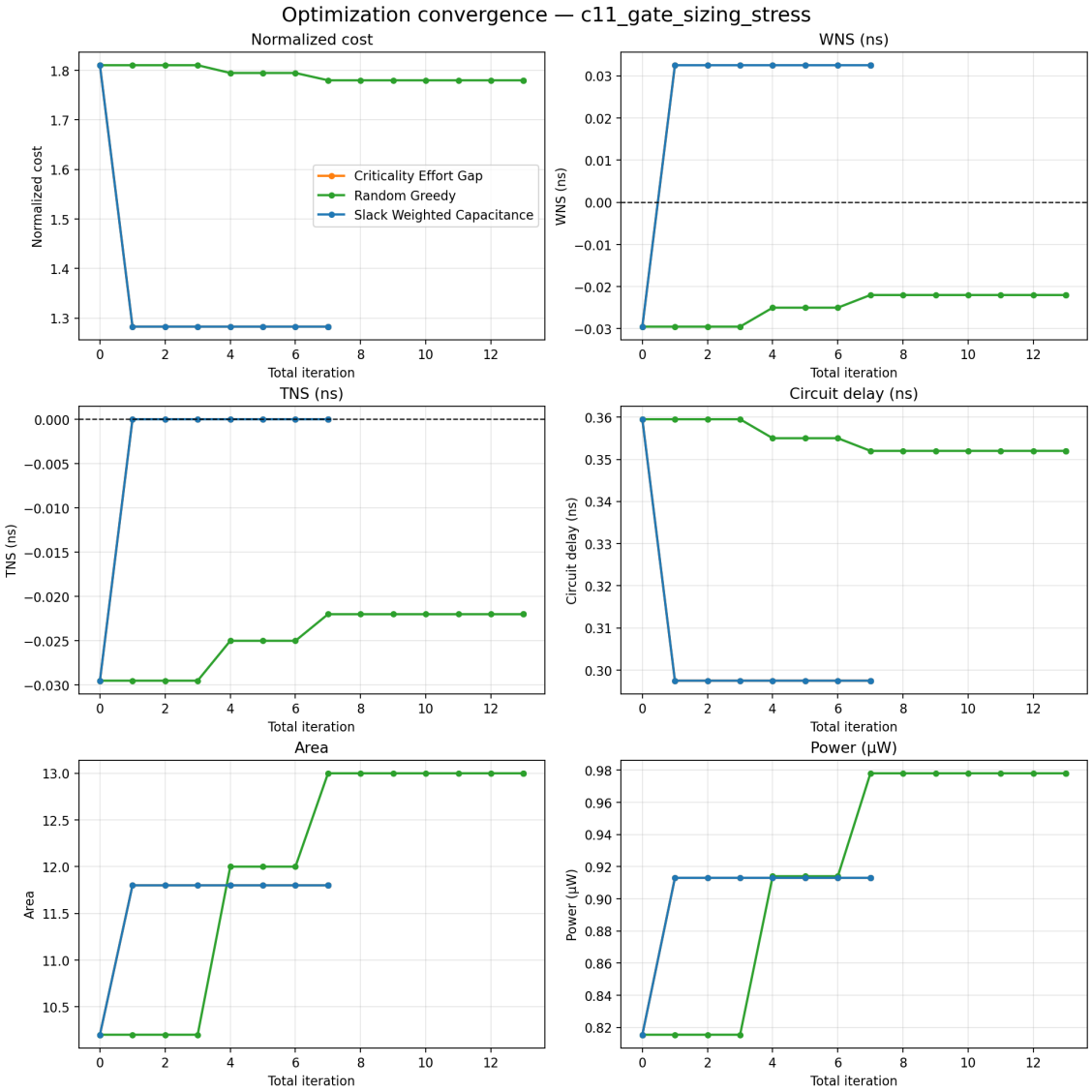



Figure B.32: `c11_gate_sizing_stress`: optimization convergence trace.

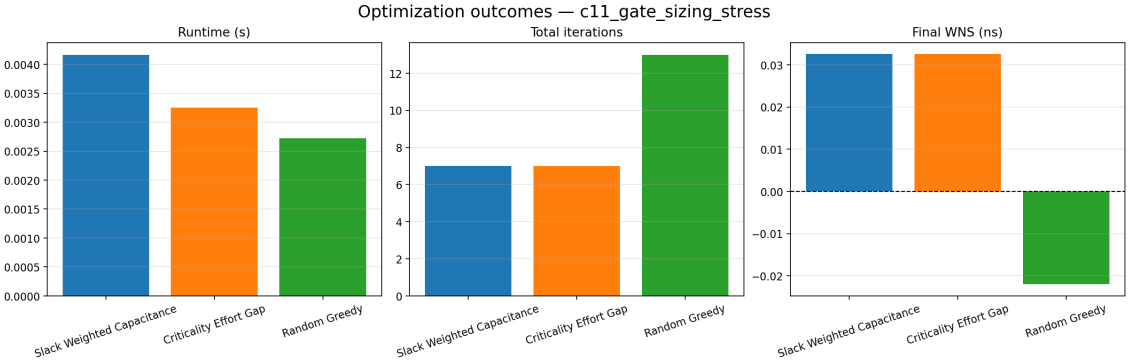
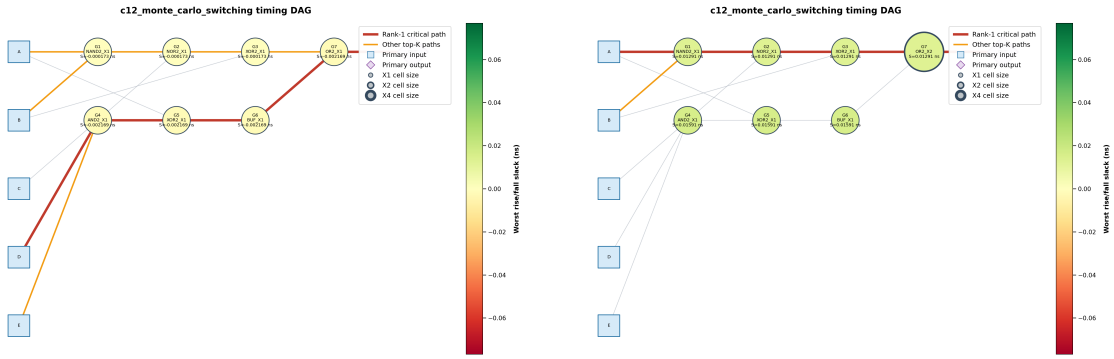



Figure B.33: `c11_gate_sizing_stress`: optimization outcomes summary.

## B.12 `c12_monte_carlo_switching` (7 gates)




(a) Pre-optimization (b) Post-optimization

Figure B.34: `c12_monte_carlo_switching`: circuit diagrams, nodes colored by slack, critical path highlighted.

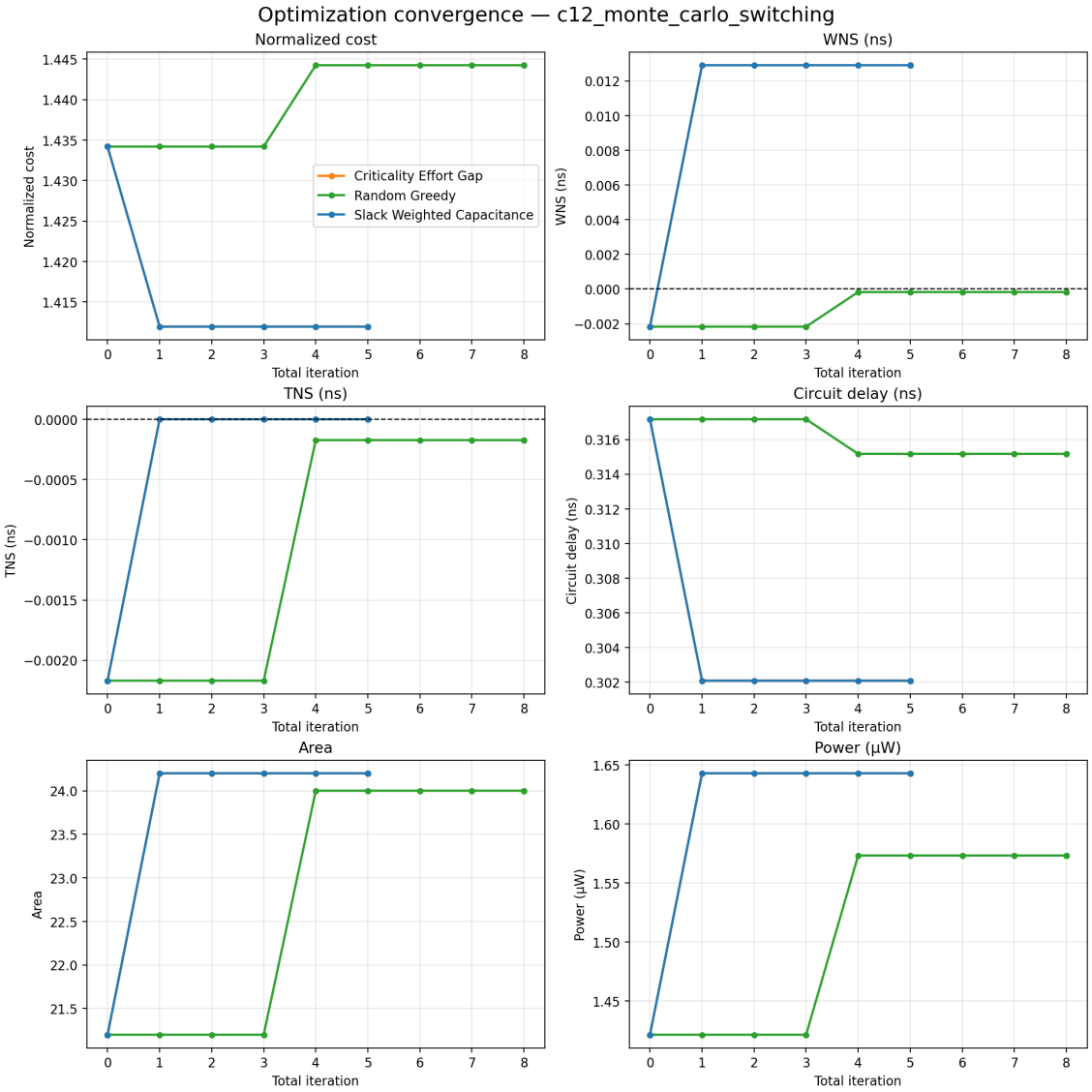



Figure B.35: `c12_monte_carlo_switching`: optimization convergence trace.

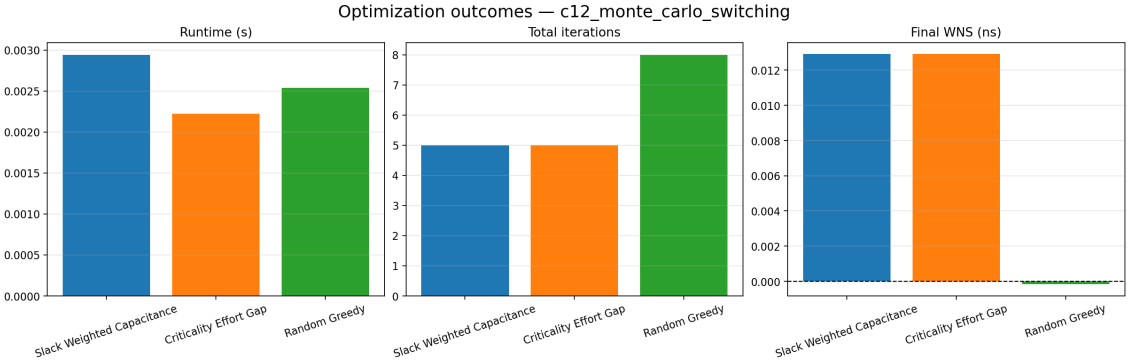
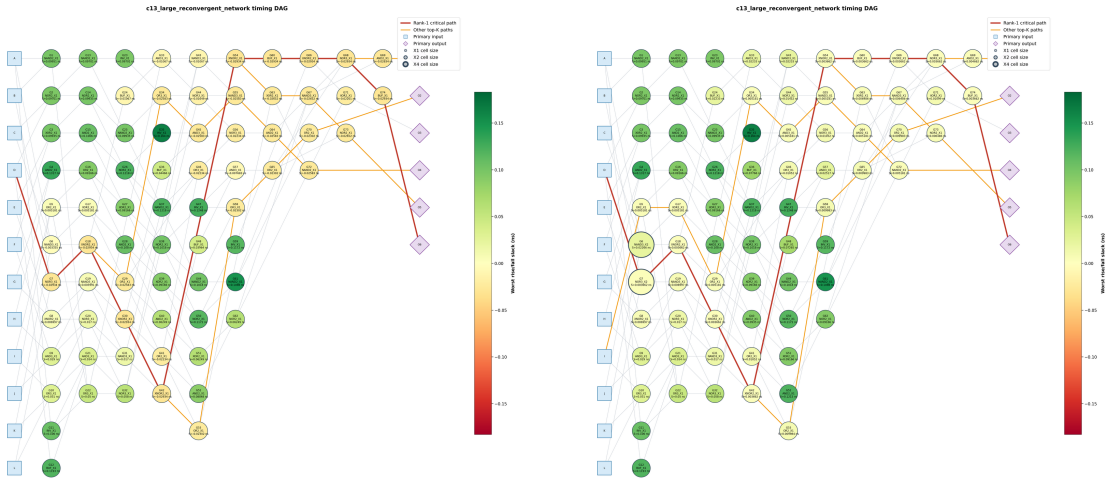



Figure B.36: `c12_monte_carlo_switching`: optimization outcomes summary.

## B.13 `c13_large_reconvergent_network` (74 gates)



(a) Pre-optimization (b) Post-optimization

Figure B.37: `c13_large_reconvergent_network`: circuit diagrams, nodes colored by slack, critical path highlighted.

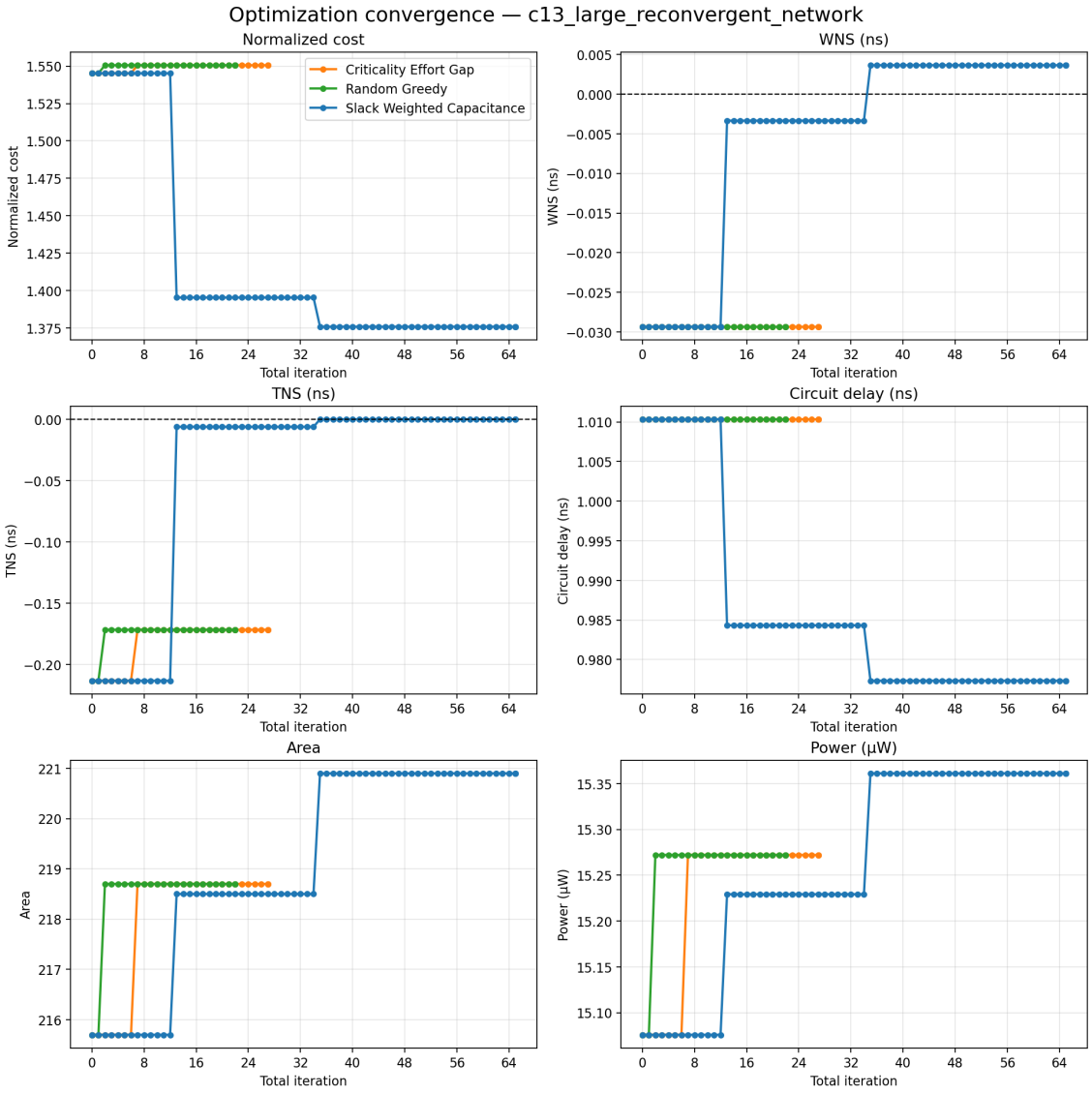



Figure B.38: `c13_large_reconvergent_network`: optimization convergence trace.

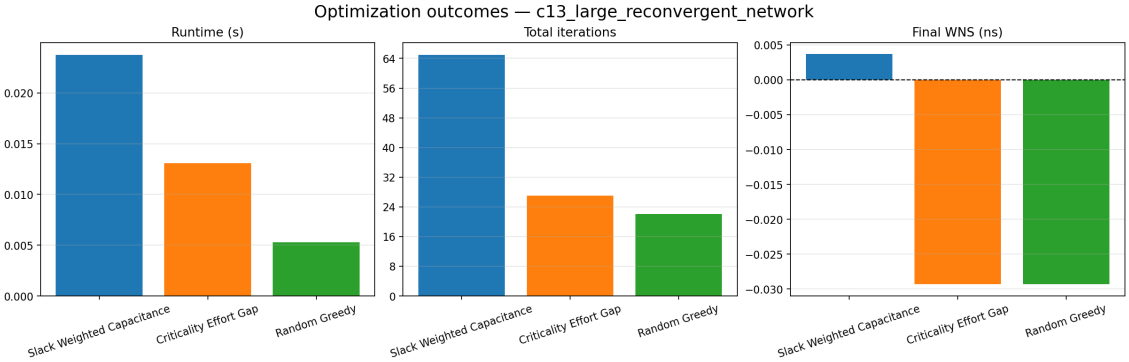
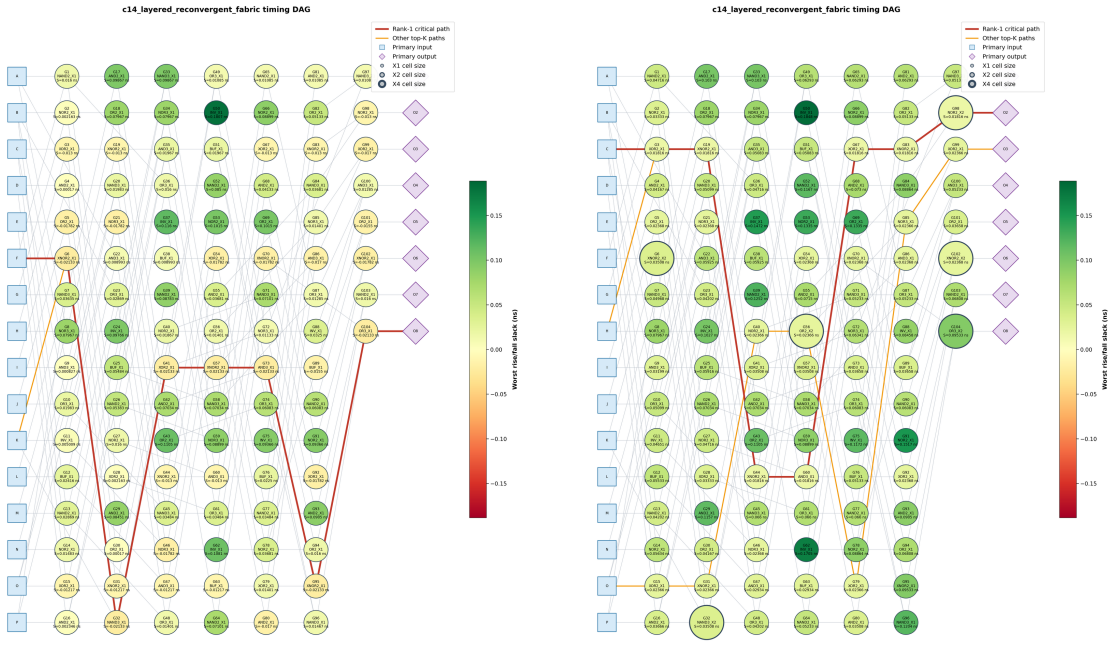



Figure B.39: `c13_large_reconvergent_network`: optimization outcomes summary.

## B.14 `c14_layered_reconvergent_fabric` (104 gates)




(a) Pre-optimization (b) Post-optimization

Figure B.40: `c14_layered_reconvergent_fabric`: circuit diagrams, nodes colored by slack, critical path highlighted.

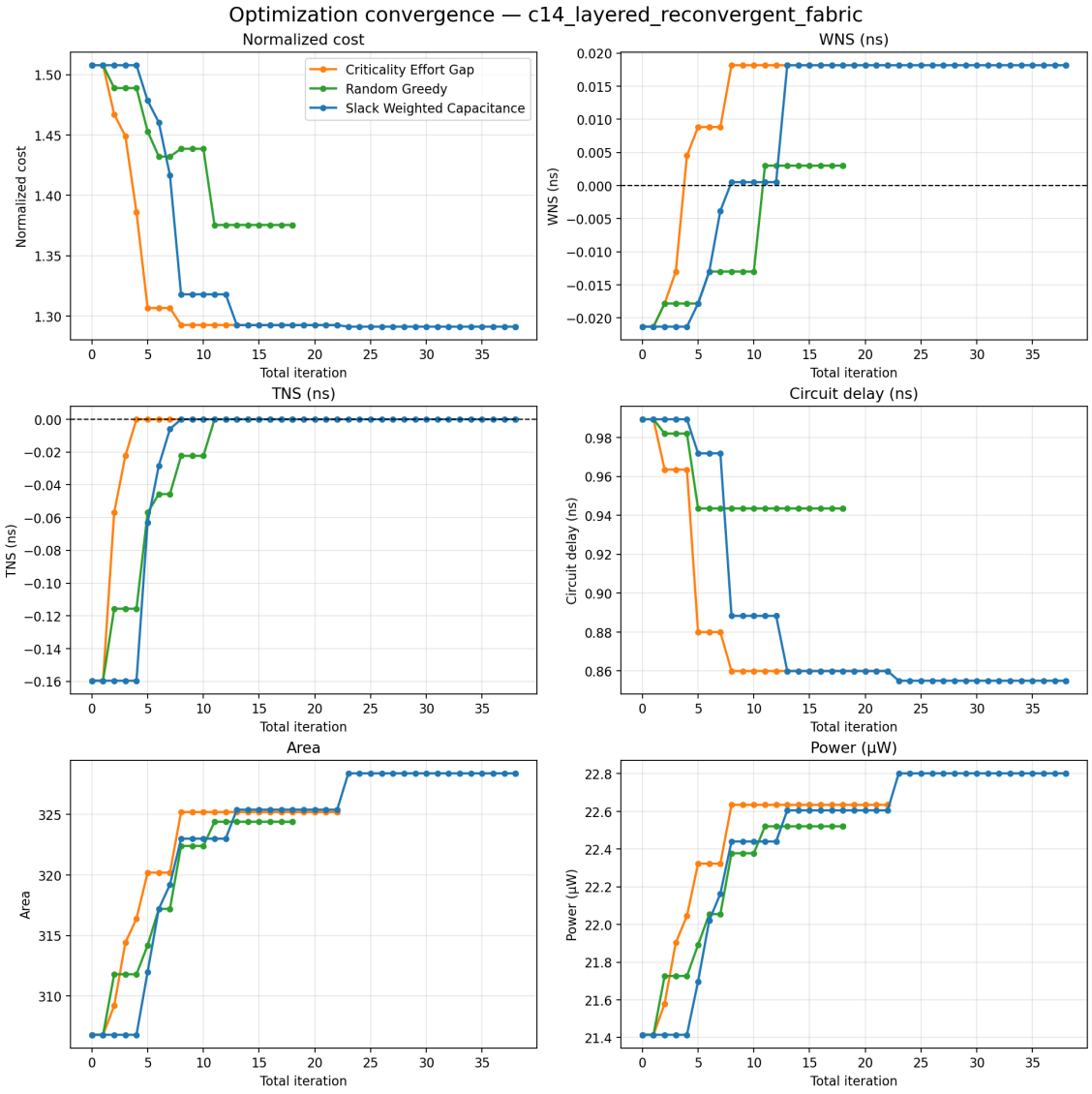



Figure B.41: `c14_layered_reconvergent_fabric`: optimization convergence trace.

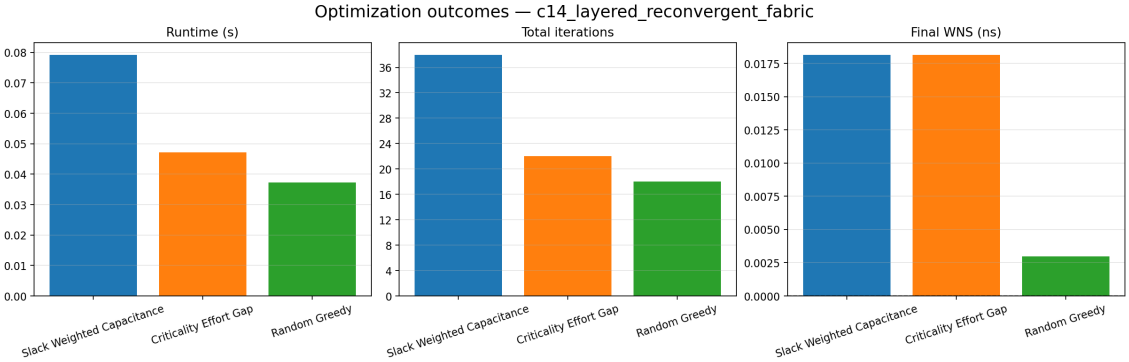
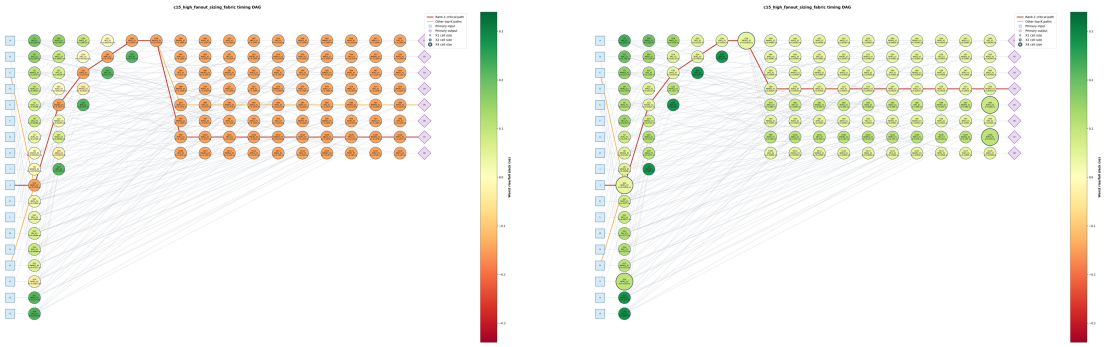



Figure B.42: `c14_layered_reconvergent_fabric`: optimization outcomes summary.

## B.15 `c15_high_fanout_sizing_fabric` (118 gates)



(a) Pre-optimization (b) Post-optimization

Figure B.43: `c15_high_fanout_sizing_fabric`: circuit diagrams, nodes colored by slack, critical path highlighted.

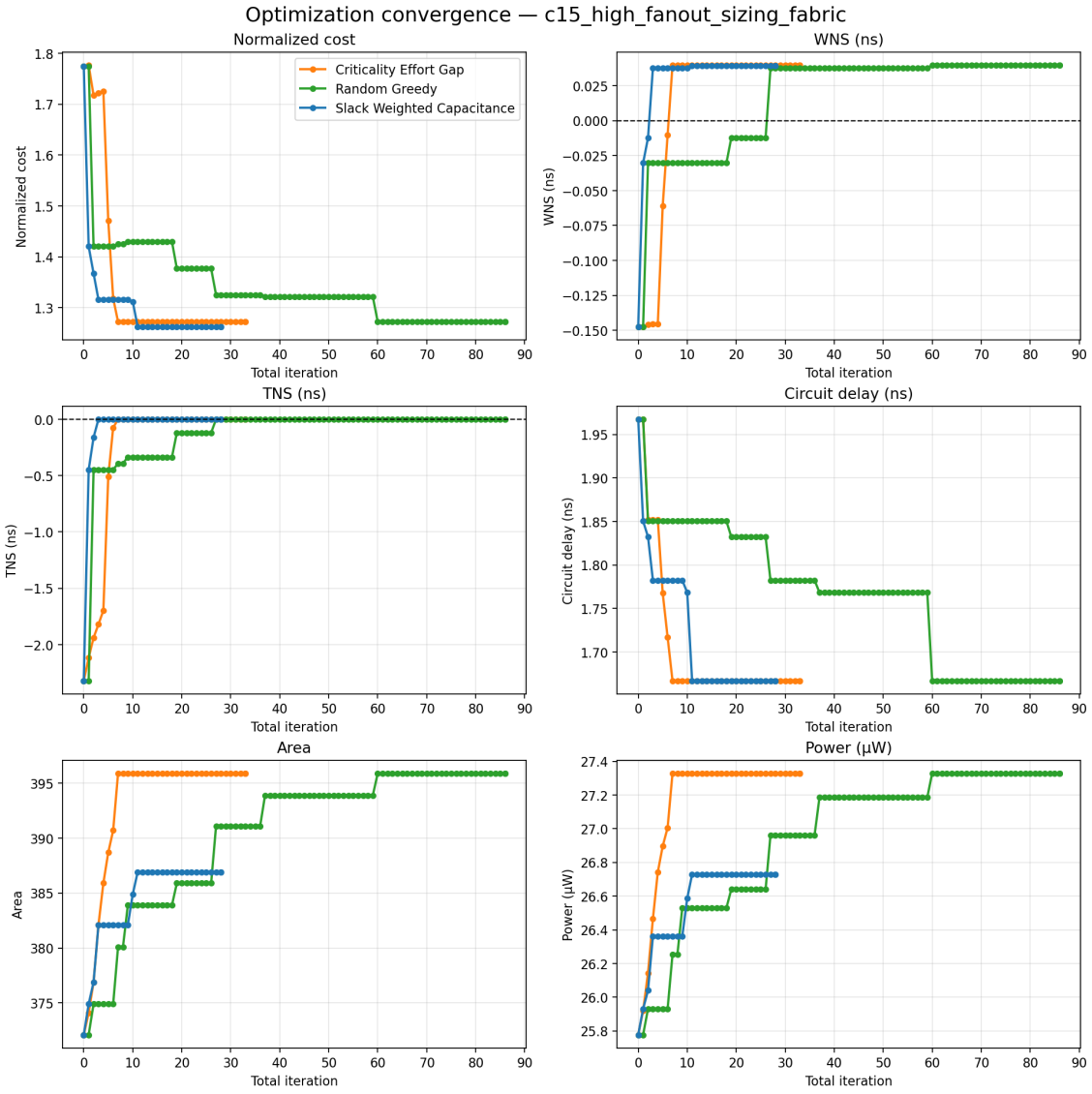



Figure B.44: `c15_high_fanout_sizing_fabric`: optimization convergence trace.

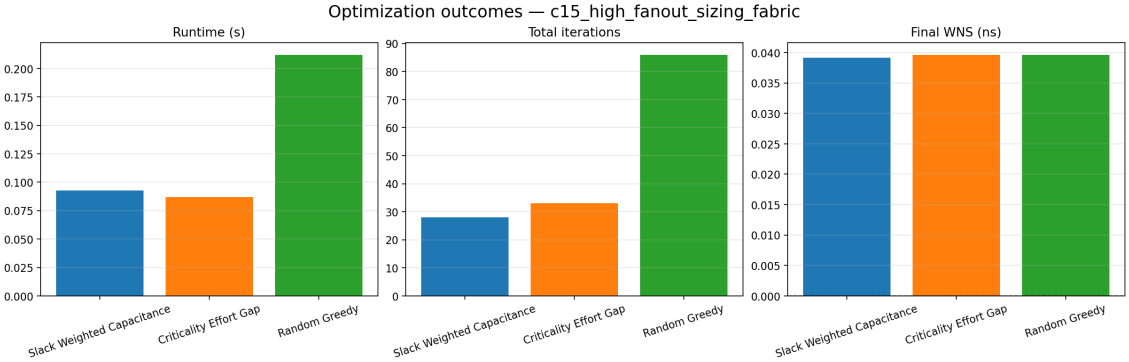



Figure B.45: `c15_high_fanout_sizing_fabric`: optimization outcomes summary.